

National University of Ho Chi Minh City
HCM University of Technology
Computer Science and Engineering Department



REPORT
SPECIALIZED PROJECT

**RESEARCH AND DEVELOPMENT A SMART
QUERY SOLUTION USING KNOWLEDGE
GRAPH AND LANGUAGE MODELS**

Major: Computer Science

Thesis Committee: HĐ 3CC Đồ án chuyên ngành

Instructor: Assoc. Prof. Thoại Nam

Secretary: Mr. Lê Đình Thuận

—o0o—

Student: Vũ Hoàng Tùng (2252886)

HỒ CHÍ MINH City, June 2026

Acknowledgement

This section is dedicated to express our sincere gratitude towards all those who have played a crucial role in the completion of the Specialized Project, also the first phase of the thesis. Your supports have been a great help for me to perform various steps from the fundamental researches and requirement specification to system design and implementation development, laying foundations for the future Capstone Project

I would like to show my foremost appreciation to my supervisor, Mr. Thoai Nam for his heartwarming support, enthusiastic guidance and constructive feedback. His invaluable expertise and advice has really strengthened my vision and understanding, thus been instrumental in shaping the direction of my work.

I am also greatly thankful to Computer Science Faculty of Department for the learning environment, the facilities, the knowledge and the opportunity to orchestrate this work that I'm proud of. The contributions of my peers and colleagues in engaging discussions and shared insights have been enriching.

To Bảo, whose knowledge and engaging discussion on system design has been a great help, I show my appreciation for his passionate and knowledgeable share and unwavering support.

To all my family and friends who offered encouragement, shared their knowledge and gave inspiration, I feel blessed that this Project and me have been supported by your love and care.

Last but not least, I acknowledge that this my work would not achieve the goal without the foundation laid by pioneering researchers and engineers, both in the field of artificial intelligence and education. The theories, methodologies, and technologies they have developed have been the bedrock upon which this project is built. The knowledge and inspiration they give me is invaluable, and I am more than honored to stand on the shoulders of giants.

This work stands firm thanks to the contribution of all those mentioned above. Your accumulated impact has been indispensable, and I cherish the roles each played in this academic endeavor.

DECLARATION OF AUTHENTICITY

I declare that this specialized project entitled "*Research and Development a smart query solution using Knowledge Graphs and Language Models.*" was entirely composed and implemented by myself under the guidance and supervision of Assoc. Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

Throughout the project, I ensure that all references and external sources utilized are appropriately cited and thoroughly documented. Although full code won't be included in report, important parts of the code are summarized in pseudo form and formally referenced to source. References are also commented in details at my source code:

Ho Chi Minh City, May 2026
The author,

Vũ Hoàng Tùng

Abstract

Current world is changing at an incredible pace with constant information and knowledge update, while many educational systems remain stagnant. The rapid evolution of Artificial Intelligence has open doors for adaptive, self-directed learning online systems, replacing Learning Management Systems that offer static contents. However, common AI techniques such as Retrieval-Augmented Generation (RAG) struggle with static content, semantic fragmentation, and a lack of profound reasoning, while LLMs have limited context window and hallucination, inconsistent output.

To address these limitations, this project introduces **SmartEdu**, a proactive, mastery-oriented educational ecosystem powered by a graph-grounded multi-agent architecture. By operating LLM on deterministic ingesting pipeline for academic materials, the system constructs an Automated Educational Knowledge Graph (EKG) that accurately maps conceptual hierarchies and prerequisite dependencies. Unstructured multimodal materials are automatically ingested via an asynchronous Producer-Consumer pipeline and processed through a novel Dual-Phase LLM extraction technique. This is reinforced by deterministic NLP filtering—including DBpedia validation and implicit clustering—to ensure strict ontological hierarchy and prerequisite acyclic constraints.

For real-time tutoring, a LangGraph-driven Teaching Assistant operates within an Adaptive Middleware harness, strictly separating reasoning contexts and action schemas. It utilizes Knowledge Graph as a wellstructured ground truth and solid reasoning frame, cooperates open-weights Small Language Models (SLMs) into workflow for localized and privacy-preserving execution, and orchestrates specialized multi-agent workflows—Retrieve, Roadmap, and Teach—to perform dynamic student mastery tracking, automated gap detection using topological algorithms, and personalized learning pathfinding, all while safeguarding student privacy and important resources through a robust Identity-Session dynamic allocation and management.

Initial evaluations of the system demonstrate successful and consistent AI operations, effectively reduce hallucinations, produce consistent results and reasoning, and maintaining structural integrity across complex academic domains. Despite current latency bottlenecks and challenges in macro-level course pathing, SmartEdu establishes a robust, highly scalable foundation for shifting the educational paradigm from linear content delivery to interactive, human-like mentorship.

The project is being actively maintained on Github: <https://github.com/ttb-hcmut/SmartEdu>

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	3
1.4	Thesis Structure	3
2	Fundamental Knowledge & Related Work	4
2.1	Fundamental Knowledge	4
2.1.1	Natural Language Processing	4
2.1.2	Large Language Models (LLMs)	6
2.1.3	Agentic Systems	8
2.1.4	System Design	12
2.1.5	Technology Stack and Frameworks	14
2.1.6	Database System Design & Management	17
2.1.7	Deployment and Containerization	21
2.1.8	Security: OAuth 2.0 Authorization	22
2.2	Related Work	23
2.2.1	Document-Knowledge Modeling	23
2.2.2	Generative AI in Interactive Teaching	25
2.2.3	Educational Intelligent System	26
2.2.4	Conclusion and Research Gaps	27
3	Requirements Specifications	29
3.1	Functional Requirements	29
3.1.1	Knowledge Modelling and Management	29
3.1.2	Intelligent Teaching Assistance	30
3.1.3	Student Interaction and System Navigation	30
3.2	Non-Functional Requirements	31
3.3	Use Cases	33
3.3.1	Use case 1: Register Account	33
3.3.2	Use case 2: Login/Logout	34
3.3.3	Use case 3: Manage Learning Session	35
3.3.4	Use case 4: Ingest Course Material	36
3.3.5	Use case 5: Chat with TA Agent	37

4	Proposed Methodology and Paradigm	38
4.1	Overview	38
4.2	Knowledge Modelling: Educational Graph	38
4.2.1	Graph Definition	39
4.2.1.1	Formal Property and Topological Definition	39
4.2.1.2	Ontological Hierarchy formulation	39
4.2.1.3	Edge Semantics and Constraints	40
4.2.2	Graph Construction	40
4.2.2.1	Input Ingestion	41
4.2.2.2	Dual Phase LLM Extraction	41
4.2.2.3	NLP-Based Concept Filtering and Verification	43
4.3	Teaching Assistance	45
4.3.1	Agent Harnessing	45
4.3.1.1	Tool Abstraction Design	45
4.3.1.2	State—based Harnessing	46
4.3.1.3	Agentic Full Design	47
4.3.2	Agentic Workflow	49
4.3.2.1	Intent Router	49
4.3.2.2	Workflow: Roadmap	49
4.3.2.3	Workflow: Retrieve	51
4.3.2.4	Workflow: Teach	52
5	System design and implementation	54
5.1	System Architecture	55
5.1.1	Overall Architecture and Module Responsibilities	55
5.1.2	Technical Setup: Docker Infrastructure	56
5.1.3	Design Paradigm	57
5.1.3.1	Singleton	57
5.1.3.2	Dependency Injection	58
5.1.3.3	MVC-Aligned Module Structure	59
5.2	Core Module	59
5.2.1	LLM Engine	60
5.2.2	Configuration Management	61
5.3	Database Design	63
5.3.1	Polyglot Persistence Design	63
5.3.1.1	Neo4j: Knowledge and Learning Graphs	63
5.3.1.2	Milvus: Vector Embedding Index	64
5.3.1.3	MongoDB: Student State and Session Context	65

5.3.1.4	SQLite: Authentication Store	66
5.3.2	Cross-Database Reference and Consistency	67
5.4	Student Module	68
5.4.1	Authentication: OAuth2 + JWT	68
5.4.2	Session Lifecycle and the Identity Firewall	68
5.5	Knowledge Module	69
5.5.1	Technical Setup	69
5.5.2	Ingestion Workflow	69
5.5.3	Graph Schema at Database Level	71
5.6	Teaching Assistant Module	71
5.6.1	Technical Setup	72
5.6.2	LangGraph Workflow Design	72
5.6.3	Tracing and Observability	74
6	Experiment	75
6.1	Application Deployment	75
6.1.1	Database Setup	75
6.1.2	FastAPI App Deployment	76
6.2	Knowledge Module	76
6.2.1	Service Endpoints	76
6.2.2	Runtime	77
6.3	Student Module	78
6.3.1	Service Endpoints	78
6.3.2	Runtime	79
6.4	Teaching Module	80
6.4.1	Service Endpoints	80
6.4.2	Runtime	80
7	Conclusion	82
7.1	Achieved Results	82
7.2	System Limitations	82
7.3	Future Plan	83

List of Figures

2.1	Transformer turn unstructured raw text into embeddings—numeric representation that hypothetically capture the core semantic	5
2.2	Comparison of LLM Deployment Strategies: Traditional approaches (a) Direct cloud processing, (b) Local-only processing, and (c) Local-Cloud framework balance performance security without leaking IP (proposed by [23])	7
2.3	Basic Single Agent diagram	9
2.4	Comparison of Microservice and Monolithic architecture	13
2.5	KGGen with multiturn LLM reasoning, grouping generation	23
2.6	The end-to-end EduKG Pipeline framework for educational knowledge construction and retrieval.	24
2.7	Hybrid architecture of GraphRAG allow minimal cost and guarantee alignment of generated to original context	24
2.8	Dynamic Knowledge Graph (DKG) approach tracking knowledge evolution over time.	25
4.1	High level visualization of how the system should work	38
4.2	Overlapping window segmentation. Adjacent batches share boundary pages. . .	41
4.3	Dual Phase Extraction	42
4.4	How agents’s context and behaviour are constructed in a workflow node on a high level	46
4.5	Agent Harness architecture demonstrating the Adaptive Harnessing based on Middleware.	47
4.6	Top-level workflow routing based on user intents and context.	49
4.7	Roadmap sub-graph.	50
4.8	Retrieval sub-graph	51
4.9	Teaching sub-graph flow: content lookup, RAG fallback, and adaptive topic progression.	52
4.10	ReAct loop in Teach Lookup.	53
5.1	System general pipeline flow. Arrows show read - write operation direction . .	54
5.2	Modular Architecture Overview of SmartEdu	56
5.3	Ownership graph in lifespan. Shared resources are injected into domain modules	58
5.4	Neo4j dual-database schema. Knowledge and learning graphs separated.	64

5.5	Login protocol with Identity Firewall (represented by the red line).	68
5.6	Asynchronous Ingestion Pipeline integrating Queues, Parallel Workers, and Storage	70
5.7	Practice of Langchain Middleware Integration, inspired by How LangChain 1.0 Simplifies Building Production-Ready LLM Applications	72
5.8	Standard operation of a LangGraph Node	73
5.9	Trace data hierarchy. Three nested observation levels.	74
6.1	Docker-base infrastructure containerization and deployment	75
6.2	Neo4j Monitor Plugins	76
6.3	DBgate monitoring both MongoDB and SQLite	76
6.4	Database Admin Monitoring and Visualizing	76
6.5	FastAPI successful lifecycle start	76
6.6	Tracking Log of Asynchronous Ingestion	77
6.7	Extracted Concept Nodes	77
6.8	Completed Knowledge Graph demonstrating no concept fragmentation	78
6.9	DBpedia accepting unusual or unnormalized node names	78
6.10	Status 409 indicating a pre-existing student ID entry in the database. A new Session ID and access credentials are concurrently created.	79
6.11	Client-side interface displaying the agentic interaction flow	81
6.12	2 snippet of Logged Agent Workflow result	81

List of Tables

2.1	Comparison of LLM Orchestration Frameworks	12
2.2	Polyglot Persistence Architecture: Strategic allocation of storage engines. . . .	17
2.3	Comparative analysis of representative educational intelligent systems based on key AI and knowledge features.	27
3.1	Use case 1 Overview: Register Account	33
3.2	Use case 2 Overview: Login/Logout	34
3.3	Use case 3 Overview: Manage Learning Session	35
3.4	Use case 4 Overview: Ingest Course Material	36
3.5	Use case 5 Overview: Chat with TA Agent	37
4.1	Summary of tools available to the Teaching Assistant agent.	48
5.1	System Module Summary	56
5.2	Docker Compose service summary.	57
5.3	LLM Profile configurations and their engineering rationale.	61
5.4	Configuration dataclasses.	62
5.5	Polyglot persistence strategy overview.	63
5.6	Milvus collection schema.	64
5.7	HNSW index configuration.	65
5.8	MongoDB student document field schema.	65
5.9	MinIO bucket directory structure.	66
5.10	SQLite students table schema.	66
5.11	Write ownership per database.	67
5.12	Knowledge Module core libraries.	69
5.13	Entity node properties in Neo4j.	71
5.14	TA Module core libraries.	72

Chapter 1

Introduction

1.1 Motivation

The rapid evolution of Artificial Intelligence (AI) has fundamentally change the global socio-economic landscape. To adapt to the new era, selfstudying and learning method are becoming more and more important. Traditional education philosophy, such as "practice makes perfect," has no longer been the absolute true; when AI can automate execution, the value of learning moves from procedural fluency and labour expertise to conceptual mastery and strategic analysis. However, current education system fail to keep pace with this evolution:

- **Stagnancy of Content:** Legacy Learning Management Systems (LMS) treat knowledge as static artifacts (textbooks, slides). These materials fail to adapt to the varying cognitive baselines of individual students.
- **The Guidance Deficit:** Without a dynamic roadmap, students navigate a "sea of information" where external resources are either too granular (leading to information overload) or too condensed (creating knowledge gaps), with little profound ground truth.
- **The Lack of Interaction:** Current systems lack a feedback loop. Textbooks exceed the average human attention span, while slides provide too condensed context insufficient for novices, leaving no room for real-time, human-like inquiry or mentorship.
- **Outdated Knowledge:** Educational materials and methods cannot catch up with the change of technology. Internet are an invaluable resource, however information is chaotic, unstructured and content are confusing to many new learners.

While Large Language Models (LLMs) offer a glimpse into automated resource synthesis and knowledge base construction, significant technical bottleneck prevent their reliable application in complex domains:

- **Semantic Fragmentation in RAG:** Traditional Retrieval-Augmented Generation (RAG) frameworks rely on discrete data chunking. This fragmentation disrupts semantic integrity, leading to "contextual fragmentation" where the model captures isolated facts but fails to comprehend the overall hierarchy of the domain.
- **Contextual Saturation vs. Precision:** Expanding the context window of LLMs often introduces diminishing returns. The injection of unrefined data frequently leads to "contextual overhead" and "noise," where the model struggles to distinguish fundamental principles from peripheral details.

- **Relational Limitations of Vector Space:** Relying exclusively on vector embeddings (typically ≈ 1024 dimensions) is insufficient for mapping the non-linear, multi-faceted dependencies inherent in academic taxonomies. Similarity-based retrieval cannot substitute for the deterministic logic required to understand prerequisite relationships.
- **The Reasoning Gap in Academic task:** Instruction is a sophisticated, non-linear reasoning task. Current AI systems are largely reactive—proficient in text generation but deficient in long-term goal tracking and proactive guidance. This results in a susceptibility to hallucinations and an inability to execute the "evaluate-and-guide" workflow essential for effective pedagogy.

1.2 Goals

To address the current limitations, this project aims to design an Intelligent System far beyond serving contents or a simple QA chatbot: a proactive, mastery-oriented learning platform tracking and adapting to student progress by developing SmartEdu—an intelligent, graph-grounded educational ecosystem.. This thesis determined for such a system which capabilities include:

- **Adaptive Mentorship:** Shifting the educational paradigm from static, linear content delivery to interactive, user friendly workflow that dynamically reform itself based on student's personality and action.
- **Data Sovereignty and Localized Execution:** Establishing a sustainable deployment framework that leverages open-weights Small Language Models (SLMs) and local inference engines, ensuring complete data privacy for student learning behaviors while eliminating recurring API token costs.
- **Consistent Educational Contents:** Intelligent contents adaptation and generation suffers from hallucinations and key info lost. Mitigating the hallucination risks inherent in generative AI by anchoring the multi-agent reasoning framework within a structured knowledge network to enforce academic truthfulness.

Targeted at University students, this educational platform facilitates student self learning, the most important skills in University. To demonstrate practical utility and fulfill the discussed goals of an actionable software solution, the application identifies the following key features:

- **Automated Educational Knowledge Graph (EKG) Engine:** The system automatically ingests unstructured academic materials (raw PDFs, textbooks, and lecture videos) and constructs a structured knowledge graph. It automatically extracts conceptual hierarchies, maps prerequisite dependencies, and group related topics into organized clusters.
- **Dynamic Student Mastery Mapping:** The application implements a continuous storing-and-tracking engine that tracks the student's real-time comprehension levels, storing historical interaction and cognitive progress securely.
- **Automated Gap Detection and Pathfinding:** Leveraging graph-traversal and heuristic algorithms, the system choose a good studying path based on the student's current mastery and state against the EKG backbone. When a target topic is selected, the application automatically identifies missing prerequisite knowledge and dynamically generates a personalized, non-linear learning path.
- **Socratic Dialogue and Document Verification:** The user interface provides an interactive teaching assistant that engages students using powerfull reasoning workflow.

1.3 Scope

To fulfill these goals, the research focuses on the following researching scope:

- **Multimodal Information Extraction:** Knowledge of courses from multiple sources (pdf, videos, textbooks) must be well extracted to consolidate them into a centralized data repository.
- **Domain Specificity:** The knowledge modeling and extraction processes are limited to well-defined academic disciplines with clear hierarchical structures. Unstructured or subjective domains that lack clear prerequisite dependencies are outside the scope of this research.
- **Pedagogical Boundaries:** The conversational assistant is strictly restricted to goal-oriented educational support, specifically for assessing student understanding and explaining course content. Open-domain dialogue, creative writing, and casual chitchat are intentionally disabled.
- **Infrastructure:** The system optimizes execution using open-weights models to maintain strict data privacy.
- **Target User Environment:** The system is modeled and evaluated for a single-institution environment, focusing on the interactions between individual learners and a localized knowledge base, rather than a multi-tenant, global-scale educational network.

1.4 Thesis Structure

There are 7 chapters in this project's report:

- Chapter 1 introduces the problems, proposes the system's goals, scopes and thesis structure
- Chapter 2 provides fundamental knowledge for project understand and deep literature review of relevant prior researches covering topic, as well as preview of open-source application.
- Chapter 3 Requirement Specifications
- Chapter 4 propose methodologies: we discuss what inferred from previous work and what ways to solve the specific usecase of ours
- Chapter 5 details the system including: System architecture and its components, settings in each components, and finally, their implementation explanation that fully illustrate how do the system put proposed methodology to practice
- Chapter 6 analyzes the system outcomes from components to final output as a whole
- Chapter 7 is the conclusion to the result: what the system achieved, its limitation and future improvements for the final Capstone Project

Chapter 2

Fundamental Knowledge & Related Work

2.1 Fundamental Knowledge

This section discusses the fundamental techniques and applications foundational to the construction of the proposed system, categorized into 3 pillars:

- **Natural Language Processing (NLP):** The foundational process of transforming vast volumes of raw, unstructured textual input into logical, structured data formats that machine can understand. This is fundamental for document processing, reshaping all subsequent cognitive actions.
- **Large Language Models (LLMs):** The core reasoning and action components of the system, responsible for interpreting user queries, orchestrating tools, and generating responses. This pillar represents the "intelligence" of the system, enabling it to perform complex tasks and adapt to user needs.
- **Agentic Orchestration:** The management layer responsible for coordinating specialized AI agents to execute complex, non-linear workflows.
- **System Design:** The architectural blueprint that ensures scalability and robustness, utilizing clean design patterns tailored for high-stakes educational environments.

2.1.1 Natural Language Processing

Natural Language Processing (NLP) is a crucial field of study aiming to process unstructured data from documents to a structured, semantically rich representation. This process is designed to harness the full potential of complex entity-relationship paradigms, moving from raw text to machine-inferable logic. As outlined in the core objectives of language understanding, the goal is to put information into a semantically precise form that allows for further inferences.

The standard pipeline for Natural Language Processing follows the sequence:

Information Extraction (IE) → Knowledge Modeling → Information Retrieval

Information Extraction (IE)

Information Extraction is the task of identifying and understanding limited, relevant portions of a text to produce a structured representation [7]. Its goal is to filter out linguistic noise and isolate the atomic units of knowledge. Traditional algorithms aim to extract keywords, following the 2 methods:

- **Keyword and Feature Extraction:** Utilizing dependency parsing and statistical methods (e.g., TF-IDF) to identify high-signal terms based on syntactic importance.
- **Named Entity Recognition (NER):** A critical sub-task used to find and classify concepts or entities (e.g., specific academic theories, formulas, or historical figures) within a text database [7]. This identifies the "nodes" of our future backbone.

To capture complex entities and relationships that rigid rule-based systems miss, the process relies on probabilistic modeling. The foundation of this approach is Tokenization, which breaks down raw text into atomic units (tokens). Statistical algorithms, such as Conditional Random Fields (CRF) or Hidden Markov Models (HMM), are then applied to these tokens to calculate the probability distribution of various interpretations. This allows the system to resolve linguistic ambiguities and predict the most likely semantic structures or labels for sequences of text.

While traditional probabilistic models logically and practically work in many usecases, they often struggle with sparse data and lack deep semantic understanding. Deep Learning represents a significant advancement by shifting the target towards generating Embeddings. Instead of merely calculating discrete probabilities, Deep Learning architectures map tokens into a continuous, high-dimensional vector space. In this embedding space, geometrically close vectors represent semantically related concepts, allowing the system to grasp the underlying meaning, context, and nuance of the information rather than relying solely on surface-level statistics.

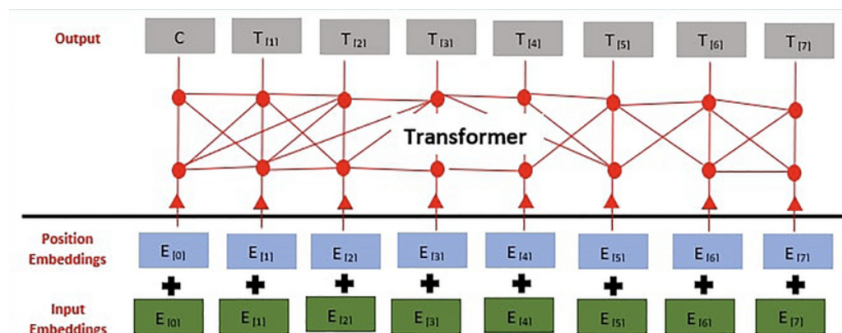


Figure 2.1: Transformer turn unstructured raw text into embeddings—numeric representation that hypothetically capture the core semantic

The Transformer architecture was introduced, being SOTAs as well as standard method nowadays. It fundamentally change the landscape of NLP and enable modern Large Language Models (LLMs). The core innovation of the Transformer is the Attention Mechanism (specifically, Self-Attention). Instead of processing tokens sequentially, Transformers process the entire sequence simultaneously. The Attention Mechanism calculates a weighted importance score for every token relative to every other token in the sequence, regardless of their distance. This allows the model to instantly capture global context and long-range dependencies, making it the most powerful method for extracting deep, complex information from unstructured texts.

Knowledge Modeling

Often overlooked in standard RAG systems, Knowledge Modeling is the construction stage where extracted entities are organized into a coherent structure that shapes how the system perceives and reasons.

- **Semantic Parsing and Logical Form:** Mapping extracted relations into a semantically precise form, such as First-Order Predicate Calculus (FOPC) or Knowledge Graphs (KG). This creates a deterministic "Backbone Graph" of the curriculum.

- **Embedding Space Topology:** Modeling knowledge within a continuous vector space where semantic similarity is represented by geometric proximity. This allows the system to handle concepts that are conceptually related but lexically distinct.

Information Retrieval (IR)

Information Retrieval is the active execution stage based on the result of construction phases IE and Knowledge Modeling, where the goal is to identify documents or data points relevant to a specific information need from a large document set [7].

- **Lexical and Keyword Matching:** Corresponding to traditional IE, this method uses exact term overlaps to find specific references or symbolic links within the knowledge base.
- **Vector Retrieval:** the most common modern approach, utilizing the embedding space to retrieve context based on semantic similarity, typically calculated via Cosine Similarity:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (2.1)$$

- **The GraphRAG Paradigm:** A new powerful emerging method, where retrieval navigates the entity-relationship graph established during the modeling phase. Unlike discrete chunk retrieval, GraphRAG performs "multi-hop" reasoning to connect disparate concepts, enabling the system to resolve complex queries that require a global understanding of the educational roadmap.

2.1.2 Large Language Models (LLMs)

Large Language Models (LLMs) are deep learning-based probabilistic models trained on massive datasets to understand and generate human-like text. Mathematically, an LLM performs the task of next-token prediction by modeling the conditional probability distribution of a token w_t given a sequence of preceding tokens $(w_1, w_2, \dots, w_{t-1})$:

$$P(w_t | w_1, w_2, \dots, w_{t-1}) \quad (2.2)$$

The structural foundation of modern LLMs is the Transformer architecture, which relies on the Self-Attention mechanism. This mechanism allows the model to compute the relative importance of different tokens in a sequence regardless of their distance. The attention score is calculated using Query (Q), Key (K), and Value (V) matrices derived from input embeddings:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.3)$$

By utilizing multi-head attention and positional encoding, Transformers can capture complex linguistic patterns and long-range dependencies more effectively than previous sequential architectures like Recurrent Neural Networks (RNNs).

[Image of Transformer architecture diagram]

LLM Ecosystem: Proprietary vs Open-Weights

The landscape of Large Language Models in the industrial layer (not research layer) is categorized by deployment strategies and the degree of architectural transparency. Closed-source and cloud-based models, such as GPT-4 and Claude, represent the proprietary tier of this ecosystem. These models are typically provided as a Service (SaaS), where interactions occur via Application Programming Interfaces (APIs) hosted on centralized, high-performance infrastructures. While these proprietary solutions consistently deliver state-of-the-art performance across a wide range of benchmarks, their internal mechanisms—including model weights, training datasets, and specific configurations—remain opaque. Furthermore, their operational use often involves per-token costs, which can become a significant factor in large-scale or iterative research applications.

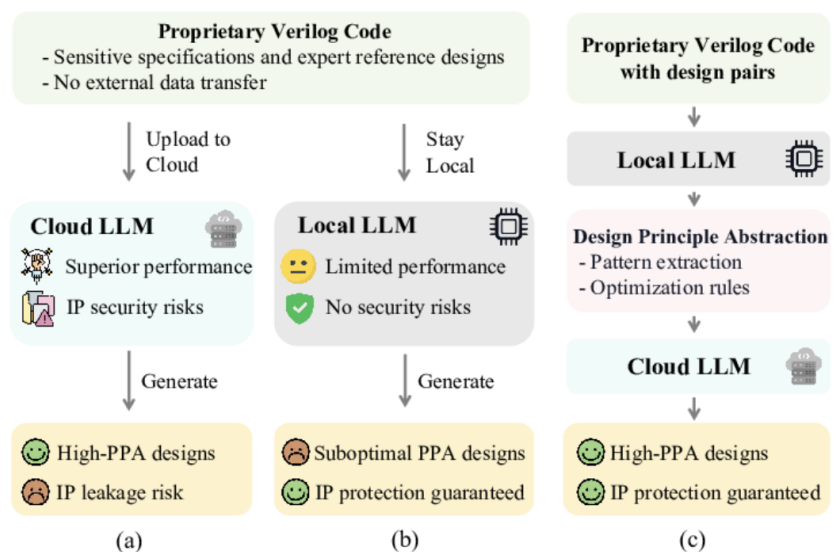


Figure 2.2: Comparison of LLM Deployment Strategies: Traditional approaches (a) Direct cloud processing, (b) Local-only processing, and (c) Local-Cloud framework balance performance security without leaking IP (proposed by [23])

In contrast to the proprietary paradigm, the emergence of open-weights and local models has introduced a significant shift toward decentralized execution and architectural transparency. Initiatives such as Meta’s Llama series and Mistral release their pre-trained model parameters to the public, allowing for deployment on private, local infrastructures. This transparency facilitates full control over data sovereignty and privacy, as information does not need to leave the local environment to be processed. Consequently, these models serve as foundational baselines, providing a stable platform for both academic research and industrial innovation without the necessity of external API dependencies or recurring subscription fees.

Building upon these general-purpose baselines, recent advancements in the field have catalyzed a trend toward specialization, where models are fine-tuned or structurally modified for high-performance niche tasks. One prominent example is the development of coding specialists, such as Qwen-coder or CodeLlama, which are optimized specifically for source code synthesis and complex logical reasoning. These specialized variants often rival or even outperform significantly larger general-purpose models in programming-centric contexts. Beyond text processing, research has also extended these baseline architectures into the multimodal and generative domains. By integrating vision encoders for visual comprehension or combining LLM backbones with diffusion-based architectures, researchers have developed sophisticated models capable of high-fidelity image generation and cross-modal reasoning.

LLM Inference

This subsection explains the utility of LLM inference parameters in controlling the generation process. During inference, the next-token probability distribution predicted by the model can be dynamically altered to balance determinism and creativity.

Temperature Temperature (τ) controls the sharpness or randomness of the predicted probability distribution. Given the logits z_i output by the final layer, the adjusted probability $P(w_i)$ for each candidate token is computed using the modified softmax function:

$$P(w_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (2.4)$$

A lower temperature ($\tau < 1$) sharpens the distribution, making the model more confident in its top choices and producing highly deterministic text, which is ideal for rigid tasks like JSON extraction. A higher temperature ($\tau > 1$) flattens the distribution, encouraging diversity and creativity.

Top-k Sampling Top- k sampling restricts the model from considering the entire vocabulary, instead keeping only the k highest-probability candidates. This approach is conceptually similar to beam search, where only the most promising paths are kept active:

$$V_k = \arg \max_{V' \subset V, |V'|=k} \sum_{w \in V'} P(w) \quad (2.5)$$

The probability mass is then redistributed among these k tokens. This truncation prevents the model from generating highly improbable and potentially nonsensical words (the "long tail" of the distribution).

Top-p (Nucleus) Sampling Top- p (nucleus) sampling provides a dynamic alternative to Top- k . Instead of keeping a fixed number of tokens, it keeps the smallest prefix subset of the vocabulary whose cumulative probability mass meets or exceeds a threshold p :

$$\min \left\{ |V'| : V' \subset V, \sum_{w \in V'} P(w) \geq p \right\} \quad (2.6)$$

This allows the candidate pool to expand or contract organically based on the model's confidence. When the model is very confident, the nucleus contains only a few tokens; when uncertain, the nucleus expands to include more options.

Repetition Penalty Repetition penalty modifies the logits of tokens that have already appeared in the generated sequence, mathematically reducing their likelihood of being selected again. This is crucial for preventing the model from entering infinite generation loops or excessively repeating specific phrases, ensuring a more natural and progressive text flow.

2.1.3 Agentic Systems

AI Agents

An Artificial Intelligence (AI) Agent is a computational system capable of autonomous task execution through the dynamic design of workflows using available tools. AI agents can encompass a wide range of functions beyond natural language processing including decision-making, problem-solving, interacting with external environments, and performing actions.

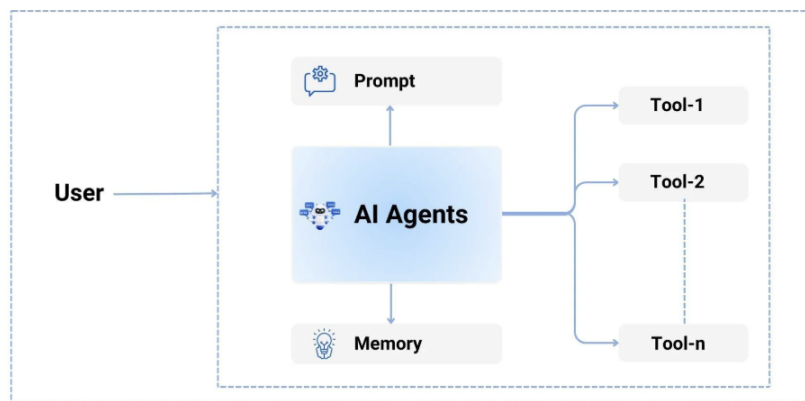


Figure 2.3: Basic Single Agent diagram

Unlike Large Language Models (LLMs), which produce responses solely based on the static data used to train them and are bounded by fixed knowledge limits, agentic technology uses tool calling on the backend to obtain up-to-date information. They optimize workflows and create subtasks autonomously to achieve complex goals. It is responsible for translating high-level natural language queries into executable workflows that complete real life tasks. What truly pushing an AI agent to achieve what LLM can't lies in the harnessing of agentic technology. Harness is the process of constructing and integrating systems or infrastructure that boosts capabilities into the system. The harnessing of agentic system is critical for the agentic performance, as it enables the system to navigate the complex, multi-step processes required in deep reasoning, complex tasks.

LLM: The Logical Core Large Language Models (LLMs) serve as the brain of the agent, represent the thinking capability. They provide the reasoning capabilities necessary to interpret vague user queries and orchestrate the appropriate tools to find answers. The LLM is responsible for parsing the user's intent, identifying missing information, and deciding on the next actions. The reasoning and decision-making depend heavily on the type of LLM used. Upon studying the current industry, mainstream types of models include:

- **Reasoning Models:** Models such as OpenAI o1 or DeepSeek R1 are optimized for Chain of Thought processing. They excel at complex logical deduction and planning multi-step workflows before taking action. This makes them ideal for the planning phase, where they must decompose difficult user queries into a clear, executable plan before invoking any tools.
- **Function Calling Models** Models such as GPT-4o or Claude 3.5 Sonnet are fine-tuned specifically to output structured data, such as JSON, rather than just conversational text. This capability is efficient for tool configuration and defining output formats, critical for the execution phase.
- **Multimodal Models** Models such as Gemini 1.5 Pro are trained with multimodal input, meaning they can natively understand both text and visual inputs.
- **Lightweight Efficiency Models** Smaller models like Llama 3 8B or GPT-4o-mini are faster and cheaper to run. They are perfect for narrow, repetitive tasks. Moreover, the ability to easily train and fine-tune these models facilitates customization in the design of Agentic systems.

Context Engineering Context engineering, represent the memorization capacity, involves crafting the environment in which the LLM operates to ensure accurate and consistent outputs. This field includes several topics that govern how the model interacts with data and users.

- **Prompting Strategies** Prompting is the primary method to instruct the agent, helping it understand the goals and constraints it is dealing with. Advanced methods like Chain of Thought (CoT) and Language of Thought (LoT) prompt the agent to expand its reasoning process step-by-step, ensuring meaningful thought process before a final answer is generated.
- **State Management** To maintain a consistent workflow, systems must store chat history and the current state. This allows the agent to understand conversation context, the results from tools or other agents. State management ensures the agent knows exactly where it is in a multi-step workflow process, preventing redundant actions or confusion.
- **Retrieval-Augmented Generation (RAG)** Agents cannot remember all the information contained in massive data repositories, so RAG is used to retrieve external information that is not stored within the LLM's internal memory. RAG ensures the agent gets the needed information by querying relevant data segments or metadata to ground its answers in factual evidence.

Tool Integration Represent the action capacity, tool integration is the process of connecting external resources (systems, software, hardware) or APIs to the agent, allowing it to perform specific functions that are beyond the capabilities of the LLM alone. This is critical for a automated system, as it enables the agent to truly interact with the external world, understand and complete its mission from practice instead of static texts, while also pose significant threats of misuse, when agent cause real life mistake and fail to judge.

Agent Frameworks

The concept of an Agentic Framework arises from a practical necessity: building reliable autonomous systems requires more than just smart models; it requires a strong structure. Retraining Large Language Models to handle every specific rule or task is extremely expensive and inefficient. Therefore, the industry has shifted towards Agent Frameworks—software architectures designed to control and improve the behavior of AI without changing the model itself.

These frameworks act as a bridge, connecting the creative nature of AI with the reliable logic of software architecture. The Agents are far more than LLMs with tools and context injection; they are intelligent systems with LLMs as the neural core and architecture following software logic. Modern frameworks leverage three main aspects in Agentic design and orchestration:

Multi-agent Orchestration In complex operations, giving a single agent too much information and too many different tasks often causes errors. A single agent has a limited attention span and context window. Frameworks solve this by using Multi-agent Systems. By breaking a large job into smaller parts handled by different agents, developers can assign specific agents to each task—one for visual analysis, one for audio processing, and one for synthesis—ensuring each agent stays focused and accurate.

Agent Memorization Intelligent systems need to remember information over time, but standard LLM treat every question as a new conversation (stateless). Frameworks solve this by building efficient memory systems with hierarchy, often using classes like Session or Memory. These classes act as data handlers containing multiple attributes and methods to help Agents interact with memory to retrieve previous context during run-time. Memory management almost always follows these paradigms:

- **Structured Memory:** Information is organized in a structured way, allowing the system to access relevant context precisely without overwhelming the agent with irrelevant details.
- **Context Window Optimization:** Frameworks implement strategies to optimize the context window. The memory is connected to an external database (usually MySQL) and agentic system get needed context through lazy memory loading of tool results, conversation topics, e.t.c., perform dynamic pruning and memory injection, ensuring that the agent has access to the most relevant information without exceeding its limits, as well as saving computational resources.

Tool Abstraction Frameworks turn standard code functions into Tools that agents can use on their own. The key difference between a normal function and a Tool lies in its data structure:

- **Description:** instructions on what it does and when to use it. The agent read this description when needed, doesn't overflow itself about the internal logic in the orchestration.
- **Input/Output Schema:** a defined structure for the input and output parameters, often in JSON format, that specifies the type and format of data the tool expects and returns. This allows the agent to understand how to call the tool correctly.
- **Logging memory:** functions results, errors logged into memory for agent inference, context injection and debugging.

This allows the framework to give the AI the right capabilities—like a calculator or a database search—allowing the agent to interact with the real world. Good tool design is critical for the performance of the agentic system, as it directly impacts the agent's ability to execute tasks effectively and efficiently. As such, tool design must ensure that the tools are not only functional but also easily accessible and interpretable by the agent, with clear documentation and error handling to facilitate smooth interactions.

Langchain and LangGraph

The specific usecase of our educational system necessitate a flexible, nonlinear, more orchestrated approach. While numerous frameworks exist to harness the capabilities of Large Language Models (LLMs), the intricate requirements of guidance necessitate a high-level, state-aware framework.

Unlike sequential or data-centric frameworks like LlamaIndex and SDK, LangGraph is built upon a *State Graph Architecture* where the system is modeled as a directed graph. In this paradigm, each node represents a specialized agent or a discrete computational step, while the edges dictate the transition logic based on the system's current state. This granular control over the workflow direction orchestration enables Non-linear Branching, allowing the system to perform multipath reasoning for unrepertitive, complex tasks. Considering educational usecase, since the workflow is globally state-aware, it can gracefully navigate diverse educational use

cases, without losing track of the student’s long-term learning objectives, a feat that is statistically difficult to achieve with stateless or purely sequential agentic frameworks.

Framework	Core	Characteristic	Strength in Education	Limitation in Education
LlamaIndex	Data-Centric: Optimizing the interface between LLMs and private data.	Retrieval-heavy; focus on indexing and query engines.	Very powerful when it comes to multistep input output manipulation of tools, LLMs and states	Often prioritizes data retrieval over complex, multi-step pedagogical reasoning.
LangGraph	Explicit state management via cyclic graphs. Supporting edges, nodes creation making full control of logic, branching	Stateful, non-linear branching with fine-grained control	Suitable for complex nonlinear task, with stateaware and context, logic localization and independence	Branching acts as a selling point and a potential logical error, when applying complex flow to simple tasks. Most of all, due to its high level abstraction, input output of LLM get unstable
Google SDK	Tool-Centric: Direct, low-latency access to model capabilities.	Functional/Linear; focus on tool-calling and result passing.	Deliver stable tool results, state and worker result if the pipeline is well designed	Becomes unstable and difficult to maintain when including branching workflows.
CrewAI	Role-Centric: Simulating human-like collaborative teams.	Autonomous, task-based collaboration between predefined roles.	High level, well LLM and human interaction facilitation, capable of complex workflow	Can lack the deterministic, full control or- chestration required for strict academic knowledge modeling.

Table 2.1: Comparison of LLM Orchestration Frameworks

The survey of various frameworks shown in 2.1 highlights LangGraph as top framework for educational system. While LlamaIndex offers stable operation with full control data, its data-centered nature often obscures the complex logic and asynchronization required for student-teacher interaction. The Google Generative AI SDK, though efficient for direct tool integration, lacks the complex state-management infrastructure necessary to handle the unstable nature of complex workflow, comparing to predecessor framework. Ability to maintain a persistent state and manage non-linear transitions of LangGraph and CrewAI provides the deterministic backbone required to facilitate a truly adaptive and reliable learning experience.

2.1.4 System Design

This section explain the software engineering principles, architectural patterns, and technological foundations utilized to construct scalable, AI integrated systems. Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects" which can contain data in the form of fields (attributes) and code in the form of procedures (methods). In the context of System Design, OOP is the dominant industry standard because the core components of

this architecture are inherently stateful. Unlike simple functional scripts that execute and exit, system components must maintain persistent internal states—including conversation history, active tool configurations. The OOP provides the natural structural to manage this state lifecycle. The system leverages the four main pillars of OOP:

- **Abstraction:** This allows the system to model complex entities by hiding internal implementation details and exposing only relevant operations, allowing the outside workflow logic to interact with high-level methods rather than raw data from lower components.
- **Encapsulation:** Groups related variables and functions into a single unit, protecting the internal state from unintended interference.
- **Inheritance:** This promotes code reusability by allowing new classes to derive properties from existing ones. Specialized classes then inherit these foundational capabilities while implementing their specific unique behaviors, significantly reducing code duplication.
- **Polymorphism:** This allows objects of different classes to be treated as objects of a common superclass. In the orchestration layer, the system can iterate through a diverse list of components—such as different retrieval engines or analysis modules—and trigger their execution loops using a uniform interface. The orchestrator does not need to know the specific type of component it is invoking, only that every component adheres to the expected interface contract.

System architecture defines the overall structure of a software system, organizing its major components and how they communicate. Major applications start with a Modular Monolithic Architecture. Unlike a traditional monolith where code is heavily tangled, a modular monolith compiles the application as a single unified program but keeps clear logical boundaries between different functions. This approach offers practical early-stage benefits: it simplifies deployment, ensures fast internal communication, and makes the codebase easier to test and debug.

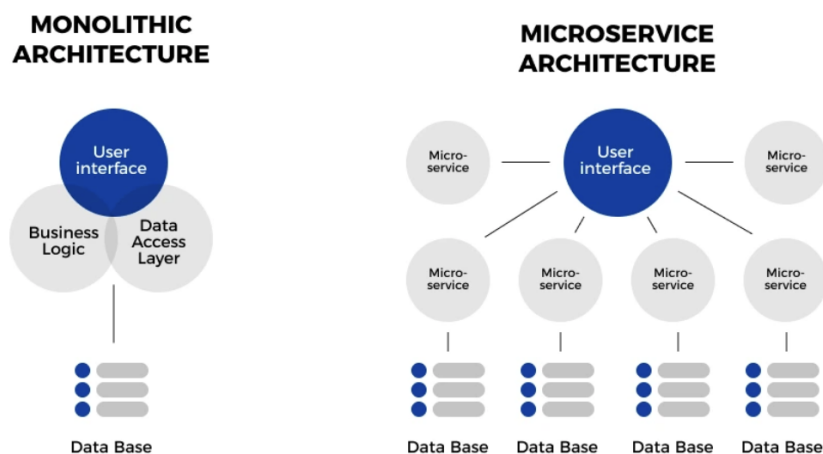


Figure 2.4: *Comparison of Microservice and Monolithic architecture*

However, as system demands grow, combining heavy processing tasks with basic web routing in a single program can cause performance bottlenecks. Microservices take the next step by turning logical modules into physical, independent services. Each service runs in its own process, often with its own separate database, and communicates with other services using standard protocols like REST APIs or message brokers. This architecture inherits the organizational advantages of the Modular Monolith, while providing several additional merits:

- **Independent Scalability:** Microservices allow different parts of the system to scale separately. For example, a heavy processing service can be upgraded with more computing power or duplicated without needing to scale the lightweight web server. This ensures efficient hardware use during high traffic.
- **Fault Isolation:** In a microservices system, software failures are contained. If a heavy processing service crashes due to memory limits, it does not bring down the entire system. Other services, such as user login and basic web browsing, remain fully operational while the crashed service restarts.
- **Technology Flexibility:** Microservices allow developers to choose the best programming language or tool for each specific task. For instance, data science components can be built using Python for its machine learning libraries, while high-speed API routing can be written in Go or Node.js.

Alongside these benefits, Microservices possess some intrinsic disadvantages. Internal function calls in a monolith are extremely fast. In microservices, these are replaced by network requests over the internet or local network, which inherently adds delay and processing overhead. Moreover, managing, monitoring, and deploying multiple independent services with separate databases is significantly harder than deploying a single program. It also requires complex strategies to keep data consistent across independent services.

2.1.5 Technology Stack and Frameworks

The selection of frameworks is driven by the need for high-performance system with seamless AI integration. The following frameworks are effective building blocks for the implementation of the application layer.

Backend Development

Backend development serves as the foundational logic of any software application, responsible for data processing, security, and the orchestration of communication between the database and the client. It handles the business logic that users do not see but rely upon for functionality. For an intelligent educational system involving heavy AI workloads, the backend must be capable of handling high-concurrency requests without latency, ensuring that the complex computations of the agents do not block the user experience.



To address these requirements, we utilize FastAPI, a modern web framework for building APIs with Python. While older frameworks like Django were built on synchronous standards known as WSGI, FastAPI adopts the Asynchronous Server Gateway Interface (ASGI). This approach allows the server to handle asynchronous code natively using Python's `await` syntax. This represents a paradigm shift for AI applications, moving away from blocking, sequential request handling to a concurrent model where I/O-bound tasks—such as waiting for an LLM response or a database query—release the CPU to handle other incoming requests, offering some key features:

- **High Concurrency:** By utilizing the ASGI standard, FastAPI can handle thousands of simultaneous connections (such as multiple users uploading videos or chatting with agents) without the blocking issues inherent in traditional synchronous frameworks.
- **Data Validation:** It integrates tightly with Pydantic, a data validation library. This ensures that the complex JSON outputs generated by AI agents are rigorously validated against strict schemas before being sent to the frontend, preventing runtime type errors.
- **Performance:** Built on top of Starlette, FastAPI offers execution speeds comparable to compiled languages like Go, which is critical for minimizing the latency overhead in the educational content processing pipeline.

UV: High-Performance Package Management

Traditional Python dependency management with pip and virtualenv introduces significant overhead in complex AI projects, where dependency resolution across dozens of GPU-accelerated libraries can take several minutes. The system adopts uv, a Rust-based Python package manager developed by Astral, which consolidates virtual environment creation, dependency resolution, and lockfile generation into a single binary.



UV achieves dependency resolution 10–100 times faster than pip by implementing a custom resolver in Rust. Its deterministic lockfile mechanism ensures that all team members and deployment environments operate with identical dependency versions, eliminating the “works on my machine” class of bugs. For this project, uv manages all backend dependencies—including LangChain, LangGraph, PyTorch, and the database client libraries—providing a consistent and reproducible build pipeline from development through production deployment.

Ollama: Local Model Inference and Optimization Infrastructure

Ollama is an advanced abstraction layer designed for managing and executing LLMs on local infrastructure. It is built upon the llama.cpp core, providing an efficient environment for model orchestration and hardware acceleration.



A critical feature of local inference is Quantization, the process of reducing the precision of model weights from high-bit floating-point formats (e.g., FP16 or BF16) to lower-bit integers (e.g., 4-bit or 8-bit). This technique significantly reduces the VRAM footprint and memory bandwidth requirements:

$$w_q = \text{round} \left(\frac{w}{\Delta} \right) \quad (2.7)$$

where w is the original weight and Δ is the scaling factor. Quantization enables large-scale models to run on consumer-grade hardware with minimal degradation in perplexity and reasoning capabilities. Moreover, it utilizes the GGUF (GPT-Generated Unified Format), a binary format designed for rapid model loading and efficient metadata storage. Key technical characteristics include:

- **mmap Compatibility:** Allows models to be mapped directly into memory, enabling nearly instantaneous startup times.
- **Unified Execution:** Supports flexible memory offloading, where layers of the model can be distributed between the GPU (VRAM) and CPU (System RAM) based on available resources.
- **Modelfile Abstraction:** Provides a structured way to define system prompts, temperature settings, and context window sizes within a single configuration file, ensuring deterministic behavior across different local environments.

The Langchain Ecosystem

As mentioned in Subsection 2.1.3, Langchain [8] is a powerful framework for our educational system, with State Graph Architecture. Moreover, it is one of the first and most popular agentic frameworks in the industry, with a large community and rich documentation. The accumulated contributions to the ecosystem and agentic research using Langchain have provided us with a wealth of tools and integrations:

- **Pre-built Agents:** Langchain offers a variety of pre-built agents that can be easily customized for specific use cases, such as question-answering, summarization, and multi-step reasoning. In 2026, it publishes DeepAgent, capable of extremely long term reasoning and complex workflow orchestration, and currently leads the harnessing researching trend in the industry.
- **Langfuse:** Langfuse, among with Langsmith, is one of the most powerful tools for agentic system development. It provides a comprehensive observability platform for monitoring and debugging agent interactions, allowing developers to trace the decision-making process of agents in real-time. This is critical for diagnosing issues in complex workflows and ensuring that the system behaves as expected, making agentic operation more transparent, manageable and less errorprone.
- **Context Management:** Langchain includes built-in context management capabilities, enabling agents to maintain context across interactions and manage state effectively. Those includes utilities for memory management (adapting to multiple external databases), conversation history, and dynamic context injection, which are essential for building complex agentic system for long interactions over time.

2.1.6 Database System Design & Management

Database is where you store necessary information where application can read, write, update and delete via query or integrated calls or methods. In the domain of modern software engineering, data handling has become increasingly complex. To address this, robust architectures utilize a Polyglot Persistence strategy, utilizing distributed storage technologies optimized for different data modalities.

Table 2.2: *Polyglot Persistence Architecture: Strategic allocation of storage engines.*

Storage Engine	Data Modality	System Role	Theoretical Justification
Relational (MySQL)	Structured Data	User Authentication, Billing, Access Logs	Enforces ACID properties for transactional integrity and supports complex relational joins for administrative analytics.
NoSQL (MongoDB)	Semi-Structured Data	Semantic Metadata, Analysis Tags, JSON Outputs	Provides schema polymorphism to handle variable outputs from different AI agents (e.g., OCR vs. Object Detection) without sparse tables.
Object Storage (MinIO)	Unstructured Binary	Lecture Materials, Educational Assets, Audio Clips	Decouples storage from compute to prevent database bloat; utilizes Erasure Coding for high-throughput parallel streaming during ingestion.
Vector Database (Milvus)	High-Dimensional Vectors	Embeddings Index, Semantic Search	Optimized for Approximate Nearest Neighbor (ANN) algorithms (e.g., HNSW), enabling efficient similarity search in latent spaces where traditional B-Tree indexing fails.
Graph Database (Neo4j)	Multi-hop Graph Data	Knowledge Graph, Prerequisite Relationships	Optimized for traversing complex relationship networks; enables recursive queries and multi-hop reasoning to connect disparate educational concepts.

Relational Database (RDBMS)

Relational Databases are mathematically founded on Set Theory and First-Order Predicate Logic. They organize data into tuples (rows) and attributes (columns) within strictly defined relations (tables). The defining characteristic of this model is the enforcement of ACID properties (Atomicity, Consistency, Isolation, Durability), which guarantees that database transactions are processed reliably.

RDBMS is typically utilized strictly for data that requires high integrity and deterministic relationships, such as User Management and Billing. It acts as the source of truth for entity relationships, enabling complex SQL JOIN operations (e.g., selecting all learning materials uploaded by a specific role within a timeframe). The adoption of RDBMS for core administrative data offers specific advantages to system stability.



MySQL is utilized as the primary Relational Database Management System. It handles core entities such as user accounts, roles, and administrative logs, ensuring that all structured interactions follow strict relational integrity.

- **Transactional Integrity:** In a multi-user environment where credits or permissions are updated frequently, RDBMS ensures that race conditions do not occur. For instance, if a user purchases a subscription while simultaneously uploading an educational asset, the ACID properties ensure the account balance is updated accurately before the upload permission is granted.
- **Data Normalization:** By structuring data into normalized tables (e.g., Users, Roles, Logs), redundancy is minimized. This ensures that a change in a user's details is reflected system-wide instantly, without needing to update thousands of duplicate records.
- **Complex Querying:** The robust SQL engine allows administrators to perform audit trails and analytics efficiently, such as tracking system usage patterns across different user groups, which is computationally expensive in non-relational systems.

Non-Relational Database (NoSQL)

To address the limitations of rigid RDBMS schemas, Non-Relational databases emerged, often adhering to the CAP Theorem (Consistency, Availability, Partition Tolerance). Unlike RDBMS which prioritizes strong Consistency, NoSQL databases like MongoDB often prioritize Partition Tolerance and Availability, offering a flexible schema-less design.

Document stores are typically the primary engine for storing the Semantic Metadata derived from educational content analysis. Metadata derived from curriculum analysis is highly variable; one resource might contain OCR text, while another contains only Audio Transcripts or Face Recognition data. Document stores handle these as dynamic BSON (Binary JSON) documents, allowing fields to be added on the fly without the sparse matrices typical of rigid SQL tables.



MongoDB serves as the high-performance document store for managing the semi-structured metadata generated by AI agents. It allows the system to store diverse analysis results—such as transcripts, OCR results, and agentic reasoning logs—within a single, flexible schema.

- **Schema Agnosticism:** AI agents are constantly evolving. If an "Object Detection" agent is upgraded to output new attributes (e.g., "Confidence Score" or "Bounding Box Coordinates"), Document stores allow the new data to be saved immediately without migrating the entire database schema or breaking existing application logic.

- **Read Performance for Aggregates:** In a Knowledge-QA scenario, the system needs to retrieve the Video Title, Description, Transcripts, and Vector IDs simultaneously. In an RDBMS, this might require joining multiple tables. In a NoSQL system, this data can be stored as a single nested document, allowing the application to retrieve the entire context of a lesson in a single, high-speed read operation.
- **Horizontal Scalability:** As archives grow to thousands of hours of lecture content, metadata storage requirements increase. Native sharding capability allows data to be distributed across multiple servers economically, ensuring lookup times remain low even as the dataset grows.

High-Performance Object Storage

While RDBMS and NoSQL excel at storing alphanumeric data, they are fundamentally inefficient for storing massive binary large objects (BLOBs) like lecture recordings, leading to database bloat and performance degradation. Object Storage systems like MinIO treat data not as a file hierarchy or a table, but as discrete objects stored in a flat address space.

High-performance architectures utilize Object Storage to handle the High Volume nature of raw educational media. It utilizes Erasure Coding—a mathematical algorithm that splits data into fragments and expands them with redundant data pieces—allowing for data recovery even if multiple drives fail. This approach decouples storage from compute; the heavy educational artifacts are offloaded to Object Storage, while the databases store only the lightweight reference pointers (URLs) to these objects.



MinIO is deployed as a high-performance, S3-compatible object storage server. It is responsible for hosting all raw multimedia assets, including lecture videos, extracted images, and audio segments used for AI training and inference.

- **Decoupling Storage and Compute:** By separating the raw files from the metadata databases, "database bloat" is prevented. This ensures that the RDBMS remains lightweight and responsive for user queries, while the Object Store handles the heavy I/O load of content streaming.
- **Parallel Streaming for AI Ingestion:** When an Agent analyzes a lecture recording, it requires high-speed access to the file to extract frames. Object Storage supports parallelized byte-range requests, allowing the AI to read multiple parts of a media file simultaneously. This significantly reduces the time required for the ingestion and preprocessing phase compared to reading from a standard file server.
- **Cost-Efficiency and Redundancy:** Erasure Coding allows the system to survive hardware failures without data loss, while consuming less storage space than traditional replication methods. This is vital for maintaining a massive archive of media files economically.

Vector Database

Traditional databases are designed for structured data and are inefficient for handling high-dimensional vector data generated by AI models. Vector databases are optimized for storing and querying high-dimensional vectors, which represent the semantic content of lecture segments. They utilize Approximate Nearest Neighbor (ANN) algorithms, such as Hierarchical Navigable Small World (HNSW) graphs, to enable efficient similarity search in latent spaces where traditional indexing methods fail.

The Vector Database is the backbone of the Retrieval-Augmented Generation (RAG) paradigm, allowing the system to perform semantic search based on the embeddings generated by the AI agents. When a user query is processed, it is transformed into an embedding vector, which is then compared against the vectors stored in the database to retrieve the most semantically relevant knowledge segments. The adoption of a Vector Database is critical for enabling the advanced retrieval capabilities required for educational analysis.

Milvus is the chosen vector database engine for SmartEdu. It indexes high-dimensional embeddings generated from course documents and lecture transcripts, providing sub-second retrieval times for semantic similarity queries across millions of vectors.

- **Efficient Similarity Search:** Traditional databases cannot efficiently query high-dimensional vectors. Vector databases provide specialized indexing structures to handle this efficiently.
- **Scalability for Large Datasets:** As the number of educational segments grows, the vector database can scale horizontally to maintain low latency for similarity searches, ensuring that retrieval times remain fast even as the dataset expands.
- **Integration with AI Workflows:** Vector databases are designed to integrate seamlessly with AI pipelines, supporting real-time insertion and retrieval of embeddings.
- **Support for Dynamic Data:** As new educational materials are ingested and analyzed, their embeddings can be added to the vector database in real-time, allowing the system to continuously evolve and improve its retrieval capabilities without downtime.

Graph Database

Neo4j is utilized to store the "Backbone Graph" of the knowledge base. It represents educational entities (concepts, lessons, modules) as nodes and their pedagogical relationships as directed edges.

- **Multi-hop Reasoning:** Unlike relational databases that require expensive JOIN operations, Neo4j allows for efficient traversal of deep relationship chains. This is critical for detecting knowledge gaps that may span multiple prerequisite levels.
- **Cypher Query Language:** The use of Cypher enables the system to express complex pedagogical patterns, such as "Find all prerequisite concepts of concept X that the student has not yet mastered," in a readable and performant manner.
- **Dynamic Schema:** The graph model allows the knowledge structure to evolve as new educational materials are added, without needing to redefine rigid table structures.

Database Management and Observation

Managing multiple database engines requires a unified observation layer to ensure data consistency and facilitate debugging during development. DbGate is an open-source, multi-database manager used as the primary dashboard for the SmartEdu infrastructure. It provides a unified web interface to inspect and manipulate data across MySQL and MongoDB instances simultaneously.

- **Unified Interface:** Reduces the overhead of switching between multiple specialized management tools.
- **Schema Visualizer:** Helps developers understand the evolving structure of NoSQL documents and Relational tables in real-time.
- **Query Benchmarking:** Allows for testing and optimizing complex queries before they are integrated into the application's repository layer.

2.1.7 Deployment and Containerization

Deployment is the process of making a software application available for real applicational usage. This section discusses the deployment strategy and the use of containerization to ensure that the system can be reliably and efficiently deployed for our AI pipeline and highperformance stacks as discussed above.

Docker [5] is a containerization platform that enables developers to bundle applications and their required components into isolated units called containers. Each container acts as a lightweight, virtualized environment that includes all the necessary elements for the application to run. Because containers are portable and consistent, they work reliably across various environments such as on-premise systems, cloud platforms, and clustered infrastructures.

- **Portability:** A major advantage of Docker is its strong portability. Containers package the application together with all its dependencies, making it simple to move workloads between different environments without compatibility issues. Whether running on a local machine or a cloud server, a Docker container behaves the same way, ensuring consistency throughout the development and deployment pipeline.
- **Isolation:** Docker provides strong isolation between applications and their dependencies. Each container operates independently, preventing interference between applications running on the same host. This isolation enhances both security and stability, especially in shared hosting environments.
- **Resource Efficiency:** Docker containers are more resource-efficient than traditional virtual machines. Since containers share the same operating system kernel, they consume far less memory and disk space. This allows more applications to run on a single machine, reducing hardware overhead and simplifying resource allocation.
- **Scalability:** Docker supports efficient scaling of applications. By using orchestration tools such as Kubernetes or Docker Swarm, developers can easily launch multiple container instances and distribute workloads as needed. This flexibility allows systems to rapidly adjust capacity to meet varying levels of demand.

- **Faster Development and Deployment:** Docker improves the speed and efficiency of both development and deployment processes. Developers can work in isolated, reproducible environments identical to production, minimizing integration issues. The deployment process becomes more automated, streamlined, and less prone to errors, saving significant time.

Docker has transformed modern software development by simplifying the creation, deployment, and management of services. Its emphasis on portability, isolation, resource efficiency, scalability, and rapid deployment makes it an indispensable tool for both development and operations teams.

2.1.8 Security: OAuth 2.0 Authorization

OAuth is a widely used authentication and authorization protocol in backend systems across various programming languages and frameworks. Here are some general benefits of incorporating OAuth into the backend system:

- **Security:** OAuth provides a secure way for users to grant third-party applications limited access to their resources without sharing their credentials. This reduces the risk of unauthorized access and protects user data.
- **User Experience:** OAuth enables seamless authentication and authorization experiences for users by allowing them to log in to your application using their existing accounts from trusted providers. This eliminates the need for users to create and manage yet another set of credentials.
- **Single Sign-On (SSO):** OAuth facilitates single sign-on capabilities, enabling users to access multiple applications and services with a single set of credentials. This improves user convenience and reduces the likelihood of password fatigue.
- **Granular Permissions:** OAuth allows you to define and enforce granular permissions for accessing resources. This ensures that third-party applications only have access to the specific resources they need, enhancing security and privacy.
- **Scalability:** By offloading user authentication and authorization to OAuth providers, your backend system can better handle scalability challenges. OAuth providers are equipped to handle large volumes of authentication requests efficiently, allowing your application to scale more effectively.
- **Standardization:** OAuth is a widely adopted and standardized protocol, supported by major technology companies and libraries across different programming languages. Leveraging OAuth in your backend system ensures compatibility with a broad ecosystem of tools and services.
- **Compliance:** OAuth implementations often come with built-in support for compliance with regulatory requirements such as GDPR (General Data Protection Regulation) or HIPAA (Health Insurance Portability and Accountability Act). This can simplify your compliance efforts and reduce legal risks.
- **Developer Ecosystem:** OAuth has a large and active developer community, providing extensive documentation, libraries, and support for various programming languages and frameworks, such as PyJWT for protocol schema, e.t.c. This makes it easier for developers to implement OAuth in their backend systems and troubleshoot any issues that may arise.

2.2 Related Work

2.2.1 Document-Knowledge Modeling

Converting documents into structured knowledge representations has become a critical task for so long in the industry and education specifically. To replace manual curation in educational contexts, which is time-consuming and not scalable, prior attempts have been focusing on the power of entity extraction, prerequisite relation reasoning, and RAG. This has been indeed a fine strategy, given the success of early supervised models like PREREQ [11] using latent topic representations with Siamese networks to infer course dependencies, which achieved over 70 % Precision@100 scores on the University Course and NPTEL MOOC datasets. However, the traditional methods of entity extraction and relation reasoning often rely on rule-based systems or pure statistical machine learning techniques, which can struggle to capture the complexity and nuance of educational content's relationships. Moreover, recent researches [25, 13] have proven that models fail drastically to access relevant information when documents exceed a certain length. That is the reason why though prior researches gained impressive results on medium retrieval benchmarks, they are not easily applied in industry when it comes to massive and unstructured enterprise data.

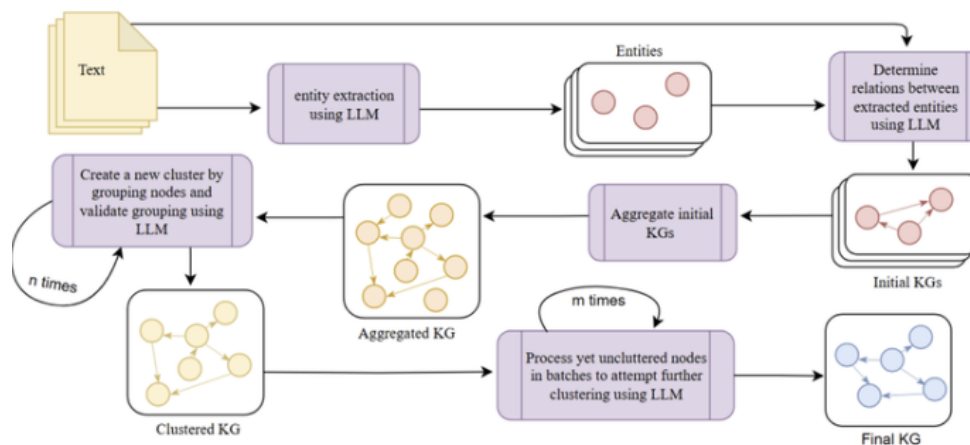


Figure 2.5: *KGGen with multiturn LLM reasoning, grouping generation*

To address this, LLM-based frameworks have been widely proposed. A prime example is KGGen [16], an automated system that utilizes a clustering algorithm to group synonymous entities and relations, resulting in denser and better-connected graphs that scored an impressive 81.03% average fact accuracy on the MINE-2 (Measure of Information in Nodes and Edges) benchmark. The EDC [27] framework extends this capability by extracting, automatically defining schemas, and canonicalizing knowledge triples without predefined schemas, consistently surpassing state-of-the-art baselines on complex datasets like WebNLG, REBEL, and Wiki-NRE.

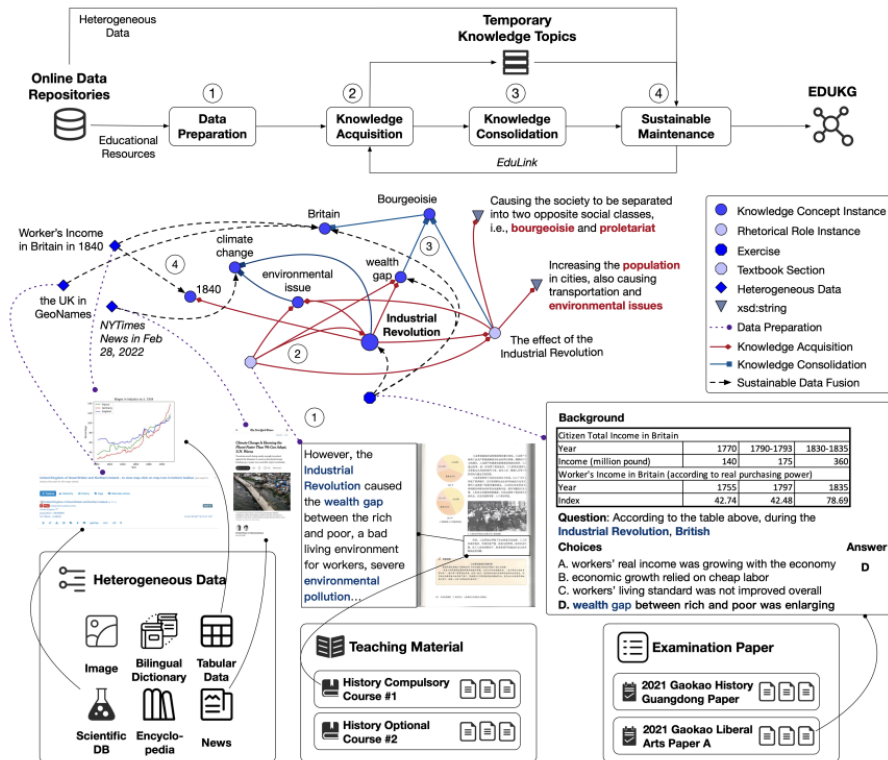


Figure 2.6: The end-to-end EduKGPipeline framework for educational knowledge construction and retrieval.

However, LLMs have deadly drawbacks. First and most popular of all is the "hallucinations problem" leading to inaccurate, destructured extractions. To mitigate this, GraphJudge [6] proposes using a fine-tuned open-source LLM as a "judge" to score and filter out incorrect knowledge triples generated by naive LLMs. Validated on general (REBEL-Sub, GenWiki) and domain-specific benchmarks (SCIERC, Re-DocRED), it achieved over 90% judgement accuracy. Furthermore, for large-scale educational and enterprise systems requiring cost and latency optimization, comprehensive hybrid pipelines have proven highly effective in practices. For instance, EduKGPipeline [1] establishes an end-to-end framework fusing traditional text extraction with transformer-based weighting, demonstrating high precision on educational transcripts from the CCI dataset.

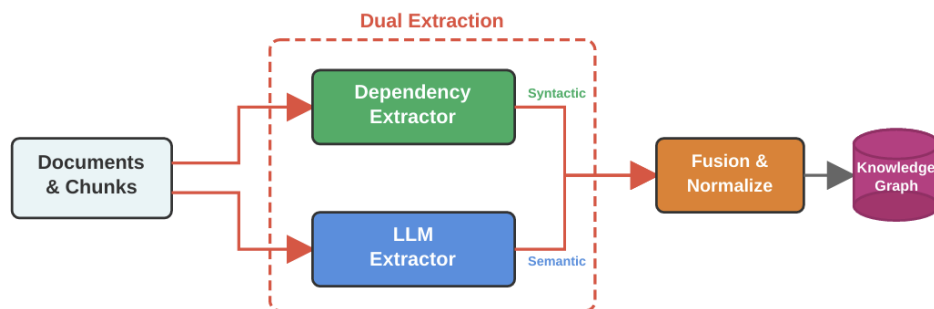


Figure 2.7: Hybrid architecture of GraphRAG allow minimal cost and guarantee alignment of generated to original context

Similarly, recent efficient GraphRAG architectures [15] substitute expensive LLM extractions with classical dependency parsing to maintain scalability, retaining 94% of LLM-based performance on RAGAS metrics when evaluated on real-world enterprise datasets like CCM (Custom Code Migration). Finally, CoDe-KG [2] utilizes syntactic decomposition to process complex

sentences, thereby significantly enhancing information recall by an 8-point gain on rare relations across the REBEL and WebNLG2 benchmarks.

Another inevitable error lies in the LLMs' limited context length, making it unable to generalize broad knowledge. It is good when discovering short-dependent context/entities/relationships in small sections but fails for long dependencies of extremely long documents and course information. Trying to inject an excessive amount of information even when cut by windows, until now with the current level, only leads to redundant costs from multiple calls, information overload, and decreased accuracy. To uncover hidden relationships and deduce learning paths beyond textual information, some studies shifted to an algorithmic approach.

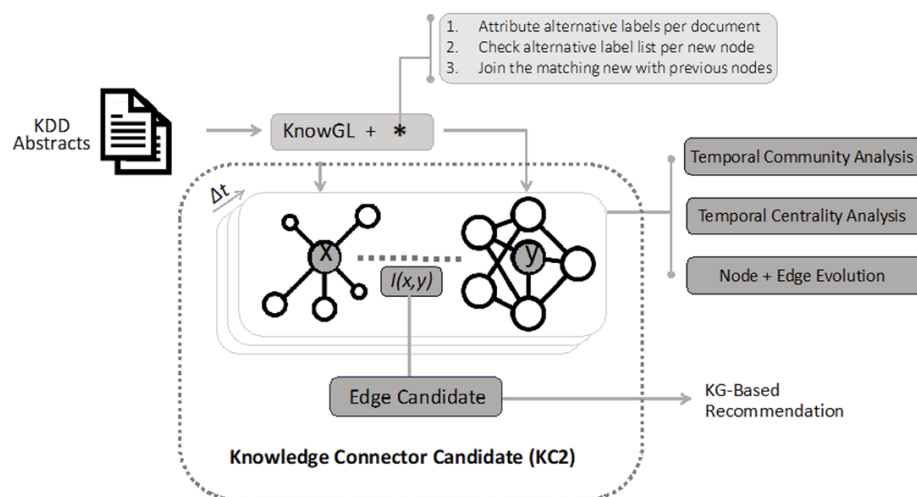


Figure 2.8: Dynamic Knowledge Graph (DKG) approach tracking knowledge evolution over time.

The Dynamic Knowledge Graph (DKG) [3] is one of the strongest representative to this approach, analyzing Knowledge Graph's macroscopic structure. To recommend strong relational paths, DKG introduces the KC2 (Knowledge Connector Candidate) algorithm, which uses closeness centrality to suggest connections between disconnected knowledge islands. Additionally, to cluster knowledge into logical chapters or modules, the Leiden [22] algorithm is utilized to optimize the CPM quality function, ensuring that knowledge communities are absolutely well-connected and not disjointed—a common flaw of the traditional Louvain algorithm. It also divides KGs into discrete time snapshots and applies network science to track knowledge evolution and clustering group changes, creating a dynamic regroup, reformulation of clusters and formulation of new knowledge groups.

2.2.2 Generative AI in Interactive Teaching

Bridging the gap between raw document extraction algorithms and end-user educational platforms is the integration of Generative AI and Large Language Models (LLMs). The increasing accessibility and reasoning capabilities of Generative AI tools have led to widespread exploration in educational settings, fundamentally transforming traditional teaching and learning paradigms [17]. Modern LLM agents are proved to be capable of autonomous reasoning, tool manipulation, and orchestrating complex academic tasks.

Recent surveys highlight the versatility of LLM agents in automating multifaceted educational workflows, ranging from curriculum design to generating highly contextualized feedback [4]. These agents operate dynamically, leveraging multi-hop reasoning and continuous in-

teraction to adapt to a student's cognitive state. A qualitative study involving educators demonstrated that the core impact of generative models spans three critical pillars: active teaching and learning, administrative automation, and personalized assessments [17].

A prominent, large-scale empirical example of this shift is the integration of AI within Harvard University's CS50 course. By deploying a suite of specialized AI-based software, the course successfully approximated a 1:1 teacher-to-student ratio. Crucially, the system was designed with pedagogical boundaries—guiding students toward conceptual understanding rather than merely offering outright code solutions. This constrained autonomy proved highly effective in code explanation, style improvement, and administrative querying, demonstrating the practical viability of AI as a ubiquitous teaching assistant [14].

Despite the profound opportunities, deploying LLM agents in educational ecosystems introduces substantial systemic and ethical challenges. The inherent probabilistic nature of LLMs frequently leads to "hallucinations," where the model confidently generates factually incorrect or flawed information. Theoretical analyses of LLM representations reveal phenomena such as "semantic collapse," where the continuous manifold of the model's activations struggles to maintain stable, logically sound ontologies over continuous, long-term reasoning tasks [26].

To pinpoint this, researches educational AI Application shift from static next-token prediction to autonomous agentic system, aiming for frameworks that tightly navigate the semantic boundaries for AI agents in an academic context, ensuring the reliability, alignment, and transparency of AI results. Architectures like the Adaptive Evaluation Multi-Agent (AEMA) framework have been one of the latest proposal to address this gap. AEMA provides an auditable, multi-step evaluation mechanism that operates under human oversight, ensuring that the agents' internal decision-making processes, tool selections, and collaborative strategies are verifiable and transparent [10]. Implementing such verifiable frameworks is essential to mitigate the risks of unconstrained autonomy, ensuring that the integration of AI into education remains both trustworthy and pedagogically effective.

2.2.3 Educational Intelligent System

Smart AI based educational systems has leverage from pure research topic and find itself introduced to commercial use. A lot of Intelligent commercial platforms have been born recently, and research institutions have also propose a fully end-to-end pipeline beyond a benchmarked study method.

In research field, [28] (along with variants like K12EduKG and MOOC-KG) has been developed to uniformly model knowledge from textbooks and online courses. The [1] framework automates KG construction from unstructured PDF documents by extracting keywords via Keybert and performing semantic linking with LLMs. Furthermore, MEduKG pushes beyond text limitations by integrating multi-modal data into the graph, thereby enhancing the system's comprehensiveness and vividness for students.

As mentioned, some systems relating to this field have also reached the market. Popular platforms worldwide like Khanmigo leverage Generative LLMs to provide personalized tutoring and real-time feedback, with enormous available resources. For Vietnamese representative, VioEdu utilizes a manually curated knowledge tree to structure its curriculum, while EdMicro, specialized for enterprises, employs a skill matrix to map out learning objectives and prerequisites. These systems represent different approaches to integrating AI into education, each with its own strengths and limitations in terms of scalability, personalization, and content structuring. Based on practical surveys and evaluation criteria, current educational systems in the market exhibit various characteristics regarding AI and knowledge structuring. A comparative analysis of these platforms is presented in Table 2.3.

Criteria	EduKG	Khan Academy	Docebo	VioEdu (FPT)	EdMicro
Adaptive learning: Personalize the learning process based on needs, abilities, and preferences.		x	x	x	x
Intelligent Recommendations: Some personalized suggestions courses, videos based on preferences, and areas of improvement.		x		x	
Natural language processing for queries: Based on the questions from learners, it can give out the answer based on the content the LMS has in its DATABASE.	x	x			
Learning analytics/Progress Tracking: AI-based analytics track and analyze user progress. Can visualize using tracking board, and leader-board.		x	x	x	x
Text-to-audio content: Provide audios for text content.		x			
Interactive learning method: Interactive content, such as quizzes, coding exercises, and simulations, can engage learners and reinforce learning.	x	x	x	x	x
Dynamic Roadmap: Automatically extract concepts and reason long-term prerequisite learning paths without manual expert input.	x				

Table 2.3: Comparative analysis of representative educational intelligent systems based on key AI and knowledge features.

2.2.4 Conclusion and Research Gaps

During the survey of mentioned representative researches, upon studying from multiple peer reviews and self-reviews of those, this report pinpoints the following core research gaps in the current landscape:

Limitations in Automation and Cost Optimization: All mentioned big systems heavily depend on manually built knowledge trees or skill matrices. Course contents, structured and ordered by expertise indexing, but lack inner modelling and semantic makes the system less adaptive to students' needs and preferences, leading to good courses recommendations but terrible inner course concept recommendations. This makes learning path become a linear process and not flexible on the micro scale, with little to no revision and update. The lack of automation also limits the scalability of these systems, as manual labeling becomes increasingly impractical as the amount of content grows.

Lack of Connectivity and Transparency in Lesson Structures: Traditional methods like clustering in text-mining often produce disconnected or fragmented knowledge groups. Existing learning platforms typically lack a standard, mathematically proven network algorithm for dividing study modules, leading to disjointed learning path that disrupt the learners' cognitive flow.

Static Roadmap Reasoning and Gap Detection: While current LMSs and AI tutors provide generated advice, they are actually still relied on traditional long form docs or RAG, lacking

the network structure and deterministic approach to map out prerequisites dynamically. The identification of critical "knowledge gaps" remains static and reactive, failing to automatically trace back to the fundamental bridge concepts (based on network centrality) that cause student bottlenecks.

So far [1] and its lineages have made a significant attempt to propose solutions to those mentioned problems with an end-to-end pipeline that fuses traditional text extraction with transformer-based weighting. [3] has proposed what [1] is missing: a mathematical efficient community detection method, furthermore solve the connectivity and transparency issues in lesson structures that almost all of previous pipeline encountered. However both have their own limitations. The former has no deterministic knowledge mapping, which causes knowledge disruption (even when the graph is not fragmented), while the latter pipeline is not as efficient in multistep extraction as the former, thus good for knowledge modeling and analysis but not inference and recommendation. Finally, a lot of great proposal here are not widely adapted due to the lack of a comprehensive system that make use of their research project.

Chapter 3

Requirements Specifications

The limitations discussed in Subsection 2.2.4 have shaped the technical vision for the proposed system. To address these gaps and deliver a robust, scalable educational platform following our goal (Section 1.2), this section outlines the explicit Functional and Non-Functional Requirements, as well as visualizing use—case design. These requirements are derived directly from the systemic constraints and the pedagogical needs of learners for a more personalized, diversified and comprehensive learning assistance system.

3.1 Functional Requirements

3.1.1 Knowledge Modelling and Management

As a key objective feature, knowledge modelling construct knowledge base, foundation to all other key features. This functional category encompasses the capabilities required to process raw educational inputs into a structured knowledge base.

- **Data Ingestion and Parsing:**

- The system must be able to ingest unstructured, text-dense academic formats (e.g., PDF textbooks, syllabus guidelines).
- The system must accurately parse text into discrete chunks while retaining crucial positional metadata (e.g., source file origin, exact page numbers).

- **Semantic Extraction and Normalization:**

- The system must extract fundamental educational concepts and their explicit pedagogical relationships from the ingested text.
- The system must normalize extracted entities to a canonical form to ensure conceptual consistency and prevent any form of semantic fragmentation (even of different course) across the knowledge base.

- **Hierarchical and Relational Organization:**

- The system must organize extracted entities into a standardized hierarchical topology.
- The system must automatically group highly related concepts into cohesive educational clusters or chapters.

- The system must explicitly define and store directional "prerequisite" relationships between concepts to enable automated learning path generation.

- **Storage Capability:**

- The persistence mechanism must guarantee a database structure that facilitates high-speed, complex relational querying and subgraph retrieval during live interactions.
- To ensure diversity of learning input, storage design must ensure efficient storing of enormous data with reasonable latency.

3.1.2 Intelligent Teaching Assistance

To utilize modeled knowledge base as well as guarantee satisfactory learning experience and result, the design of agentic system that serves as our Teaching Assistance is crucial. This category defines the required behaviors and reasoning capabilities of the automated teaching assistant.

- **Intent Recognition and Context Orchestration:**

- The system must classify user intents (e.g., Retrieve information, Generate roadmap, Clarify concepts) and autonomously route the request to the appropriate internal workflow.
- The system must dynamically gather necessary context before responding, which includes retrieving the student's personal mastery history and the relevant subset of the educational knowledge base.

- **Intelligent Reasoning and Supervision:**

- The system must support a Socratic teaching interaction; it must be capable of generating probing questions to assess student comprehension rather than merely providing direct answers.
- The system must objectively evaluate the student's response to determine whether to authorize progression to the next prerequisite topic.
- The system must be able to autonomously detect when it lacks sufficient context to answer a question and execute a fallback strategy that synthesizes information from the broader knowledge base.

- **Response Verifiability and Fallback:**

- To guarantee grounded responses, the system must prioritize explicit document referencing, citing exact source pages for its generated answers.
- The system must execute a broader knowledge synthesis fallback only when direct contextual concepts are missing from the immediate document.

3.1.3 Student Interaction and System Navigation

This category defines the functional requirements concerning the end-user's experience, accessibility, and direct interaction with the intelligent system.

- **Identity and Access Security:**

- The system must provide secure authentication capabilities (Login/Logout) to uniquely identify users.
- The system must maintain isolated, tamper-proof session states for each user to securely record individual learning progress without cross-user data exposure.
- **Interactive Dialogue and Visualization:**
 - The system must provide a natural language chat interface that supports continuous, multi-turn conversational interactions and real-time streaming responses.
 - The system must dynamically render visual learning roadmaps, empowering students to make informed, user-centric choices regarding their educational progression rather than following a static curriculum.
 - The interface must consolidate essential learning tools, providing seamlessly integrated functional components such as conversational history logs, a synchronized document viewer, and progress summarization dashboards.
- **Advanced Navigation and Content Discovery:**
 - The system must implement an intuitive, multi-level navigation structure (e.g., breadcrumb trails) to help students orient themselves within complex knowledge hierarchies.
 - The system must incorporate a powerful semantic search utility, allowing users to rapidly query and locate specific educational materials, graph concepts, or historical interactions.
- **Contextual and Responsive Feedback:**
 - The system must be capable of emitting explicit interface triggers alongside its textual responses to automatically synchronize the user's interface (e.g., auto-scrolling the document viewer to the exact referenced page).
 - Notifications and intelligent pedagogical prompts must be delivered in a non-intrusive, accessible manner that captures user attention while preserving the fluidity of the learning experience.

3.2 Non-Functional Requirements

To create a powerful, reliable, and user-friendly educational platform, the Smart Learning System cannot stop at being functional, but need focusing on efficiency and overall system performance. Thus, it must also adhere to a set of non-functional requirements that govern its performance, scalability, security, and overall user experience.

- **Scalability and Modularity:**
 - The backend architecture must be designed as stateless, relying exclusively on external persistence layers for state management to ensure readiness for future distributed scaling.
 - System design must be modular, with clear separation of components responsibility and dependency to facilitate future enhancements, maintenance, and potential integration of additional features or third-party services.

- **Resource Optimization and Latency Minimization:**

- Operating under the constraint of localized execution, the system must optimize memory and computational overhead, loading resource-heavy models into memory only when explicitly demanded by an active workflow.
- All resource must be efficiently released immediately after use to minimize the system's memory footprint and ensure responsiveness.
- System must be scheduled to be constantly running, minimizing downtime and waiting time for any task.
- All resource must be manageable, specifically for developers and system administrators, but crucial for maintaining system stability and performance.

- **Reliability and Determinism:**

- The system must mitigate the inherent unpredictability of generative AI by enforcing strict, programmatic data schemas for all internal system communications and decision-making logic.
- Complex analytical tasks, such as calculating knowledge gaps, must be executed via deterministic programmatic logic rather than relying on probabilistic generative approximations.

- **Observability and Traceability:**

- The system must maintain comprehensive telemetry across all automated workflows.
- Database state changes, including knowledge graph updates and student progress logs, must be monitored and saved to enable historical analysis and rollback capabilities.
- Every automated decision, context retrieval action, and resource consumption metric must be actively logged to provide a transparent audit trail for both system debugging and pedagogical evaluation.

- **User Experience and Interface Requirements:**

- User interface is focus on the simplicity and intuitiveness, ensuring that users of varying technical proficiency can navigate the system effectively.
- User interface must be responsive and accessible, adhering to modern web standards to ensure compatibility across a wide range of devices and screen sizes (phones, tablets, desktops).
- Client side shouldn't be pulled by heavy backend response latency. It should maintain a fluid, real-time interaction experience, minimizing latency in response generation and interface updates. Minimizing "user experience latency" regardless of system performance is a strategic design move to preserve conversational engagement as well as learning interest.

3.3 Use Cases

3.3.1 Use case 1: Register Account

Use Case Name	Register Account
Actor	User (Student)
Description	The user registers a new account in the system to access personalized learning features.
Preconditions	The user does not already have an account with the chosen student ID/username.
Postconditions	A new user account is successfully created in the database.
Normal Flow	1. User are navigated to the registration page. 2. User enters student ID and password. 3. System validates that the student ID is not already taken. 4. System creates the account and securely stores credentials. 5. User is redirected to the login page.
Alternative Flow	Username taken: A1. User submits an already registered student ID. A2. System displays an error message asking the user to choose another ID or log in.
Exceptions	If the backend database is unreachable, show a “Service temporarily unavailable” message.

Table 3.1: *Use case 1 Overview: Register Account*

3.3.2 Use case 2: Login/Logout

Use Case Name	Login/Logout
Actor	User
Description	The user logs in to the system to access authorized features and logs out to end the session.
Preconditions	The user has a valid account. The system is connected to the authentication service.
Postconditions	User is successfully logged in (receiving a valid access token) or logged out.
Normal Flow	Login: 1. User navigates to the login page. 2. User enters username and password. 3. System validates credentials and generates JWT token. 4. User is redirected to the dashboard/homepage. Logout: 5. User clicks the logout button. 6. System clears the session token and redirects to the login page.
Alternative Flow	Invalid login attempt: A1. User submits incorrect credentials. A2. System displays an error message and asks user to retry.
Exceptions	If the authentication service is unavailable, show a “Service temporarily unavailable” message.

Table 3.2: Use case 2 Overview: Login/Logout

3.3.3 Use case 3: Manage Learning Session

Use Case Name	Manage Learning Session
Actor	User (Student)
Description	The user starts a new learning session to begin interacting with the curriculum, and ends it upon finishing.
Preconditions	The user is logged in and authenticated.
Postconditions	A unique session state is initialized or terminated for tracking the student's progress and history.
Normal Flow	Start Session: 1. User clicks to start studying. 2. System initializes a new session ID and prepares memory context. 3. System returns the session ID to the client interface. End Session: 4. User leaves the learning interface or clicks end session. 5. System clears the active session context from memory but retains history.
Alternative Flow	Session Timeout: A1. User is inactive for an extended period. A2. System automatically ends the session and prompts the user to start a new one upon return.
Exceptions	If the session database fails to respond, gracefully fall back and notify the user to refresh the page.

Table 3.3: *Use case 3 Overview: Manage Learning Session*

3.3.4 Use case 4: Ingest Course Material

Use Case Name	Ingest Course Material
Actor	Administrator / Instructor
Description	The actor uploads course slides and textbooks so the system can extract concepts and build the Educational Knowledge Graph.
Preconditions	The actor has valid administrative or instructional privileges. The raw PDF files are accessible.
Postconditions	The knowledge base (Neo4j, Milvus) is populated with parsed nodes, embeddings, and relationships.
Normal Flow	1. Actor submits the course name and target files via the ingestion endpoint. 2. System immediately returns a 200 SUCCESS status, indicating the task is running in the background. 3. System parses PDFs, segments windows, and extracts concepts (Dual-Phase Extraction). 4. System constructs the final Knowledge Graph and Vector store.
Alternative Flow	Invalid file format: A1. Actor uploads corrupted or unsupported file types. A2. System aborts ingestion and logs an error without modifying the database.
Exceptions	If the LLM provider for extraction times out, the ingestion task pauses or retries based on internal middleware logic.

Table 3.4: *Use case 4 Overview: Ingest Course Material*

3.3.5 Use case 5: Chat with TA Agent

Use Case Name	Chat with TA Agent
Actor	User (Student)
Description	The user submits a learning query to the AI Teaching Assistant and receives an educational response or pedagogical guidance.
Preconditions	The user is logged in and has an active session ID.
Postconditions	The user receives an appropriate textual answer, and UI actions (like PDF page scrolling) are triggered.
Normal Flow	1. User submits a query (e.g., “What is Machine Learning?”) in the chat interface. 2. System immediately accepts the request and returns a unique <code>task_id</code>. 3. Client continuously polls the status of the <code>task_id</code> in the background. 4. TA Agent orchestrates the response using the workflow (e.g., Teach, Retrieve, Roadmap). 5. Upon completion, the polling mechanism receives the final output (text and UI actions). 6. Client renders the response and updates the UI (e.g., highlights document).
Alternative Flow	TA Workflow Failure: - A1. The TA agent encounters an internal error during orchestration. - A2. The polling endpoint returns a ‘Fail’ status with error details. - A3. Client notifies the user that the TA encountered a problem.
Exceptions	Polling timeout: If the TA takes too long to respond (e.g., > 300s), the client aborts polling and alerts the user.

Table 3.5: *Use case 5 Overview: Chat with TA Agent*

Chapter 4

Proposed Methodology and Paradigm

Founded on the vision outlined in the requirements specification (Chapter 3), this research aims to propose a novel system that combines the strengths of classical Graph-based architectures with the generative capabilities of Large Language Models (LLMs). This chapter details the foundational methodology of our system.

4.1 Overview

The proposed paradigm can be constructed and visualized as the below pipeline

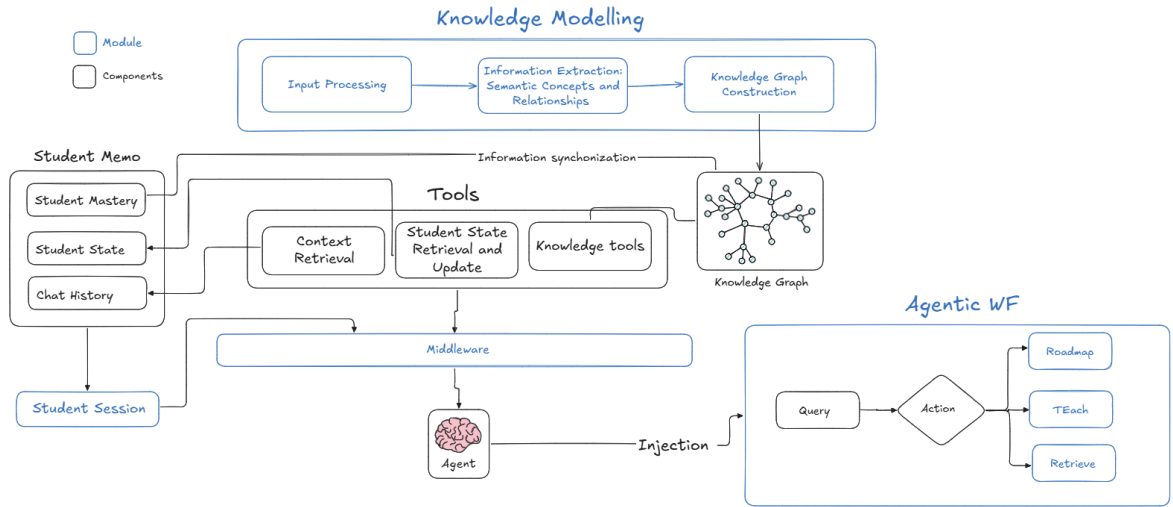


Figure 4.1: High level visualization of how the system should work

A key security boundary is established at the system's entry point. Upon user authentication, the student's Session and Memo contexts are directly injected into the computational tools. As a result, the agents use these pre-configured tools instead of interacting directly with the database. This approach ensures strict data isolation and prevents the LLM from making unauthorized or destructive database transactions.

4.2 Knowledge Modelling: Educational Graph

This section proposes a hybrid methodology that anchors the generative flexibility of LLMs to the deterministic rigor of Graph Construction and Natural Language Processing. By formulat-

ing the educational domain as a Property Graph, the proposed system (SmartEdu) achieves a necessary synthesis of semantic richness and logical consistency, overcoming the unstructured chaos often associated with pure generative models.

4.2.1 Graph Definition

To construct a graph, we need to define and regulate it with a connected, rigorous structure. Simulating educational concepts, the graph contains ontologies as well as topic's relationship like prerequisite. The subsequent contents in this subsection explain the deterministic algebra representation and rules explanation of it.

4.2.1.1 Formal Property and Topological Definition

To accurately represent the vast, interconnected semantic structure inherent in educational curricula, our methodology employs a formal Property Graph formulation. Unlike simple directed graphs, a property graph allows both vertices and edges to possess complex internal states (properties). This multidimensionality is highly suitable for educational knowledge representation, where a concept is not just a node, but a container of definitions, mathematical formulas, and historical context.

We define the Educational Knowledge Graph (EKG) as a heterogeneous, directed property graph, formally denoted as:

$$G = (V, E, \Lambda, \Pi) \quad (4.1)$$

where:

- V is the set of vertices (nodes), representing distinct educational entities spanning multiple levels of abstraction.
- $E \subseteq V \times V$ is the set of directed edges, representing the semantic and structural relationships between entities.
- Λ is a finite, predefined set of labels, acting as ontological categories that strictly classify the nodes to prevent semantic drift.
- Π is a set of properties, expressed as key-value pairs $\pi_{key} = value$, associated with both nodes and edges to store metadata, raw textual content, and dynamically updated quantitative metrics.

The necessity of this mathematical definition lies in its ability to enforce data integrity. By treating the knowledge base as a set V and a mapping E , we can apply rigorous graph-theoretic algorithms (such as topological sorting and shortest-path routing) directly to the curriculum, effectively translating human pedagogical intuition into computationally solvable problems.

4.2.1.2 Ontological Hierarchy formulation

To accurately model the multi-scalar nature of educational curricula—ranging from entire academic disciplines down to specific explanatory sentences in a textbook—we artificially constrain the label set Λ such that $\Lambda = \{\text{CommunityNode}, \text{TopicNode}, \text{ConceptNode}, \text{RhetoricalNode}\}$. This rigorous ontological taxonomy ensures structural consistency across different domains:

- **CommunityNode**: Represents the highest level of educational aggregation. It functions as the macroscopic backbone of a course or a major disciplinary cluster, grouping broad fields of study (e.g., "Computer Science", "Artificial Intelligence").
- **TopicNode**: Represents a specific subject matter, module, or chapter nested strictly within a parent community (e.g., "Machine Learning", "Neural Networks").
- **ConceptNode**: The atomic unit of educational instruction. It represents distinct, indivisible ideas, theories, algorithms, or historical events (e.g., "Backpropagation", "Decision Tree").
- **RhetoricalNode**: Encapsulates the instructional function of the raw text. It contains specific properties $\pi_{rrole} \in \Pi$ that designate the rhetorical role of the information (e.g., Definition, Application, Statement, Theorem). This allows the system to distinguish between *what* a concept is and *how* it is explained.

By enforcing this strict hierarchy, the system prevents the "flattening" of knowledge, ensuring that the AI agent can distinguish between a broad topic that requires a multi-week syllabus and a specific concept that can be explained in a single paragraph.

4.2.1.3 Edge Semantics and Constraints

The topological structure of G is governed by the relation types assigned to the edges $e \in E$, formulated as ordered pairs $e = (v_i, v_j, \lambda_e)$, where λ_e is the edge type. The system defines several core edge types to represent real-world pedagogical connections: *PART_OF*, *RELATED_TO*, and *PREREQUISITE_OF*.

The mathematical enforcement of the *PREREQUISITE_OF* edge is of paramount educational importance. In human learning, knowledge is inherently sequential; one cannot grasp Calculus without first mastering Algebra. To ensure logical learning progressions computationally, the sub-graph induced by all *PREREQUISITE_OF* edges must strictly form a Directed Acyclic Graph (DAG).

Let $G_{prereq} = (V, E_{prereq})$ where $E_{prereq} = \{e \in E \mid \lambda_e = \text{PREREQUISITE_OF}\}$. The methodology mandates that no directed cycles exist within G_{prereq} :

$$\forall v \in V, \nexists \text{ path } P = (v, \dots, v) \text{ in } G_{prereq} \quad (4.2)$$

This acyclic constraint is not merely a theoretical preference; it is a foundational mathematical requirement. If a cycle were to exist (e.g., A is a prerequisite for B, B is a prerequisite for C, and C is a prerequisite for A), the student would be permanently trapped in a circular dependency, unable to begin learning any of the concepts. Enforcing the DAG constraint guarantees that algorithms like Topological Sorting can be successfully applied to generate optimal, conflict-free learning roadmaps.

4.2.2 Graph Construction

A major challenge in building an automated knowledge base is extracting structured relationships from unstructured educational text. Single-pass LLM extractions—where the model tries

to identify concepts, classify them, and link them all at once—are extremely prone to failure. Multiple frameworks propose multiturn LLM synthesis, but costly and still not guarantee a solid, hallucination immunity graph as subsequence turns increase entropy and the context total amount. I propose a hybrid method of Dual LLM extraction combining with NLP based filtering and normalization, guarantee a consistent graph.

4.2.2.1 Input Ingestion

Before the dual-phase extraction pipeline can operate, raw educational documents must be segmented into manageable batches that respect both the language model’s context budget and the semantic continuity of the source material. The system employs an overlapping sliding window strategy that partitions each document into consecutive page batches with deliberate redundancy at their boundaries.

The process begins with a document parser that converts each PDF into a sequence of page-indexed text blocks, filtering out non-informational elements such as lecturer contact details, rendering artifacts, and fragments shorter than five characters. The resulting pages are then grouped into fixed-width batches: fifteen pages per batch for lecture slides and ten pages for textbooks (By default). These widths are calibrated to remain within the extraction model’s context capacity.

Rather than advancing the window by its full width, the system steps forward by approximately two thirds of the window size, leaving an overlap of several pages shared between consecutive batches. For instance, a fifteen-page window advances by eleven pages, so each pair of adjacent batches shares four pages of content at their boundary.

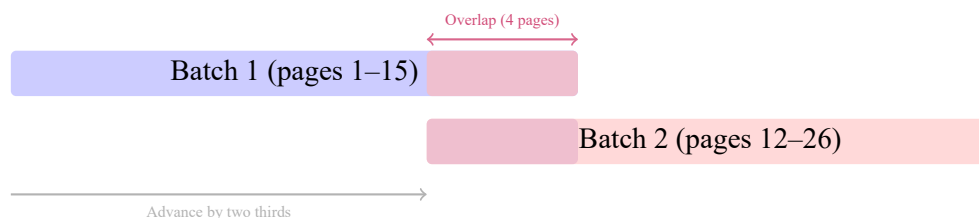


Figure 4.2: *Overlapping window segmentation. Adjacent batches share boundary pages.*

The design most important purpose is to preserve cross-boundary continuity: educational explanations that span a page break are captured intact in at least one complete batch, preventing the extraction model from receiving truncated input. This help greatly during graph construction when semantic overlap help knowledge island stay connected. As a small benefit to make up the overhead, the redundancy provides graceful degradation—if LLM encounter error when working with 1 batch, its overlapping neighbour still contains the boundary content, avoiding knowledge missing.

4.2.2.2 Dual Phase LLM Extraction

Dual Phase Extraction Pipeline is one that methodologically decouples top-down hierarchical decomposition from lateral relational mapping. Let $\mathcal{T}$ represent the continuous corpus of unstructured educational text, ingested from raw documents. The overall extraction objective is to discover an approximation function $\Phi : \mathcal{T} \rightarrow (V, E)$ that maximizes semantic fidelity. We decompose Φ into two sequential, deterministic mappings.

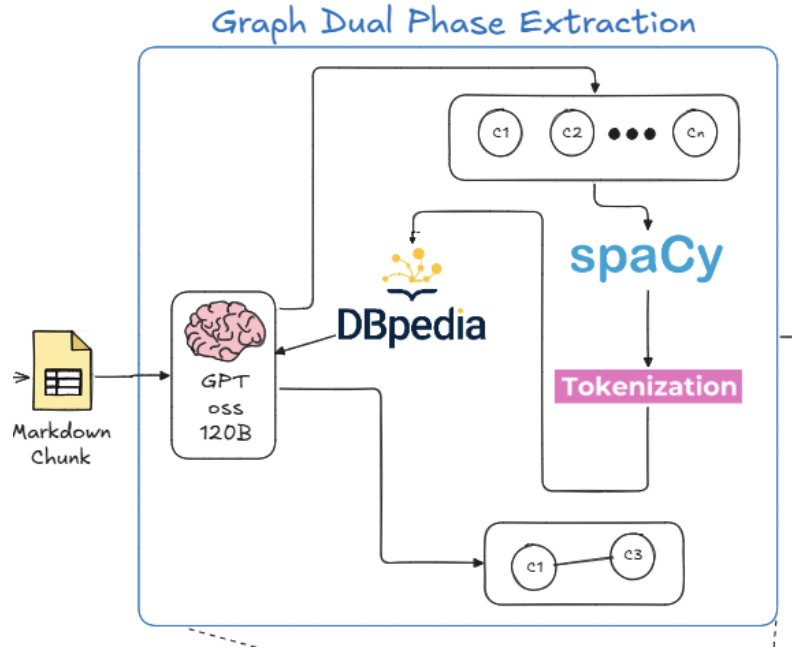


Figure 4.3: Dual Phase Extraction

Phase 1: Top-Down Hierarchical Extraction (Skeleton Phase)

The primary objective of the first phase is to establish the taxonomic structure of the knowledge domain without the cognitive burden of cross-referencing. The LLM's action space is strictly confined to identifying entities and their subsumption (parent-child) relationships.

We define the skeleton extraction function Φ_{skeleton} as:

$$\Phi_{\text{skeleton}} : \mathcal{T} \rightarrow (V_{\text{tree}}, E_{\text{tree}}) \quad (4.3)$$

where $E_{\text{tree}} \subset V_{\text{tree}} \times V_{\text{tree}}$ contains strictly hierarchical edges (e.g., *PART_OF*).

To eliminate ontological drift, the LLM is forced to output data conforming to a rigid, predefined syntactic schema (e.g., strict JSON schema validation). The system parses this structural output to instantiate CommunityNode and TopicNode entities, ensuring that every atomic concept is firmly anchored to a macroscopic curriculum node.

Mathematically, this phase yields disjoint taxonomic trees. Because the LLM is only tasked with hierarchical assignment, the spatial complexity is bounded to $O(N)$ parent-child assignments. This drastic reduction in the problem space significantly mitigates the probability of generation errors. To complete the graph, phase 2 is executed subsequently.

Phase 2: Lateral Relational Mapping (Relation Phase)

With the hierarchical skeleton established as the logically structured frame, the second phase focuses on inferring the non-hierarchical, semantic linkages across different branches of the tree. Given the isolated, immutable entity set V_{tree} , the relational extraction function Φ_{relation} is defined as:

$$\Phi_{\text{relation}} : (V_{\text{tree}}, \mathcal{T}) \rightarrow E_{\text{lateral}} \quad (4.4)$$

where E_{lateral} encompasses cross-cutting educational dependencies such as *PREREQUISITE_OF* or *RELATED_TO*.

By presenting the LLM with a predefined, validated list of concepts (V_{tree}) and tasking it only with edge generation, the problem space is reduced to a binary classification task over possible pairs. The final Educational Knowledge Graph is the formal union of these two sub-spaces:

$$G = (V_{\text{tree}}, E_{\text{tree}} \cup E_{\text{lateral}}) \quad (4.5)$$

To summarize, it is the operation of extracting potential candidate concepts as taxonomy trees and combine it to make a graph. This paradigm ensures high consistency in the generated graph. It bridges the gap between unstructured educational data and structured property graph by artificially constraining the LLM's action space, partially preventing hallucinations. By partially, there is a big problem: Phase 2 relies mainly on Phase 1 for a structured and stable output, but what if Phase 1 itself is not a good frame skeleton? To better reconstruct and verify the frame, we use deterministic algorithm and techniques

4.2.2.3 NLP-Based Concept Filtering and Verification

The idea the Dual-Phase LLM Extraction (Section 4.2.2.2) is to reduce hallucination by constraining the model's action space of subsequent step using result from previous steps as an anchor and ground truth. However the raw output of Phase 1 remains inconsistent: semantically equivalent concepts may appear under distinct lexical forms, concepts extracted might not fit the full course content and finally overly granular entities might be extracted just because it is mentioned once. In conclusion, using LLM to correct LLM has its limit. Inspired by BPE tokenization, a deterministic NLP pipeline is integrated between Phase 1 and Phase 2. This pipeline operates exclusively on the candidate node set V_{tree} and produces a refined, verified set $V^* \subseteq V_{\text{tree}}$ that is subsequently passed to the Relation Phase. It is in fact a Entity Filterer proceeding through four sequential steps.

Step 1 — Normalization and Tokenization. Each candidate concept label $v_i \in V_{\text{tree}}$ is first subjected to a standard text normalization procedure. This entails lowercasing, punctuation stripping, morphological lemmatization (reducing inflected word forms to their base lemma), and the removal of domain-agnostic stop words. The resulting form is then decomposed into a set of meaningful unigrams and bigrams, yielding a normalized token set $\mathcal{N}(v_i)$. Formally:

$$\mathcal{N}(v_i) = \text{tokenize}(\text{lemmatize}(\text{lower}(v_i))) \setminus \mathcal{S} \quad (4.6)$$

where $\mathcal{S}$ denotes the stop-word vocabulary. This normalization step establishes a canonical surface representation for every concept, ensuring that subsequent comparison operations are performed on semantically uniform token sets rather than raw, noisy strings.

Step 2 — External Wiki Validation Following normalization, each candidate concept is submitted to the DBpedia Spotlight entity linking service. For a given concept v_i , the system issues a query against the DBpedia knowledge base to retrieve a set of k candidate labels, denoted as L_k . To guarantee consistency and filter out unreliable terms, the system first identifies the most relevant label l^* from the candidate set by maximizing the relevance score:

$$l^* = \arg \max_{l \in L_k} \text{relevance}(l, v_i) \quad (4.7)$$

Once the optimal label l^* is identified, the concept is accepted and integrated into the knowledge graph only if this best match yields a positive relevance score:

$$\text{relevance}(l^*, v_i) > 0 \quad (4.8)$$

If $\text{relevance}(l^*, v_i) = 0$, it indicates that even the most probable candidate of that entity definition yields no reliable definition on DBpedia. Concepts failing this criterion are usually either hallucinated constructs or excessively fine-grained pseudo-entities that lack grounding in established encyclopedic knowledge, and are subsequently pruned from V_{tree} . This external validation step introduces a strict layer of epistemic accountability, anchoring the graph topology to verifiable, real-world semantics rather than model-generated approximations.

However, given the fact that today's researchers constantly discovering new paradigm and concepts that might have not been updated in Wiki, the system deliberately keeps it as a Leaf Node with low weight. Though the node itself alone is not reliable, if its surrounding nodes are meaningful, its meaning can also be implicitly derived from the graph structure, proposing a novel, robust architecture to new paradigm and innovation.

Step 3 — Implicit Clustering Upon completion of filtering, the retained concepts undergo pairwise token similarity analysis. The Jaccard similarity between two concept representations is computed as:

$$J(v_i, v_j) = \frac{|\mathcal{N}(v_i) \cap \mathcal{N}(v_j)|}{|\mathcal{N}(v_i) \cup \mathcal{N}(v_j)|} \quad (4.9)$$

This measurement serves two complementary functions. First, concept pairs satisfying $J(v_i, v_j) \geq \tau_{\text{merge}}$ are treated as near-duplicates and consolidated into a single canonical node, thereby eliminating redundancy introduced by paraphrastic variation in the source text. Second, and more significantly, concepts that share a non-trivial common token subset — without necessarily crossing the merge threshold — are grouped into an implicit cluster. The intersection $\mathcal{N}(v_i) \cap \mathcal{N}(v_j)$ is used to infer a shared parent topic label; for instance, the concepts “Machine Learning Algorithms” and “Machine Learning Optimization” both yield the shared token “Machine Learning”, from which a TopicNode bearing that label is derived. These implicitly discovered topic nodes are then cross-validated against the hierarchical skeleton produced by Phase 1 to either confirm existing TopicNode entries or surface structural gaps in the original extraction.

Step 4 — Node Caching. Once a concept node has successfully traversed Steps 1 through 3 — normalized, externally validated, and cluster-assigned — its canonical representation is persisted in a key-value cache, indexed by its normalized token set $\mathcal{N}(v_i)$. This cache serves three purposes. First, it ensures idempotency: if the same concept surface form is encountered across multiple ingested documents, it resolves deterministically to the same canonical node without re-executing the NLP pipeline. Second, it supports incremental graph construction: as new educational materials are added to the corpus over time, only genuinely novel concepts need to traverse the full verification pipeline, while previously verified nodes are retrieved directly from cache. Third, it provides the Relation Phase (Phase 2) with a stable, pre-validated entity vocabulary, eliminating the need for redundant lookups during edge generation. The final verified and deduplicated set V^* is then passed forward as the immutable input to Φ_{relation} .

4.3 Teaching Assistance

The knowledge graph G serves as the static, structural foundation of the system. However, real-time interaction with a human learner demands a dynamic, adaptive reasoning engine capable of navigating G to produce personalized educational responses. A single monolithic language model, no matter how capable, cannot reliably handle the diverse reasoning tasks involved in tutoring—from classifying student intent, to retrieving factual content with minimal cost and latency, to generating structured lectures, to evaluating learning outcomes. Each of these tasks requires a different workflow design.

To address this challenge, the proposed system decomposes the tutoring process into a multi-agent workflow, where each reasoning step is handled by a dedicated agent node with its own tailored prompt, tools, and output schema. The following subsections describe the core architectural decisions that make this decomposition both practical and effective: its applied harnessing as well as workflow design.

4.3.1 Agent Harnessing

Rather than relying on a single, prompt-heavy LLM to manage all tutoring logic, the Teaching Assistant is structured as a collection of specialized agent nodes coordinated through what we term an Agent Harness. This term was officially recognized worldwide this year, referring to all the infrastructure and pipeline revolves around the LLM. My proposed harnessing techniques govern how each node receives its instructions, what context it can access, and what format its output must follow. The design philosophy, from this point called the "Single Paradigm", is straightforward—each agent node should have exactly one responsibility tied to one instruction set, one consistent context, and exactly one input/output structure/schema definition.

Chat Summarization and Memory Management

As a tutoring session progresses, raw chat history accumulates rapidly—exhausting the model's context window and degrading reasoning quality. The system addresses this with a sliding-window memorization mechanism: each turn is condensed into a structured triplet (query → thought → answer), retaining only 8 entries at any time.

This history is exposed selectively: lightweight classification nodes (e.g., the intent router) receive only heading summaries (skim mode), while nodes requiring deeper context—such as lecture generation or evaluation—receive the full summarized messages.

4.3.1.1 Tool Abstraction Design

Direct LLM access to the system's databases (Neo4j, Milvus, MongoDB, MinIO) risks malformed queries, unauthorized access, and destructive writes. A tool abstraction layer therefore sits between each agent and the underlying infrastructure: each tool exposes exactly one well-defined function, and agents have no direct access to raw query languages.

These tools are pre-configured with the student's session context before any agent invocation, making each call a simple, self-contained operation from the agent's perspective. This design simultaneously simplifies the agent's decision space and enforces strict data isolation across sessions.

4.3.1.2 State—based Harnessing

Even with many traditional context optimization techniques, agents still prone to context overhead when too many task instruction and tools metadata are injected, weaken their performance by miles and increase hallucination. The Dynamic Harness resolves this through state-dependent context injection: each workflow node receives its own dynamically populated prompt at runtime. Retrieval nodes get only retrieval-specific instructions; evaluation nodes receive evaluation criteria; synthesis nodes additionally receive upstream reasoning from prior nodes—yielding a minimal, purpose-built context window per node that collectively produces coherent multi-step reasoning.

Achieving this per-node dynamism requires intercepting agent execution at each workflow step—something standard frameworks (LangChain, CrewAI, AutoGen) do not natively support, as they fix agent configuration at initialization. The solution proposed is the *Adaptive Middleware*, which maintains an internal registry of workflow node names. When invoked at a node, it inspects the current node identifier, retrieves its registered harnessing protocol, and reconstructs the agent's configuration before execution.

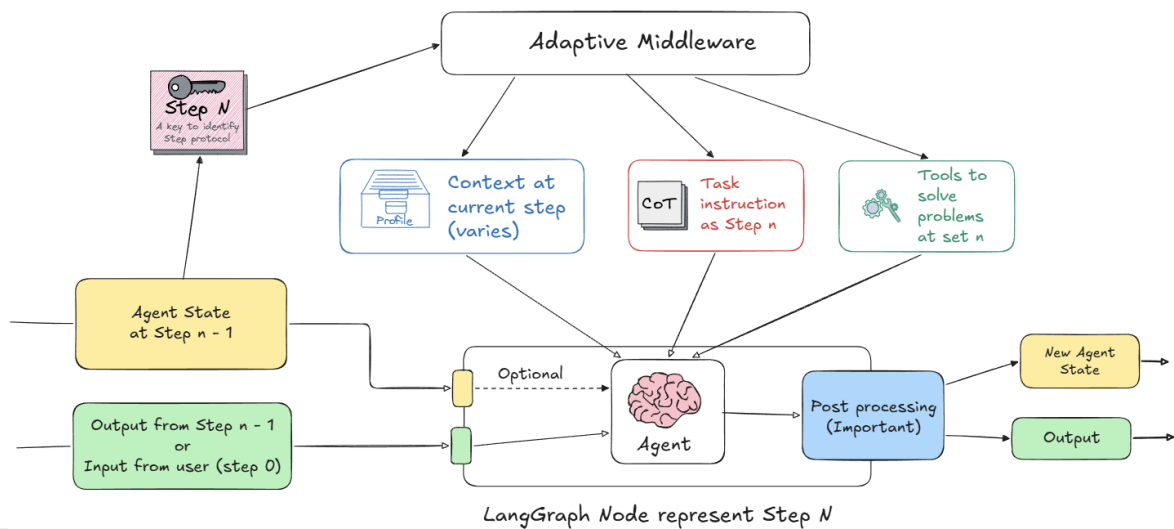


Figure 4.4: How agents's context and behaviour are constructed in a workflow node on a high level

This design offers 4 novel benefits:

1. Single Paradigm is applied when a workflow node is registered with its own harnessing protocol, preventing the confusion that arises when a model must choose among multiple possible reasoning paths, requirements, and methods.
2. Because the middleware handles schema routing automatically, the prompt templates for each node can focus exclusively on the reasoning task at hand, without needing to include format-selection instructions.
3. By constraining the output space to a predefined set, the system significantly reduces the probability of the model generating structurally invalid or semantically drifted responses. Each agent node operates within a tightly bounded action space—one task, one context, one schema—which collectively yields a more reliable and debuggable multi-step reasoning pipeline.
4. With the registry techniques, agents are not hardcoded in nodes during initialization. This allows easy behaviour debug from the central registry, making agentic design more flexible and scalable.

4.3.1.3 Agentic Full Design

Figure 4.5 visualized a full harnessing of an agent, as well as its injection toward the workflow nodes to perform its task in a large LangGraph workflow, while table 4.1 show us actionable protocols provided for agents in the form of Tools.

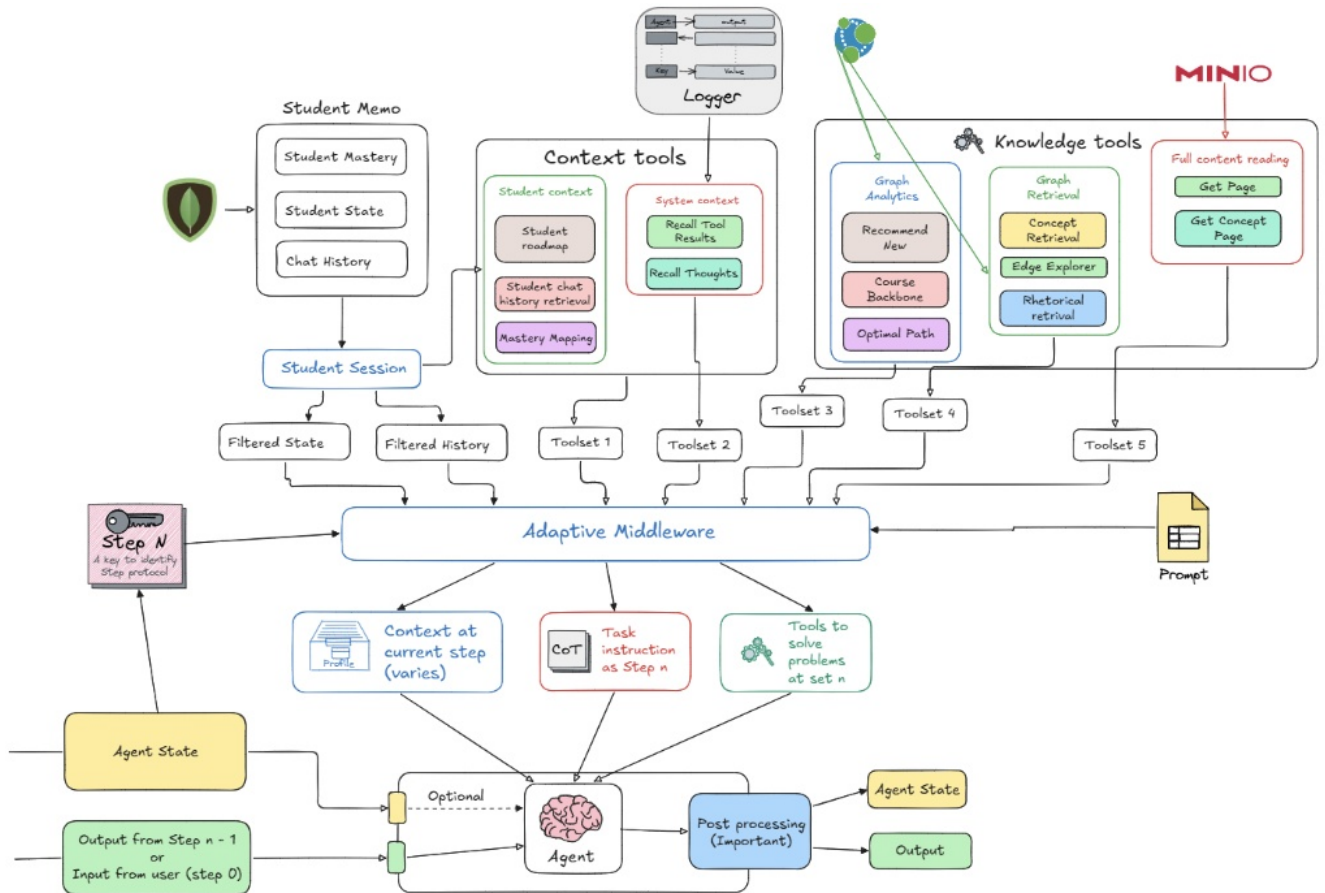


Figure 4.5: Agent Harness architecture demonstrating the Adaptive Harnessing based on Middleware.

Tool Name	Group	Description	Parameters	Database
Context tools				
Student roadmap	Student context	Retrieve current student learning path and progress.	session_id	MongoDB
Student chat history retrieval	Student context	Retrieve recent conversation history for context injection.	session_id, limit	MongoDB
Mastery Mapping	Student context	Fetch student mastery scores for concepts.	session_id	MongoDB
Recall Tool Results	System context	Retrieve recent tool execution results to prevent redundant calls.	limit, agent_type	Logger (In-memory)
Recall Thoughts	System context	Retrieve recent agent reasoning traces for self-correction.	limit	Logger (In-memory)
Knowledge tools				
Recommend New	Graph Analytics	Find top hub nodes ranked by out-degree for a starting point.	course_filter, from_node	Neo4j
Course Backbone	Graph Analytics	Extract course skeletal structure via top hub concepts and relations.	course_name, max_hubs	Neo4j
Optimal Path	Graph Analytics	Find the shortest weighted path between two concepts.	start_node, end_node	Neo4j
Concept Retrieval	Graph Retrieval	Resolve entity names to canonical IDs using graph or Wikidata.	query	Neo4j
Edge Explorer	Graph Retrieval	Explore multi-hop semantic relationships (excluding CONTENT edges).	node_id	Neo4j
Rhetorical retrieval	Graph Retrieval	Retrieve educational content nodes by rhetorical role.	node_id, role, limit	Neo4j
Get Page	Full content reading	Extract literal text from specific pages of a course document.	pages, destination	MinIO
Get Concept Page	Full content reading	Quickly locate page numbers referencing a specific concept.	concept	Neo4j

Table 4.1: *Summary of tools available to the Teaching Assistant agent.*

4.3.2 Agentic Workflow

The Teaching Assistant’s reasoning process is organized as a hierarchical state machine. At the top level, a router node classifies each incoming student query into one of five intent categories and dispatches it to the corresponding sub-workflow. Each sub-workflow is a directed graph of specialized nodes, where each node performs a single reasoning or data-retrieval step. Upon completion, every sub-workflow passes its results to a synthesis node that formats the final student-facing response and persists the session state. Figure 4.6 illustrates this top-level architecture.

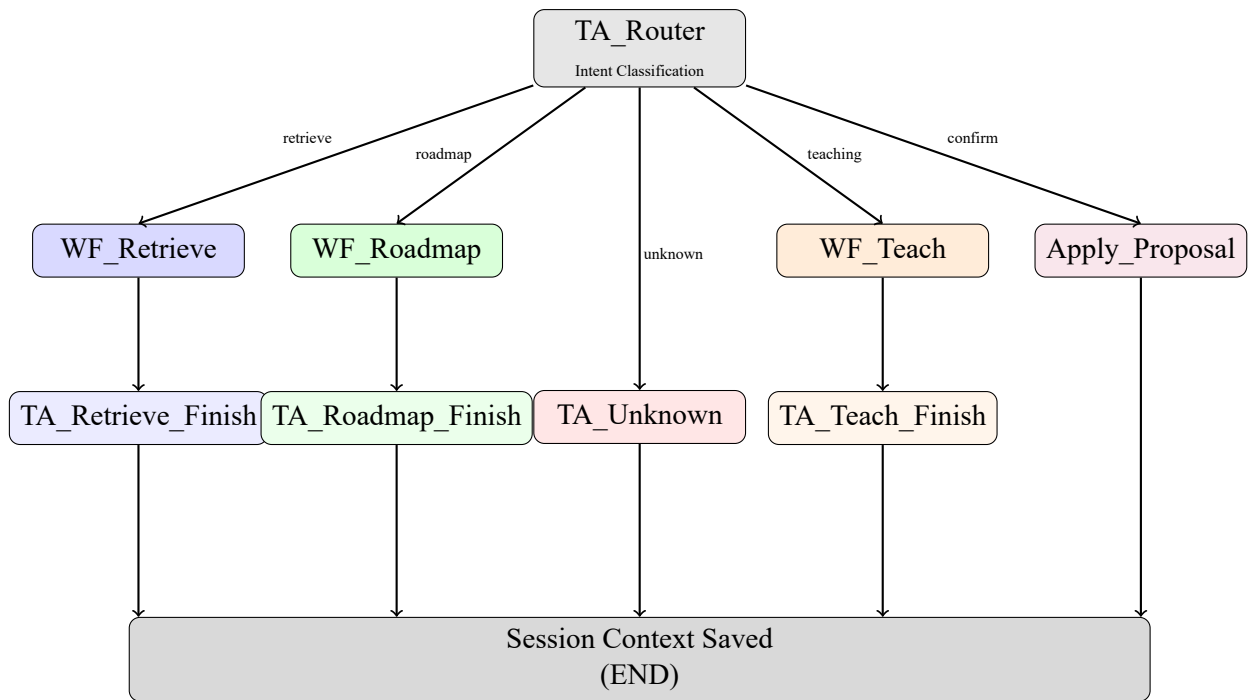


Figure 4.6: Top-level workflow routing based on user intents and context.

4.3.2.1 Intent Router

Every query must be dispatched to the correct sub-workflow before reasoning begins—misrouting wastes computation and produces structurally wrong responses. The router receives three inputs: (i) the student’s current learning state, (ii) a skim-mode chat summary, and (iii) the raw query—together disambiguating queries that appear identical but carry different intents across session contexts.

The prompting of the router goes for a straight forward few-shot classification: boundary-case examples for each routing choices. The output is a single constrained token from { retrieve, roadmap, teaching, confirm, unknown} at temperature zero, with no tool invocation. This remove any time-cost reasoning chain, maximize speed and focus straight on real user input and well defined guildlines.

4.3.2.2 Workflow: Roadmap

The Roadmap workflow addresses the planning problem: what should the student learn next, and in what order? Unlike factual retrieval, this requires structural graph reasoning—the system must evaluate not only what exists in the knowledge base, but which concepts are most foundational, which the student has already mastered, and in what topological order they should be

encountered. All structural analyses in this workflow consistently consider only semantic edges (prerequisite, hierarchical, associative) and explicitly exclude content-bearing edges, ensuring that centrality measures and path algorithms reflect prerequisite dependency rather than rhetorical attachment.

Out-Degree Centrality and Backbone Identification. The knowledge graph’s density creates a navigation challenge—without a principled ranking, any learning path risks overwhelming the student or selecting concepts arbitrarily. The system addresses this with Out-Degree Centrality as a structural heuristic. Heavily inspired by [3], this is actually an algorithm to detect candidate central concept. Since the analytics is done on one type of edge, I propose a simplified version, ensured to be a built-in feature in Database. For a given node v , the semantic out-degree $C_{deg}(v)$ counts the number of outgoing semantic edges:

$$C_{deg}(v) = \sum_{\substack{w \in V \\ (v,w) \in E_{sem}}} 1 \quad (4.10)$$

where E_{sem} excludes content-bearing edges. High- C_{deg} nodes serve as structural anchors and form the course backbone. This centrality measure is reused uniformly across all three sub-workflows: path planning (Roadmap), gap detection (Retrieve), and next-concept selection (Teach).

Mastery State and the Knowledge Gap. The student’s proficiency is represented as a mastery function $\mu : V \rightarrow [0, 1]$:

$$\mu(v) = \begin{cases} s_v, & \text{if } v \in V_{explored} \\ 0, & \text{otherwise} \end{cases} \quad (4.11)$$

where $s_v \in [0, 1]$ is the recorded Bloom’s taxonomy mastery score for explored nodes, defaulting to zero otherwise. For any proposed backbone node b , the set of unmastered prerequisites is:

$$\Delta(b) = \{u \in \mathcal{P}(b) \mid \mu(u) < \tau\} \quad (4.12)$$

where $\mathcal{P}(b)$ is the immediate prerequisite set of b and τ is the mastery threshold. A non-empty $\Delta(b)$ signals a knowledge gap that must be addressed before b is reachable.

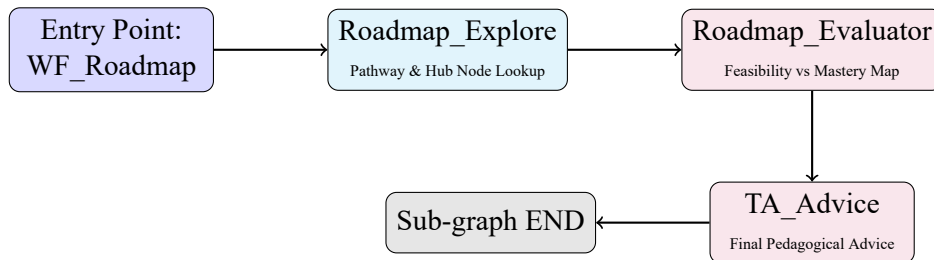


Figure 4.7: Roadmap sub-graph.

Given the student’s current position v_{cur} , the system identifies the course backbone—its most foundationally connected concepts, ranked by C_{deg} —and computes the shortest prerequisite path

to each backbone hub b_i over the prerequisite sub-graph:

$$\text{Path}(v_{\text{cur}}, b_i) = [v_1, v_2, \dots, v_k] \quad (4.13)$$

These paths are merged and topologically sorted into a candidate list $\mathcal{L}$. To keep the result navigable, a sliding window retains only the top- k nodes by C_{deg} per segment $W_i \subset \mathcal{L}$:

$$\text{Selected}(W_i) = \text{top-k}(W_i, C_{deg}) \quad (4.14)$$

yielding a condensed path where prerequisites always precede the concepts that depend on them:

$$P_{\text{roadmap}} = \text{Selected}(W_1) \oplus \text{Selected}(W_2) \oplus \dots \oplus \text{Selected}(W_m) \quad (4.15)$$

Before the proposal reaches the student, it passes through a *critic-actor* pass: a dedicated evaluator cross-references each backbone node against μ , checking for unresolved gaps $\Delta(b) \neq \emptyset$, gatekeeper necessity, and cognitive load across the sequence. Its structured critique is then reconciled by an advisory agent into actionable guidance. Crucially, the resulting *Learning Proposal* requires explicit student confirmation before modifying session state—unlike the auto-applied bridge proposals of the Retrieve workflow. Changing an entire learning trajectory is a high-stakes decision; the system is designed to inform and consent, not act unilaterally.

4.3.2.3 Workflow: Retrieve

The Retrieve workflow answers factual questions by navigating the Educational Knowledge Graph. The workflow separates retrieval into two tiers—a fast, bounded lookup that satisfies most queries, and an optional deep structural analysis that activates only when a significant knowledge gap is detected.

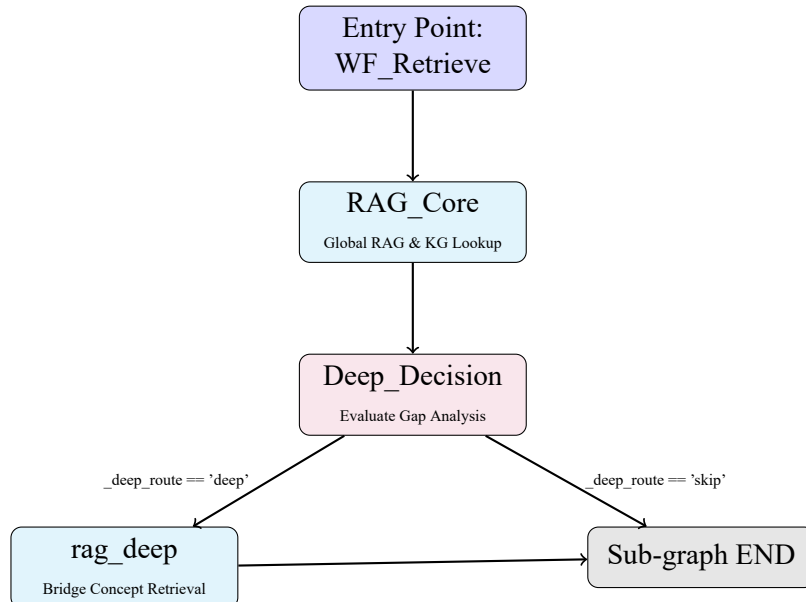


Figure 4.8: Retrieval sub-graph

The two-tier design reflects a deliberate efficiency-first principle: the majority of factual queries are definitional and can be resolved with a single graph traversal—resolving the concept to a canonical entity and fetching its rhetorical content (definition, formula, theorem, example). Forcing deep structural analysis on every query would inflate latency without benefit. A dedicated no-tool classifier therefore makes a binary routing decision after the initial lookup: simple

queries exit directly to synthesis, while relational queries (*how does X relate to Y*) activate the deep tier.

When deep analysis is triggered, the workflow shifts from content retrieval to *structural gap bridging*: it traverses multi-hop semantic relationships to identify prerequisite nodes on the path between the student's current position and the queried concept, ranked by C_{deg} . A knowledge gap score $\gamma \in [0, 1]$ quantifies the structural distance, and identified bridge concepts are packaged into a proposal that is applied automatically—reflecting the lower stakes of a targeted path adjustment compared to a full roadmap change.

4.3.2.4 Workflow: Teach

The Teach workflow manages the continuous learning loop, delivering lectures, assessing mastery, and advancing the student's knowledge state through iterative interaction.

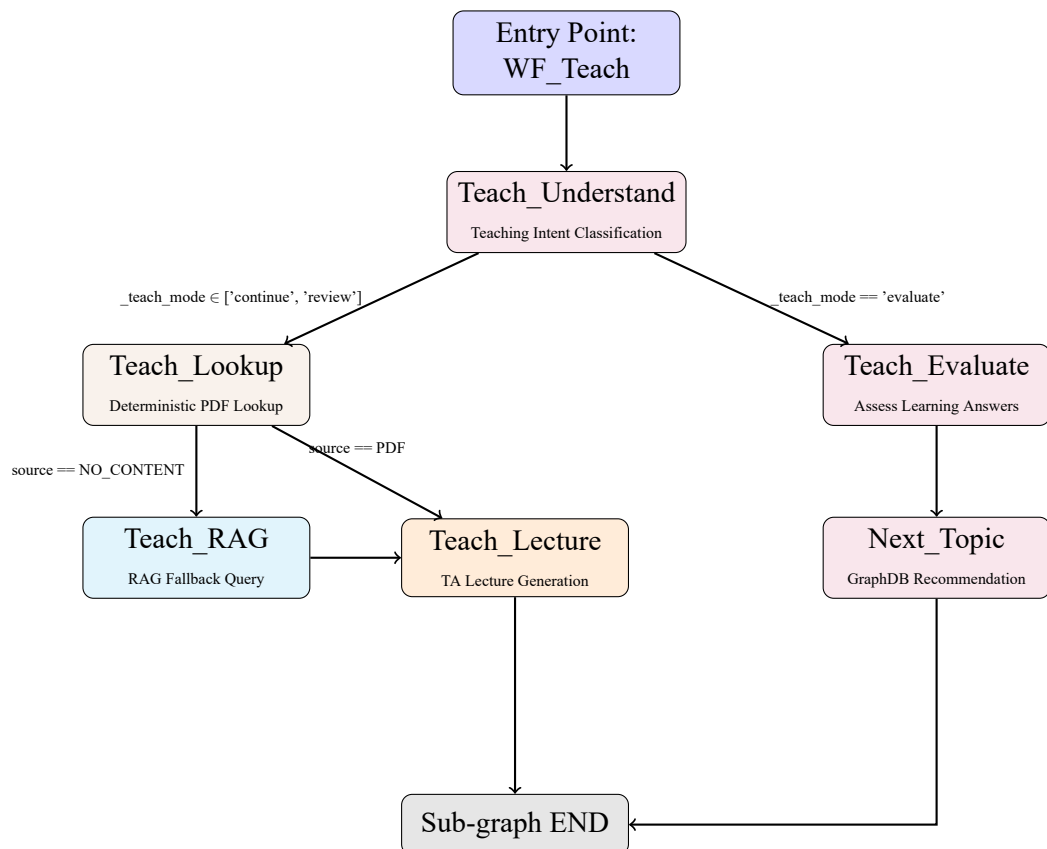


Figure 4.9: Teaching sub-graph flow: content lookup, RAG fallback, and adaptive topic progression.

The central design philosophy is a *Socratic loop*: deliver content, elicit a response, assess understanding, and either advance the student's position in the EKG or reinforce the current concept. A fine-grained entry classifier distinguishes three teaching modes—*continue* (advance), *review* (revisit), and *evaluate* (assess)—ensuring the same surface query maps to the correct instructional behavior given the session state.

Content collection for lecture generation uses an iterative Reason-Act (ReAct) loop rather than a single retrieval call, allowing the agent to reason about what material is still needed and continue calling tools until the context $\mathcal{C}$ is sufficient (Figure 4.10). Two tools drive this loop: `get_concept` resolves the current node to a structured document reference (ID, MinIO path, page number), and `get_pages` extracts the corresponding PDF pages; if no direct reference exists, the flow falls back to the Retrieve pipeline.

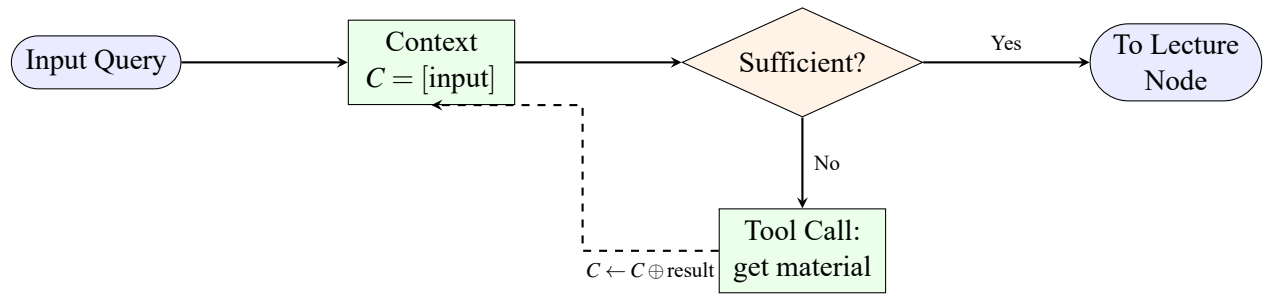


Figure 4.10: *ReAct loop in Teach Lookup.*

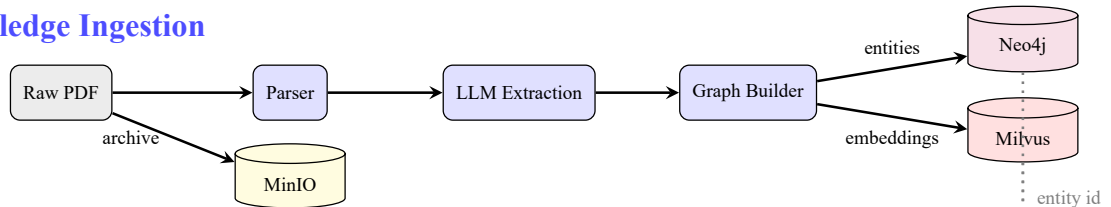
Lecture delivery is calibrated by the student’s Bloom’s taxonomy level: lower mastery receives definitions and grounded examples; higher mastery receives application problems and synthesis challenges. A closing challenge question is posed in *continue* mode to close the teaching moment, while *review* mode poses targeted recall questions over recently covered concepts. Assessment is deliberately tool-free—the agent judges mastery from conversational evidence alone, distinguishing genuine conceptual understanding from surface-level recall. The binary verdict then drives the session forward: a passing judgment triggers an auto-applied *advance* proposal that moves the student’s graph position to a high-centrality neighboring concept ranked by C_{deg} ; a failing judgment returns a STAY action, preserving the current position for reinforcement.

Chapter 5

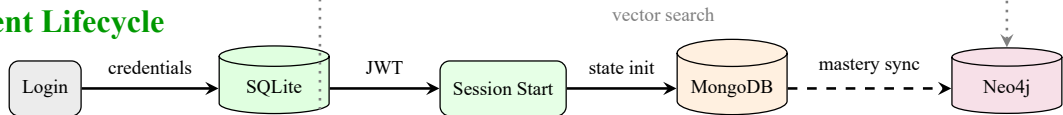
System design and implementation

This chapter presents the concrete design and implementation of the SmartEdu system, translating the methodology outlined in Chapter 4 into a functioning software implementation. Such implementation are generalized in Figure 5.1, an illustration of three principal flow paths through the system, functioning as a complete summarization of the operational pipeline.

Knowledge Ingestion



Student Lifecycle



Runtime Interaction

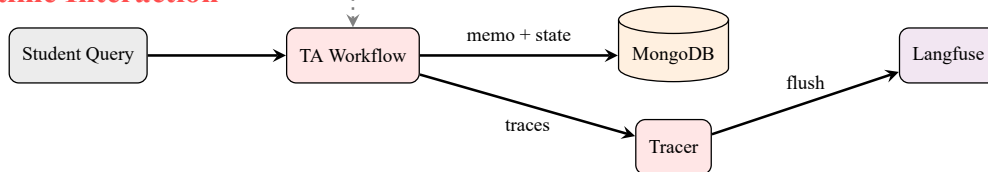


Figure 5.1: System general pipeline flow. Arrows show read - write operation direction

As illustrated in Figure 5.1, the system modules share data and state information across different pipelines utilizing common databases. This choice is necessitated by storage limits and hardware resource constraints. However, granting multiple modules access to the same storage engines introduces significant synchronization risks. To mitigate these risks, the architectural design enforces an universal constraint across the system: While modules shares common infrastructures, they perform internal logical operations in strict isolation. No logical module is allowed to directly invoke or interfere with the logic of another, maintaining clean boundaries through dependency inversion. Modules cannot override other module results, or have effect in any way possible.

In the pursuit of implementing a robust system that satisfies these constraints, the subse-

quent sections present the optimal implementations, design paradigms, and structural layouts covering the following aspects:

I. System Architecture and Design Patterns: A high-level description of the system's architecture, components, their responsibilities and interactions. It underlies design choices and principles that ensure modularity, scalability, and maintainability.

II. Core Module: The foundational utility supporting all other modules, including LLM inference engine management, database clients, centralized configuration handling, and shared data schemas.

III. Database Design: An overview of the polyglot persistence DB design, database cross reference and detailed explanation of the "One-Write" policy solving the constraint.

IV. Student Module: The implementation of authentication, session management, and the Student Tracker façade, which abstracts away persistence details while enforcing strict privacy.

V. Knowledge Module: The offline ingestion pipeline responsible for constructing the educational knowledge graph from raw course materials, covering extraction, transformation, and load operations.

VI. Teaching Assistant (TA) Module: Explain how LangGraph framework are used in the implementation of agentic workflow.

5.1 System Architecture

5.1.1 Overall Architecture and Module Responsibilities

The SmartEdu backend is a monolithic FastAPI application composed of five functionally distinct, loosely coupled modules. This Modular Monolith design was deliberately architected such that: consolidating modules into a single process eliminates network-induced latency between the components of the multi-step agentic workflow, while the strict dependency inversion between modules (discussed in Section 3.2) ensures that each module can be extracted into an independent microservice with no changes to its internal business logic.

Table 5.1: *System Module Summary*

Module	Primary Responsibility	Key Classes
core	Shared infrastructure: LLM engine, all DB repository clients, configuration, shared schemas.	CoreLLMEngine, GraphDB, MilvusDB, MinioDB
student	Authentication, session life-cycle, unified learning-state façade.	Student_Tracker, StudentSession, Memo
knowledge	Offline EKG construction pipeline from raw course materials.	KnowledgeModule, CourseIngestionService
TA	Real-time agentic workflow; generates pedagogically grounded responses.	TAModule, SmartEdu, AgentInjector
tracing	Per-session observability; async trace buffering and Langfuse flush.	AgentTracer, TraceWriter

The inter-module dependency graph is strictly acyclic: core is the sole shared foundation depended on by all other modules; student, knowledge, and TA are peer modules that do not import one another; tracing is consumed exclusively by TA. This topology is illustrated in Figure 5.2.

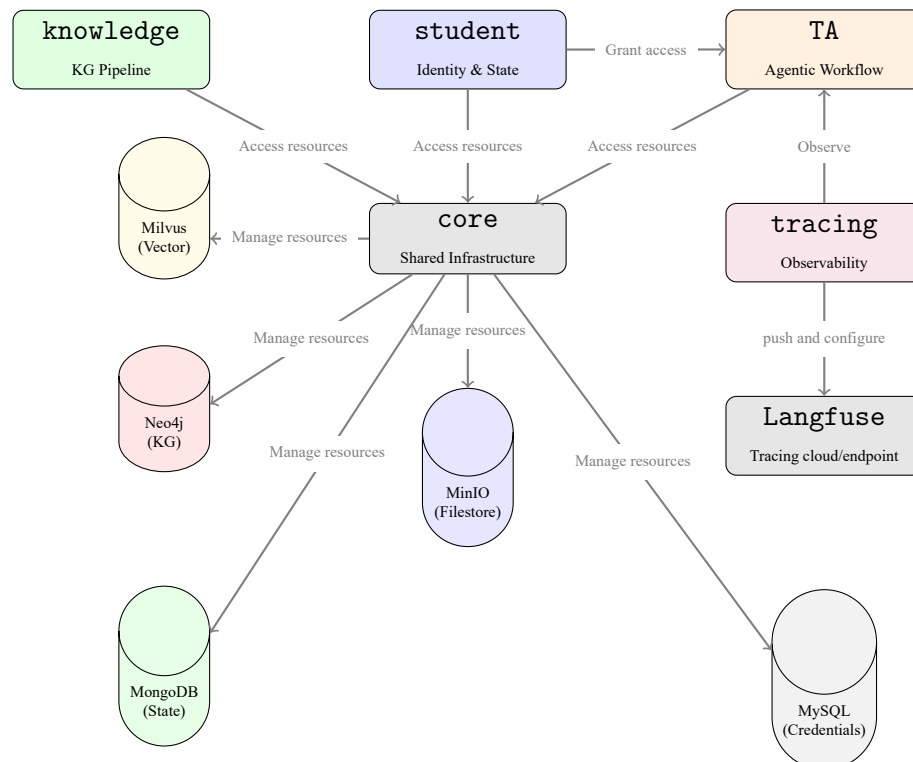


Figure 5.2: *Modular Architecture Overview of SmartEdu*

5.1.2 Technical Setup: Docker Infrastructure

The system's five database engines and their supporting services are deployed as Docker containers, orchestrated through a modular Docker Compose configuration. A root compose file

includes per-engine definition files, each specifying one service and its dependencies. All containers communicate over a shared bridge network.

Table 5.2: *Docker Compose service summary.*

Service	Image	Memory	Ports	Role
neo4j-core	neo4j:2025-enterprise	4 GB	7474, 7687	Knowledge and Student graph databases with APOC and GDS plugins
mongo-core	mongo:latest	2 GB	27018	Student state, chat memo, and session context persistence
milvus-standalone	milvusdb/milvus:v2.4.0	2.5 GB	19530	Vector similarity search engine for concept embeddings
minio-core	minio/minio:latest	1.5 GB	9000, 9001	S3-compatible object storage for PDF materials and text chunks
etcd	coreos/etcd:v3.5.5	—	—	Metadata store required by Milvus for distributed coordination
attu	zilliz/attu:latest	512 MB	8000	Web-based GUI for Milvus collection inspection and debugging
dbgate	dbgate/dbgate	—	3000	Unified database administration interface for MongoDB

The root compose file uses the “include” directive to import per-service YAML definitions, keeping each engine’s configuration isolated and independently version-controlled. All services join a single shared bridge network, enabling inter-container communication by container name. Health checks are configured for critical services (Neo4j, MinIO, etcd) to ensure that dependent containers only start after their prerequisites are verified as operational.

5.1.3 Design Paradigm

5.1.3.1 Singleton

Common system design faces a central engineering challenges: expensive, long-lived resources — LLM model instances and database connection pools — are costly operation. Constructing these resources per-request would incur prohibitive latency and memory exhaustion. Optimiza-

tion for a smooth system requires centralized resource management systems instead of reconstructing stateless models and connections over and over across code base.

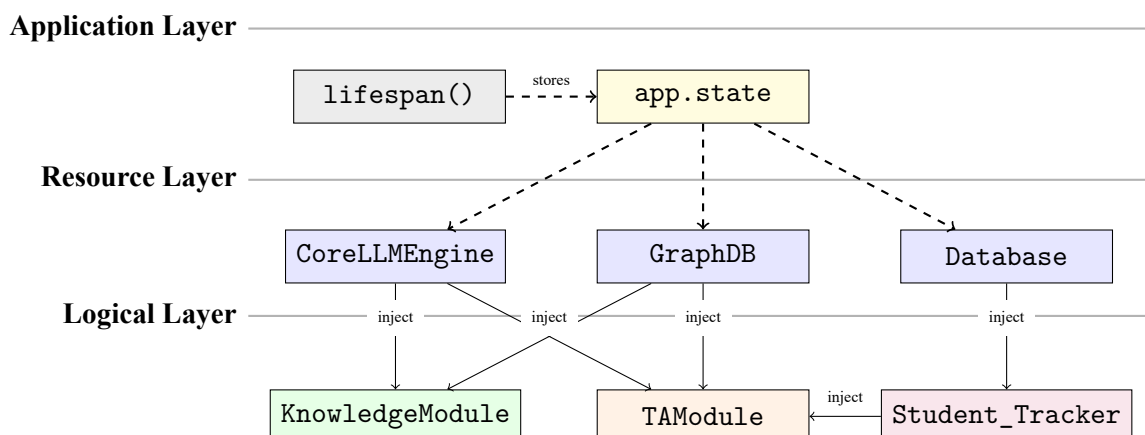


Figure 5.3: Ownership graph in lifespan. Shared resources are injected into domain modules

The solution lies in Figure 5.3, illustrating the construction of application and initialization of subsequence resources and logic modules. The app lifecycle and its modules are managed through FastAPI's lifespan context manager (`core/api/life_span.py`), which implements the Singleton pattern at the application scope. During startup, lifespan performs sequentially these actions:

1. Configuration import: at the start, all configuration (database URIs, model names, API keys) is imported and centralized at the startup. This guarantee that the entire application operates on a unified configuration for its entire lifetime.
2. Resources initialization: Resource managers receive configs to construct itself with completed methods and configs needed to init and manage resource instances like models, connection pools.
3. Life span allocates these managers on `app.state`, global and unique across the whole app. This behaviour guarantee: resources are easiliy accessed, and no matter what logic below, the resources cannot be duplicated. All references will be guaranteed to be to the global instances, not copies.
4. On shutdown, it performs explicit disconnection and graceful resource realeases. Since all resources managers are Singleton directly managed by lifespan, the system ensures no garbage instances or connection left when the app is down, leaving no burden to our hardware.

5.1.3.2 Dependency Injection

To further enhance FastAPI backend capability and Singleton Paradigm, the system proposes a dedicated Dependency Injection (DI) layer in `core/dependencies.py`. This file defines thin provider functions — one per injectable resource — that each return the corresponding singleton from `app.state`. Route handlers declare their dependencies as typed parameters annotated with `Depends()`, and FastAPI's IoC container resolves and injects the correct costly instance at request time.

This indirection yields 4 engineering benefits:

- Decoupling of business logic: Route handlers are pure functions of their typed inputs, completely unaware of how their dependencies are constructed.
- Independence across all modules: The injection allows no import needed. Therefore subsequence modules cannot import methods to create duplicates, further strengthen Singleton Paradigm, and also avoid circular dependency caused by tangled import of shared components. It allows isolated refactorization or debug of functionality.
- Flexible implementations: Replacing even the most sensitive backbone infrastructure requires modifying only the provider function, and changes are effective across all systems.
- Testability: The test suite overrides any provider via `app.dependency_overrides`, enabling full unit testing without live database or LLM connections.

5.1.3.3 MVC-Aligned Module Structure

The system is designed to support complex stateful workflows while remaining strictly modularity. Each domain module follows an MVC-aligned internal layout that enforces a clean separation of concerns:

- Model (Schema): Pydantic models defined in `core/schema/` act as the system-wide data contract. No module constructs ad-hoc dictionaries to communicate state; all inter-module data is typed and validated through shared classes — for example, `ConceptNode` and `EduEdge` (graph ontology shared between Knowledge and TA modules), `KG_Instance` (Knowledge → Core contract), and `LearningProposal` (TA → Student contract). This shared schema layer is what makes each module independently testable and replaceable.
- Controller (API Router): Each module exposes its functionality through a dedicated FastAPI router (e.g., `TA/api/route.py`). The router handles only HTTP-level concerns: parsing requests, invoking the service layer, and serializing responses. It contains no business logic.
- Service (Domain Module): Domain classes (`TAModule`, `KnowledgeModule`) encapsulate all business logic. They receive infrastructure dependencies exclusively via constructor injection and communicate with the controller only through shared Pydantic schemas.

This strict layering has a crucial architectural consequence: because no module's business logic contains direct imports of another module's internal classes — only shared schemas from core — each module can be extracted into an independent microservice by simply changing its entry point. The DI mechanism is replaced by an HTTP client call, and the Pydantic schemas serve as the API contract, with zero modification to domain logic. This makes the system scalable to microservices by architectural design.

5.2 Core Module

The core package is the dependency foundation of the entire system. It owns no domain business logic; instead, it supplies the primitive, reusable services and shared data contracts upon which all domain modules are built. As established in Section 5.1.3.1, all core resources are constructed once during the lifespan phase and injected by reference.

5.2.1 LLM Engine

CoreLLMEngine is the central gateway for all LLM interactions. It implements the Multiton pattern — a generalization of Singleton where a fixed, named pool of instances is managed. Internally it maintains a named registry of model connectors keyed by a profile name. When an agent issues a model retrieval request, the engine performs a cache lookup: if the profile is absent, it reads the corresponding LLMProfile from `core/llm/config.py`, constructs a model connector object, stores it in the registry, and returns it; subsequent calls for the same profile return the cached object directly without reconstruction.

Lazy Instantiation and GPU Memory Lifecycle

A critical property of this design is its zero-cost initialization. Upon construction, the engine’s internal registry is empty — no model weights are loaded into GPU memory at application startup. A model connector is instantiated only on the first retrieval request for a given profile, and only that specific model is loaded at that moment. This ensures that GPU memory is occupied exclusively by models that have been actively requested.

Memory retention after a request is governed by an idle retention timeout configured per profile. This value is forwarded directly to the backend runtime, which holds the model’s weights in GPU memory for the specified duration after the last inference call. Once the window expires, the runtime evicts the weights, freeing resources for other workloads. The registered connector object persists in the cache; the next invocation transparently triggers an automatic weight reload by the runtime. Profiles without an explicit timeout inherit the runtime’s global default retention policy.

The pairing of lazy instantiation with per-profile retention windows creates a tiered memory strategy: high-frequency agents are kept warm for the full duration of a tutoring session, moderate-frequency agents are evicted after a shorter idle period, and lightweight classification workers are evicted aggressively since their small size makes cold-start reloads negligibly cheap.

Profile Configuration

Each LLMProfile dataclass encapsulates the full operational parameter set for one agent role. Table 5.3 lists the six profiles defined in the current deployment, along with the rationale for each configuration choice.

Table 5.3: *LLM Profile configurations and their engineering rationale.*

Profile	Model	Temp	Context (tok)	GPU Retention	Role and Rationale
graph	gpt-oss:120b-cloud	0.2	8 192	—	Knowledge graph extraction. Large context window accommodates full slide batches; low temperature enforces deterministic structured JSON output.
ta	gpt-oss:120b-cloud	0.67	4 096	1 h	Conversational tutor. Higher temperature yields more natural dialogue; kept hot in GPU memory for the full duration of a tutoring session.
rag	qwen3:8b	0.0	2 048	30 min	Retrieval reasoning agent. Temperature zero ensures deterministic tool-calling decisions; a fixed output token cap prevents runaway generation during structured output.
evaluator	qwen3:8b	0.3	2 048	—	Assessment nodes. Low temperature maintains a consistent scoring rubric across evaluations.
generator	gpt-oss:120b-cloud	0.4	4 096	—	Lecture generation. Moderate temperature balances creative expression with factual grounding.
worker	gemma3:1b	0.0	768	5 min	Lightweight intent classification. Smallest available model; minimal context and aggressive eviction keep resource cost negligible.

5.2.2 Configuration Management

All system configuration is expressed as Python `@dataclass` objects in `core/config.py`. Each subsystem has a dedicated configuration class — Neo for the knowledge graph database, `Mil_conf` for the vector store, Environment variables are read exactly once at module import time via `python-dotenv`, which loads values from a centralized `.env` file. The value are loaded once and binded into runtime via objects, guarantee consistent config setting and management. All `dataclass` fields default to sensible values if the corresponding variable is absent.

Table 5.4: *Configuration dataclasses.*

Class	Infrastructure	Key Parameters	Purpose
<i>Infrastructure Connection Configs</i>			
Neo	Neo4j	Connection URI, credentials, target database name	Graph database connection. Instantiated twice: once for the knowledge graph (test), once for the student learning graph (students).
Mil_conf	Milvus	Service URI, collection name, vector dimensionality (768), retry budget	Vector store connection for concept embedding storage and approximate nearest-neighbour similarity search.
Emb_conf	HuggingFace	Model identifier, vector dimensionality (768), maximum token length	Embedding model parameters for scientific text vectorisation (default: SciBERT).
Minio_conf	MinIO	Service endpoint, access credentials, TLS flag	Object storage connection for raw PDF course materials and extracted text chunks.
Mongo_conf	MongoDB	Host address, credentials, target database name	Document store connection for student state, chat history, and session persistence.
<i>Module Logic Configs</i>			
Ingest_param	Knowledge pipeline	Source data path, pages per textbook batch (10), pages per slide batch (15)	Controls PDF chunking granularity during course ingestion; batch sizes tune the trade-off between LLM context utilisation and extraction precision.
K_conf	LLM Engine	Extraction LLM profile name	Selects which model profile (see Table 5.3) handles knowledge graph extraction tasks.
TA_conf	TA pipeline	Agent name, system prompt, and output schema per agent	Declares the set of agents instantiated at startup; adding an entry is sufficient to register a new agent into the teaching workflow.
Config_Tracer	Langfuse	Enable flag, secret and public API keys, host URL	Opt-in observability configuration; all fields default to empty strings, disabling telemetry silently when credentials are absent.
<i>Domain Model</i>			
Bloom	Mastery scoring	Six cognitive levels: Remember (1) through Create (6)	Defines the Bloom's Taxonomy ordinal scale used to represent and advance a student's conceptual mastery.

5.3 Database Design

The system employs a polyglot persistence strategy, distributing data across five specialised storage engines. Each engine is selected for its optimal alignment with the data modality it serves, allowing distributed micro manageable infrastructure.

5.3.1 Polyglot Persistence Design

Table 5.5: *Polyglot persistence strategy overview.*

Engine	Data Type	System Role	Selection Rationale
Neo4j	Graph data	Knowledge Graph (entities, edges, clusters) and Student Learning Graph (mastery, progress)	Optimised for multi-hop traversal and recursive prerequisite queries; native Cypher query language enables expressive graph pattern matching.
Milvus	Vector data	768-dimensional SciBERT embeddings for semantic similarity search	Purpose-built for approximate nearest neighbour search using HNSW indexing; scales to millions of vectors with sub-millisecond query latency.
MongoDB	Document data	Student learning state, session context, chat memorisation, and tool result caching	Schema-flexible document store accommodating the variable structure of agent outputs; supports atomic field-level updates for concurrent session safety.
MinIO	Binary objects	Raw PDF course materials, extracted text chunks, and lecture assets	S3-compatible object storage decoupling binary files from the relational layer; erasure coding provides fault-tolerant parallel streaming.
SQLite	Relational data	Student authentication credentials and account metadata	Lightweight embedded database requiring zero configuration; enforces ACID properties for credential integrity without external service dependencies.

5.3.1.1 Neo4j: Knowledge and Learning Graphs

The system maintains two logically separated graph databases on the same Neo4j instance, preventing cross-contamination between educational content and student progress data.

The Knowledge Database stores the Educational Knowledge Graph constructed by the Knowledge Module. Entity nodes carry the properties defined in Table 5.13, connected by the five edge types (PART OF, RELATED TO, PREREQUISITE OF, CONTENT, ELABORATES ON). The database enforces uniqueness constraints on both node identifiers and names, and

maintains a full-text search index for efficient keyword-based retrieval. A dedicated index on the rhetorical role property enables the teaching workflow to perform role-filtered queries (e.g., retrieving only Definition-type nodes for explanatory responses).

The Student Database tracks individual learning progress. Each student is represented as a node connected to concept nodes via directed MASTERY edges. These edges carry a proficiency level (one through six, aligned with Bloom’s Taxonomy) and a timestamp indicating the last assessment. The teaching workflow’s Advance node writes to this database upon successful evaluation, creating new mastery edges or updating existing proficiency levels.

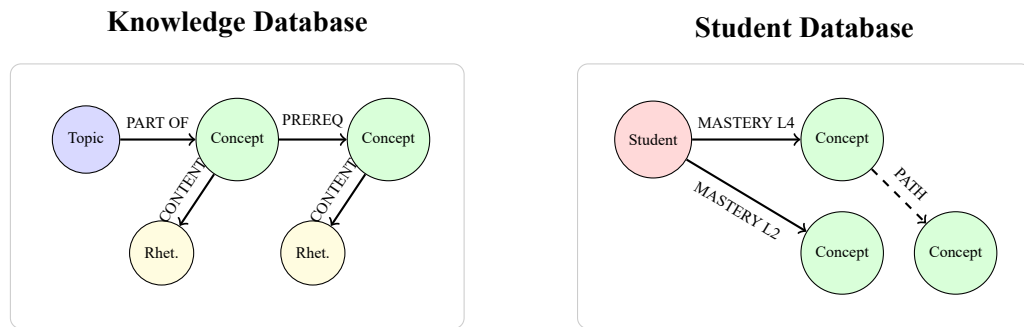


Figure 5.4: Neo4j dual-database schema. Knowledge and learning graphs separated.

5.3.1.2 Milvus: Vector Embedding Index

The Milvus vector database stores 768-dimensional SciBERT embeddings generated during the knowledge ingestion pipeline. Each record in the collection consists of three fields described in Table 5.6.

Table 5.6: Milvus collection schema.

Field	Data Type	Constraint	Description
id	VARCHAR	Primary Key	Unique entity identifier, shared with Neo4j for cross-database reference
text	VARCHAR	—	Concatenation of concept name and content, used as the embedding source text
embedding	FLOAT_VECTOR (768)	Indexed	SciBERT-generated dense vector for semantic similarity search

The collection employs a Hierarchical Navigable Small World (HNSW) index optimised for the L2 (Euclidean) distance metric. Table 5.7 details the index and search parameters.

Table 5.7: *HNSW index configuration.*

Parameter	Value	Purpose
M (bi-directional links)	8	Maximum number of neighbours per graph node during index construction
efConstruction	64	Search depth during index build; higher values improve recall at the cost of build time
ef (query-time)	64	Search depth during query; balances retrieval accuracy against latency
Metric Type	L2 (Euclidean)	Distance metric for approximate nearest-neighbour search

5.3.1.3 MongoDB: Student State and Session Context

MongoDB serves as the primary persistence layer for all student-related runtime data. Each student is represented as a single document keyed by their unique identifier, containing three major nested structures:

Table 5.8 provides the detailed field-level schema of the student document.

Table 5.8: *MongoDB student document field schema.*

Field Path	Type	Description
<i>state (Learning State)</i>		
finished_communities	Array	List of completed knowledge graph communities
current_pos	Object / null	Currently active concept node in the learning path
active_resource	String / null	Currently loaded material reference
mastery_map	Object	Key-value map of concept name to Bloom’s mastery level (0–6)
previous_nodes	Array	Ordered history of recently visited concept nodes
upcoming_nodes	Array	Queue of next recommended concepts from roadmap
summary	String	Compressed natural-language summary of learning progress
pending_proposal	Object / null	Unconfirmed learning path update awaiting student approval
<i>memo (Chat History) — nested by session_id</i>		
role	String	Speaker identifier: “student” or “TA”
heading	String	Topic heading for memo display
message	String	Full message content
timestamp	DateTime	Server-side UTC timestamp
<i>session_context — nested by session_id and chat_id</i>		
tool_results	Array	Cached tool execution outputs for within-request recall
thoughts	Array	Agent reasoning traces for debugging and observability

All write operations use atomic field-level updates (MongoDB \$set and \$push operators)

rather than full document replacement. This ensures that concurrent sessions belonging to the same student can safely update disjoint fields (e.g., one session updating the mastery map while another appends to the chat memo) without overwriting each other's changes.

MinIO provides S3-compatible object storage for all binary educational assets. All objects reside in a single bucket named “courses”. The storage hierarchy follows a deterministic three-level path convention detailed in Table 5.9, enabling the teaching workflow to reconstruct any object path programmatically from the concept's source reference property.

Table 5.9: *MinIO bucket directory structure.*

Path	Content Type	Description
courses/	Bucket root	Single top-level bucket containing all course data
{course}/{material}/{material}.pdf	application/pdf	Original raw PDF archived at ingestion time
{course}/{material}/chunks/{chunk_id}.txt	text/plain	Extracted text chunk stored individually for granular retrieval

The deterministic naming convention ensures that any component in the system can reconstruct an object path from two known values: the course name and the material name. The TA workflow exploits this property to serve PDF page references directly to the student frontend without querying a separate metadata index.

5.3.1.4 SQLite: Authentication Store

Student authentication is managed through a lightweight embedded SQLite database. Table 5.10 describes the schema of the single “students” table.

Table 5.10: *SQLite students table schema.*

Column	Type	Constraint	Description
id	TEXT	PRIMARY KEY	Unique student identifier
username	TEXT	UNIQUE	Login credential, enforced unique
name	TEXT	—	Student display name
email	TEXT	—	Contact email address
password	TEXT	—	SHA-256 hashed password
created_at	TIMESTAMP	DEFAULT NOW	Account creation timestamp

The deliberate choice of an embedded database eliminates external service dependencies for authentication while providing ACID guarantees for credential operations. Password hashing uses SHA-256 at the application layer before storage.

5.3.2 Cross-Database Reference and Consistency

Multiple system modules access to multiple database, and a lot of data of different source are aggregated and updated during run time. To stabilize this structure, two architectural paradigms govern how the five storage engines coexist without data duplication or race conditions.

Unique Key Cross-Database Reference

Entity identifiers serve as the primary cross-reference between Neo4j and Milvus: the same string identifier stored as a node property in Neo4j functions as the primary key in the Milvus collection, enabling the RAG agent to seamlessly translate between graph traversal results and vector similarity searches. When an agent performs a semantic search in Milvus, the returned identifiers are directly usable as lookup keys in Neo4j Cypher queries, eliminating the need for an intermediate translation layer.

Student identifiers bridge SQLite (authentication), MongoDB (learning state), and the Neo4j Student Database (mastery graph). A single login event propagates the authenticated identity consistently across all three stores: SQLite validates the credential and returns the student identifier, which MongoDB uses as its document primary key, and which Neo4j uses as the Student node property. This shared key ensures that mastery updates written by the TA workflow in Neo4j are immediately visible when the Student Tracker reads the learning state from MongoDB, without any synchronisation protocol beyond the identifier itself.

One-Write Policy

With consistent cross-database reference, modules can easily access many database, a dangerous convenience that may hinge race condition risk. To solve this, "One-Write Policy" is proposed: each database collection or table is assigned exactly one write-owning module, with strictly governed method. All other modules are restricted to read-only access. Table 5.11 enumerates the write ownership assignments.

Table 5.11: *Write ownership per database.*

Database	Collection	Writer	Consumers
Neo4j (Knowledge)	Entity nodes, edges	Knowledge Module	TA Module
Neo4j (Student)	Student, MASTERY edges	TA Module	Student Module
Milvus	Concept embeddings	Knowledge Module	TA Module
MongoDB	students collection	Student Module	TA Module
MongoDB	learning_logs collection	Student Module	TA Module
MinIO	courses bucket	Knowledge Module	TA Module
SQLite	students table	Student Module	—

This strict ownership eliminates the possibility of two modules concurrently writing to the same field. Combined with MongoDB's atomic field-level update operators, the policy ensures that even within a single write-owning module, concurrent sessions update disjoint document paths without overwriting each other's changes.

5.4 Student Module

5.4.1 Authentication: OAuth2 + JWT

The student module implements a standard OAuth2 password flow using PyJWT for JWT minting. The `/student/login` endpoint validates credentials against SHA-256 hashed passwords in SQLite and returns a short-lived JWT (`sub = student_id`). All subsequent requests attach this token as a Bearer credential; `get_current_student()` (a FastAPI dependency) decodes and validates it on each call.

5.4.2 Session Lifecycle and the Identity Firewall

The idea of this module is to initiate access to sensitive components and resources and enable logic modules only after login, via a controlled restricted manner. Moreover, it must guarantee an abstracted, hidden access that cannot be fully comprehend from application layer, but can only invoke via monitored request protocols.

The discussed paradigms goes into practice by a authentication and permission granted through a session lifecycle design described in Figure 5.5. The process goes by:

1. After login, the student calls `POST /student/session/start` with their JWT. The server generates a random UUID (`session_id`) and registers it in `Student_Tracker`.
2. All subsequent TA interactions supply only `session_id`. The TA Module never receives `student_id`.
3. Inside `Student_Tracker.resolve()`, the private mapping `session_id → student_id` translates transparently before any database call.

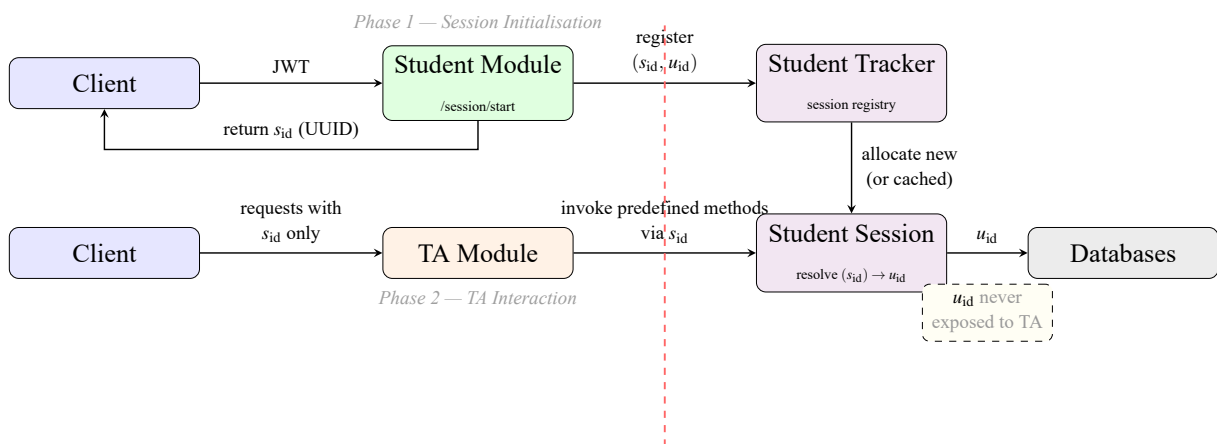


Figure 5.5: Login protocol with Identity Firewall (represented by the red line).

This pattern is analogous to a hotel key-card system: the key (`session_id`) grants access to resources, but revealing it does not expose the guest's credentials (`student_id`). Most importantly, all modules, tools can only gain abstracted access to student metadata can only be done once login. This prevent dangerous activity from AI or outside parties to sensible databases and resources.

As such, `Student Tracker` acts as a Singleton Manager that provide, cache and clean resources of unused Session. In the current implementation, it manages two distinct layers of state:

1. Session-Specific State (StudentSession): Keyed by session_id, this layer isolates chat history and session-specific context (like recent PDF pages). This prevents "Identity Race Conditions" where multiple browser tabs contaminate each other's chat logs.
2. Shared Learning State: Keyed by student_id, this layer caches the student's learning progress (current position, mastery map, etc.) and is shared across all active sessions of the same student.

To ensure data integrity during concurrent updates, the tracker utilizes Atomic Field Updates in MongoDB. Instead of overwriting the entire state object, the system updates specific fields (e.g., updating only the current_pos or appending to finished_communities). This design effectively mitigates "Data Race Conditions" where one session might accidentally revert progress made in another concurrent session.

5.5 Knowledge Module

The Knowledge Module operates as an offline administrative pipeline responsible for transforming raw educational documents into the structured Educational Knowledge Graph consumed by the Teaching Assistant at runtime. This module is accessible exclusively through a dedicated REST endpoint, invoked by administrators upon uploading new course materials.

5.5.1 Technical Setup

Table 5.12 summarises the principal libraries employed by the Knowledge Module. All dependencies are resolved through the uv package manager, ensuring deterministic, lockfile-driven builds across development and deployment environments.

Table 5.12: Knowledge Module core libraries.

Library	Role in Pipeline
Docling	PDF parser that converts each page into structured text blocks with provenance metadata such as page number and bounding region.
spaCy	Natural language processing library providing lemmatisation, synonym resolution, and stopword filtering for the post-extraction deduplication stage.
Pydantic	Schema validation framework that enforces strict structural contracts on every LLM extraction output, rejecting malformed responses before graph insertion.
SciBERT	Domain-specific sentence transformer (768 dimensions) trained on scientific corpora, generating embeddings that capture academic semantic similarity.

5.5.2 Ingestion Workflow

The ingestion pipeline supports two distinct material types—lecture slides and textbooks—each following a tailored processing path. Both paths share a common entry stage (document parsing and window segmentation as described in Section 4.2.2.1), but diverge at the knowledge extraction stage due to the fundamentally different structural roles these materials serve in the educational knowledge base.

To maximize throughput and minimize hardware idle time, both pipelines are engineered as fully asynchronous systems. This architectural choice prevents blocking operations during heavy I/O tasks or Large Language Model (LLM) inference, effectively turning a sequential bottleneck into a continuous stream of data processing.

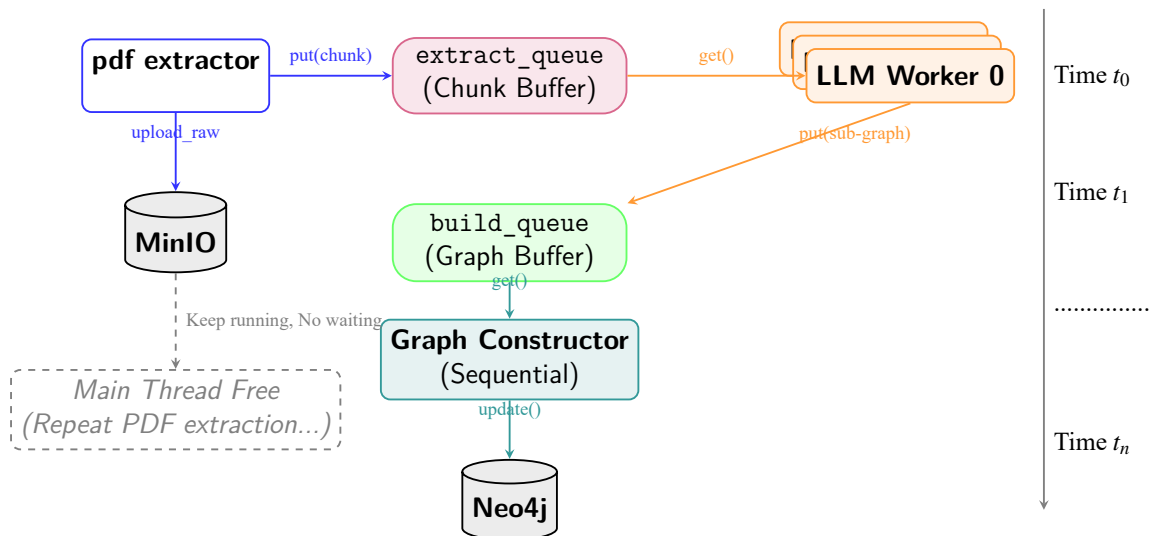


Figure 5.6: Asynchronous Ingestion Pipeline integrating Queues, Parallel Workers, and Storage

Lecture slides follow the primary extraction path. Because slide extraction involves complex, interdependent LLM operations and strict database insertion rules, it is architected as a Three-Station Producer-Consumer pipeline interconnected by bounded asynchronous queues. This design guarantees that data extraction happens continuously without being bottlenecked by slower downstream database operations.

- **Station 1 (Producer):** The operation on the main threads continuously parses the PDF document, chunking pages and immediately uploading the raw segments to MinIO. As soon as a chunk is ready, it is pushed into the Extraction Queue, then it leaves all the work to the background consumer and move the next document.
- **Station 2 (LLM Workers):** A pool of concurrent worker coroutines consumes items from the Extraction Queue. These workers operate independently, forwarding chunks to the LLM for the Dual-Phase Extraction (concept mapping and relation discovery). The asynchronous task are yielded to wait for the LLM result from cloud, costing little memory but effectively reduces inference waiting times. Once the LLM yields the structural representation, the result is forwarded to the Build Queue.
- **Station 3 (Graph Builder):** A single sequential getting extracted nodes, edges result from the Build Queue. It constructs the Knowledge Graph and ensures safe, transaction-free graph updates to Neo4j. Keeping this station single-threaded intentionally prevents race conditions and topological conflicts during the graph assembly phase, while the heavier workload (LLM inference) remains aggressively parallelized upstream.

Backpressure mechanisms are naturally enforced by setting maximum capacity limits on the interconnecting queues, ensuring that memory usage remains stable even if the PDF parsing station outpaces the LLM workers. Experiments shows that results are 30% faster with this approach.

5.5.3 Graph Schema at Database Level

The Knowledge Graph stored in Neo4j follows the formal Property Graph definition established in Section 4.2.1. Each entity node carries a set of standardised properties that support both structural navigation and semantic retrieval:

Property	Description
Name	Canonical concept label, enforced unique across the database
Identifier	Machine-generated unique key for programmatic reference
Content	Full textual description or definition of the concept
Node Type	Ontological category: Community, Topic, Concept, or Rhetorical
Rhetorical Role	Instructional function (Definition, Theorem, Application), assigned only to Rhetorical-type nodes
Cluster	Community identifier assigned during graph construction
Source Reference	Serialised pointer to the original material location in object storage

Table 5.13: *Entity node properties in Neo4j.*

Five directed edge types connect these entities: *PART OF* (hierarchical containment), *RELATED TO* (semantic association), *PREREQUISITE OF* (learning dependency), *CONTENT* (rhetorical elaboration from concept to its explanatory nodes), and *ELABORATES ON* (text-book enrichment). The database enforces structural integrity through four constraints: unique identifiers, unique names, a full-text search index across both fields, and a dedicated index on the rhetorical role property for role-filtered retrieval queries.

5.6 Teaching Assistant Module

The Teaching Assistant (TA) Module is the real-time component of the system, responsible for processing student queries and generating pedagogically grounded responses by orchestrating tool-calling agents across a state-driven workflow. Unlike the Knowledge Module which operates offline, the TA Module handles concurrent student sessions with sub-second latency requirements.

5.6.1 Technical Setup

Table 5.14: *TA Module core libraries.*

Library	Role in Module
LangGraph	Workflow graph builder providing typed state containers, conditional routing, and compiled sub-graph composition for deterministic agent orchestration.
LangChain	Agent abstraction layer supplying tool binding, middleware protocol, structured output parsing, and the ReAct execution loop.
Pydantic	Structured output schemas ensuring every agent response conforms to a predefined contract before reaching the student.
asyncio	Non-blocking concurrent execution enabling background persistence, parallel database writes, and real-time streaming without blocking the response pipeline.

In a conventional single-agent architecture of many framework including, the system prompt, context, and tools are fixed at initialization—forcing the model to simultaneously manage routing, retrieval, evaluation, and generation instructions, which degrades performance as task complexity grows.

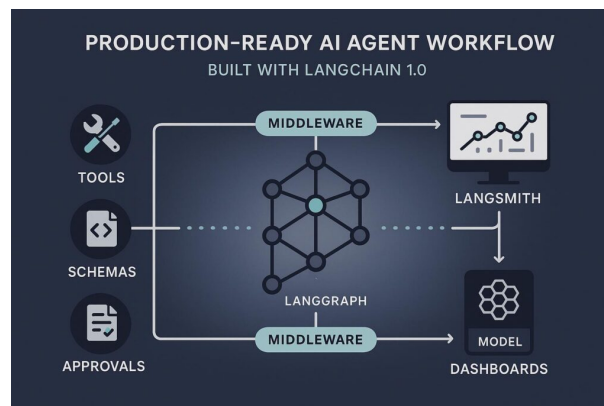


Figure 5.7: *Practice of Langchain Middleware Integration, inspired by [How LangChain 1.0 Simplifies Building Production-Ready LLM Applications](#)*

Langchain Middleware [9], released in December 2025, is one of the latest features providing a composable interceptor that modifies the model’s request before it reaches the language model, enabling a single agent to exhibit distinct behaviours per workflow node. Unlike role-based frameworks where changing an agent’s behaviour requires spawning a new instance entirely, the middleware approach eliminates redundant model connections while preserving per-node specialisation. The Figure 5.6.1 demonstrates the reference to the practice of LangChain Middleware Integration.

5.6.2 LangGraph Workflow Design

The TA Module’s orchestration layer is built as a LangGraph state machine where each node represents either a specialised processing step or a compiled sub-workflow. For each node design, the components within the schema orchestrate data flow through two primary mechanisms. As the agent state and student state traverse the nodes alongside runtime configurations, an exclusion mechanism is applied: the system filters the context so that each agent only sees the

information strictly necessary for its current task, keeping the prompt clean and preventing interference from irrelevant data.

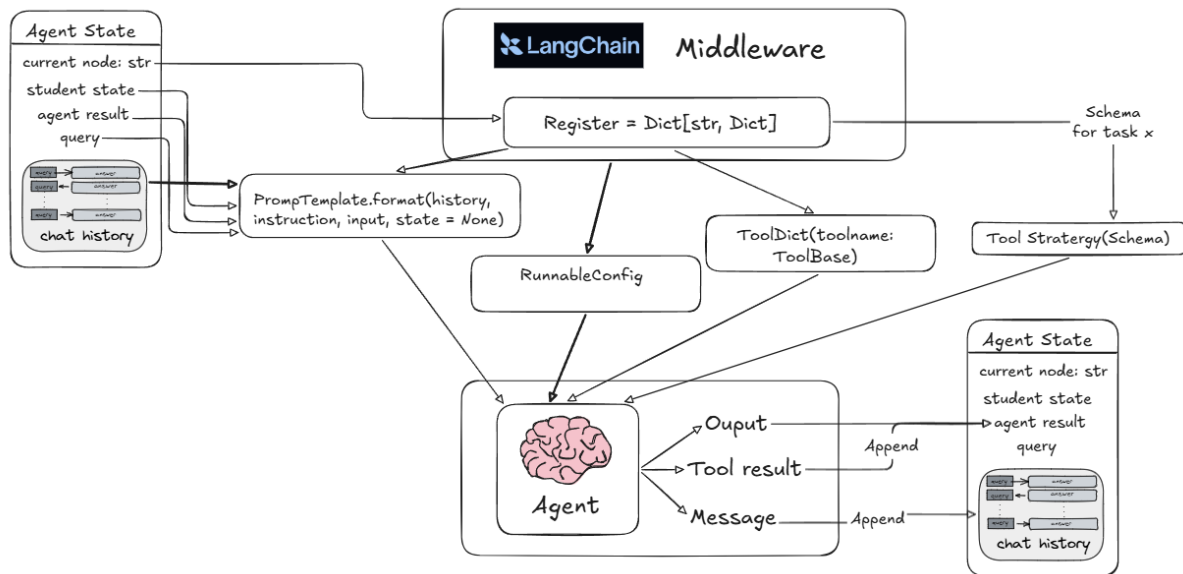


Figure 5.8: Standard operation of a LangGraph Node

AgentState: Representing the dynamic data payload and memory of the system.

- Query: Input from the user
- Agent result: Other Agents non message results accumulated from previous nodes in the workflow such as intermediate reasoning, tool results buffer.
- Chat history: An array containing the conversation history results.
- Current Node: name of the current node, used to find its preregistered means and resources in the Register

Middleware: Building upon the core middleware framework, the system applies the Adaptive Middleware pattern inherited from Langchain. My design ties its behaviour with the Register, which is a nested dictionary. The LangGraph Node name act as a key to access an inner dictionary containing the following items:

- Prompt Template: Created by LangChain PromptTemplate. Constructs AgentState and Query by combining predefined Meta Context and task instruction, forming a detailed instruction set to invoke agents. For TA, the Prompt template include chat history and optionally agent result
- Tools: Grouped according to task and registered during initialization.
- ToolStrategy: Override schema runtime during
- Runnable Config: Registered configurations that change agent behaviour according to the task. For example, enforcing low temperatures for rigid retrieval tasks.

The Middleware use the node name to get the dictionary, then pass it to the builtin Langchain call() method overriding agent calls, redefining agent behaviour and output. Agent output will append to Agent State as in the figure. This ensure state and context management across the whole workflow, while still preserve local context and agentic behaviour in that local setting.

5.6.3 Tracing and Observability

Agentic systems is an unstable and blackbox pipeline. Ensuring system in production requires comprehensive observability of agentic input, output and all intermediate step including reasoning chain to diagnose reasoning failures, measure latency bottlenecks, and audit the decision-making process. The Tracing Module captures a structured record of every agent interaction, providing both local persistence for debugging and long-term monitoring.

The tracing system organises observability data into a three-level hierarchy, illustrated in Figure 5.9. Each level captures progressively finer-grained information about the agent's execution.

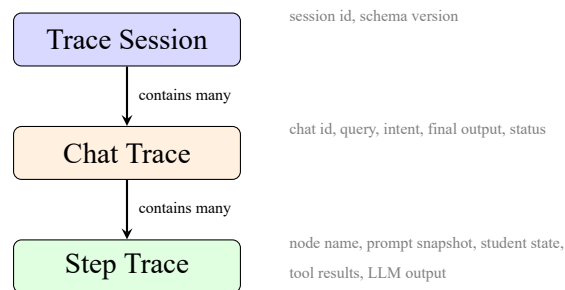


Figure 5.9: Trace data hierarchy. Three nested observation levels.

At the top level, a Trace Session spans the student's entire connected session and serves as the root container. Each student message creates a Chat Trace containing the original query, the classified intent, and the final response. Within each chat, Step Traces record the execution of each workflow node: the prompt to invoke language model (truncated for context window fit and storage efficiency), a snapshot of the AgentState at that moment containing student's learning state, any tool execution results, and the model's output.

Each active session maintains an in-memory trace buffer. As workflow nodes execute, each node appends a step record to the current chat's buffer. When a chat turn completes, the full session trace is serialised to JSON and written to a persistent file. The writer employs dual locking mechanisms—a threading lock for synchronous contexts and an asynchronous lock for coroutine contexts—ensuring thread safety when multiple workflow nodes attempt concurrent trace writes.

For production monitoring, the system integrates with Langfuse, a cloud-based observability platform built for LLM applications. The integration operates through LangChain's callback protocol: a Langfuse callback handler is injected into the runtime configuration at a session starts, automatically capturing every LLM invocation, tool call precisely from inner system.

Chapter 6

Experiment

Following the development phase, the core backbone engine has been successfully implemented. This chapter presents the operational results of the system architecture discussed previously. Specifically, each logical module exposes dedicated API endpoints to handle application requests. Furthermore, the underlying infrastructure has been robustly deployed, ensuring a seamless environment for the application to function.

6.1 Application Deployment

6.1.1 Database Setup

The initialization of Docker containers was executed successfully, establishing the foundational database infrastructure.

 minio-core minio/minio:latest 9000:9000  Show all ports (2)	  	<pre>cd:2379"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.942 Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable" ,"variable-name":"ETCD_AUTO_COMPACTION_MODE","variable-value":"revision"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.942 Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable" ,"variable-name":"ETCD_AUTO_COMPACTION_RETENTION","variable-value":"1"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.942 Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable" ,"variable-name":"ETCD_LISTEN_CLIENT_URLS","variable-value":"http://0.0.0.0:2379 "} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.942 Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable" ,"variable-name":"ETCD_QUOTA_BACKEND_BYTES","variable-value":"4294967296"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.942 Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable" ,"variable-name":"ETCD_SNAPSHOT_COUNT","variable-value":"50000"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.943 Z","caller":"etcdmain/etcd.go:73","msg":"Running: ", "args":["/usr/local/bin/etcd "]} 2026-05-19 23:33:04 milvus-etcd {"level":"warn","ts":"2026-05-19T16:33:04.943 Z","caller":"etcdmain/etcd.go:105","msg":"'data-dir' was empty; using default"," data-dir":"default.etcd"} 2026-05-19 23:33:04 milvus-etcd {"level":"info","ts":"2026-05-19T16:33:04.944 Z","caller":"etcdmain/etcd.go:116","msg":"server has been already initialized","</pre>
 mysql-core mysql:8.0 3307:3306 	  	
 neo4j-core neo4j:2025.12.1-ente 7474:7474  Show all ports (2)	  	
 milvus-attu zilliz/attu:latest 8000:3000 	  	
 milvus-etcd coreos/etcd:v3.5.5 	  	

Figure 6.1: Docker-base infrastructure containerization and deployment

Additionally, the system provides comprehensive database monitoring tools for all data containers, facilitating efficient debugging and direct tool testing.

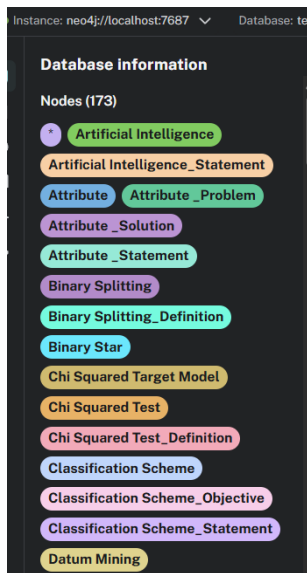


Figure 6.2: Neo4j Monitor Plugins

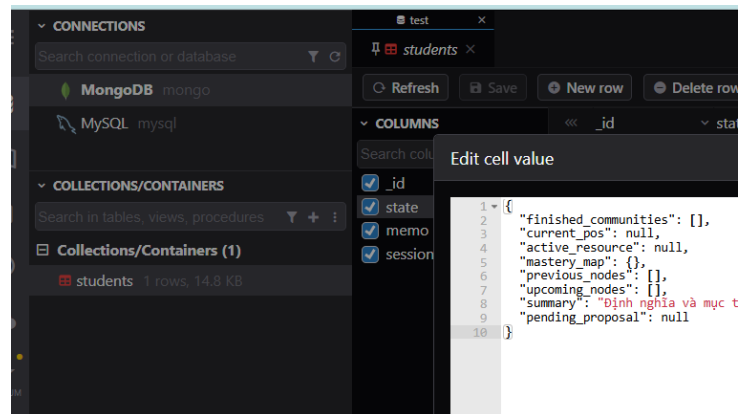


Figure 6.3: DBgate monitoring both MongoDB and SQLite

Figure 6.4: Database Admin Monitoring and Visualizing

6.1.2 FastAPI App Deployment

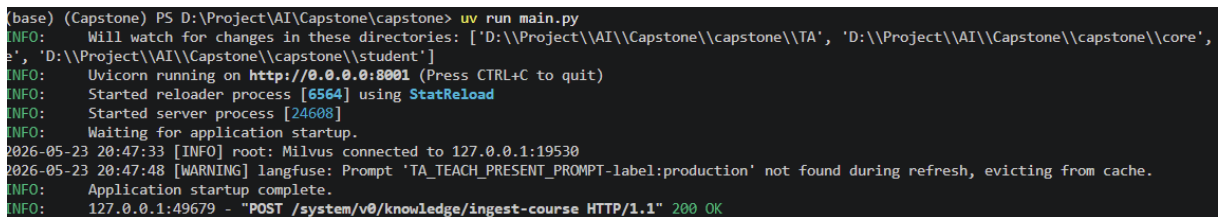


Figure 6.5: FastAPI successful lifecycle start

The App is hosted via uvicorn. The lifespan create connection pool, initialize Singleton Modules and listen on port 8001. The Figure 6.5 shows that all deployments are success and endpoints of main components have been publicly exposed for usage.

6.2 Knowledge Module

6.2.1 Service Endpoints

- POST /ingest-course
 - Use case: Enqueues a course ingestion request, validates the presence of uploaded PDF documents, and triggers the background processing pipeline to extract semantic concepts and construct the Knowledge Graph.
 - Input schema:
 - * course_name (String): The title of the course being ingested.
 - * slide_files (List of Strings): A list of slide file names to be processed.

- * textbook_files (List of Strings): A list of textbook file names to be processed.
- * reset (Boolean): A flag indicating whether to reset the existing course data before proceeding.
- Output schema:
 - * status (String): The acceptance status of the background task.
 - * message (String): A descriptive confirmation message.
 - * details (Object): Additional metrics, including the count of slides and textbooks queued.

6.2.2 Runtime

During execution, the client transmits documents via an HTTP stream of bytes, which are subsequently loaded as PDF files.

```
2026-05-23 20:50:53 [INFO] docling.models.stages.ocr.auto_ocr_model: Auto OCR model selected rapidocr with torch.
2026-05-23 20:50:53 [INFO] docling.utils.accelerator_utils: Accelerator device: 'cuda:0'
2026-05-23 20:50:55 [INFO] docling.utils.accelerator_utils: Accelerator device: 'cuda:0'
2026-05-23 20:50:56 [INFO] docling.pipeline.base_pipeline: Processing document Chapter 2 - Decision Tree.pdf
2026-05-23 20:51:32 [INFO] docling.document_converter: Finished converting document Chapter 2 - Decision Tree.pdf in 40.22 sec.
=====
Extraction completed in 40.24s. Created 4 batch chunks.
[Worker-1] Chunk 1 done in 56.8s
[Builder] Built chunk 1 (Attribute
), total: 1
[Worker-2] Chunk 2 done in 64.1s
[Worker-2] Poison pill received.
[Builder] Built chunk 2 (Decision Tree
), total: 2
[Worker-0] Chunk 0 done in 76.6s
[Worker-0] Poison pill received.
[Builder] Built chunk 0 (Decision Tree Learning), total: 3
[Worker-1] Chunk 3 done in 62.4s
[Worker-1] Poison pill received.
[Builder] Built chunk 3 (Decision Tree Subtree Replacement
```

Figure 6.6: *Tracking Log of Asynchronous Ingestion*

Those file goes through an asynchronous, long extraction process to formulating normalized filtered nodes and edges.

```
"Q16766476": {
  "id": "Q16766476",
  "name": "Decision Tree Learning",
  "content": "machine learning algorithm",
  "hard_ref": {
    "db": "minio",
    "id": "minio://courses/ML/ML/Chapter 2 - Decision Tree/chunks/c2af088090067b83919688f1c92cefce_slide0.txt",
    "name": "decision tree part 1",
    "summary": "Machine Learning Decision Tree Contents Decision-Tree Learning Decision-Trees Decision-Tree Learning .....",
    "p_num": [
      1,
      15
    ]
  }
}
```

Annotations in the image:

- Unique Index from DBpedia as Concept ID (points to "id": "Q16766476")
- Concept name (normalized token) (points to "name": "Decision Tree Learning")
- Minio Path for full context storage and retrieval (points to "id": "minio://courses/ML/ML/Chapter 2 - Decision Tree/chunks/c2af088090067b83919688f1c92cefce_slide0.txt")
- Page range (points to "p_num": [1, 15])

Figure 6.7: *Extracted Concept Nodes*

All those valuable Artifacts are pushed and saved on Neo4J.

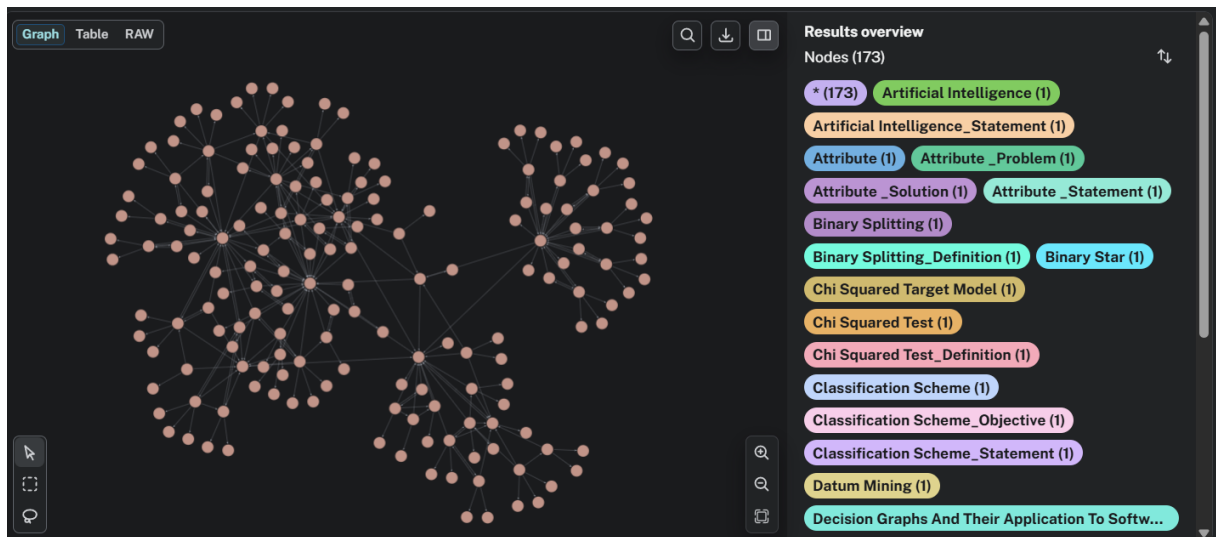


Figure 6.8: Completed Knowledge Graph demonstrating no concept fragmentation

However, the external knowledge base, DBpedia, occasionally provides inconsistent name and definitions. This manifests as words receiving inaccurate definitions or unnormalized representations.



Figure 6.9: DBpedia accepting unusual or unnormalized node names

Furthermore, we observed the following operational constraints:

- An overly detailed Knowledge Graph becomes challenging to visualize, especially when exceeding 100 nodes and generating over 300 relationships per 20 pages.
- The ingestion process requires approximately 250 seconds per 50 pages, even when utilizing 3 asynchronous task workers.

6.3 Student Module

6.3.1 Service Endpoints

The Student Module exposes several key endpoints for identity and session management:

- POST /register
 - Use case: Registers new student profiles, hashes credentials, and initializes a default state document.

- Input schema:
 - * `student_id` (String): The unique identifier chosen by the student.
 - * `password` (String): The plain-text password to be securely hashed.
- Output schema:
 - * `detail` (String): Confirmation message of successful registration.
- `POST /login`
 - Use case: Validates student credentials and generates a secure JWT authentication payload.
 - Input schema:
 - * `username` (String): The student's unique identifier.
 - * `password` (String): The student's password.
 - Output schema:
 - * `access_token` (String): The JWT payload used for subsequent authenticated requests.
 - * `token_type` (String): The token category, typically set to "bearer".
- `POST /session/start`
 - Use case: Generates a session-specific UUID token to isolate chat interactions.
 - Input schema:
 - * `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
 - Output schema:
 - * `session_id` (String): A uniquely generated UUID functioning as an opaque token for a specific chat instance.
- `DELETE /session/end`
 - Use case: Discards the active session token to cleanly terminate the session.
 - Input schema:
 - * `session_id` (Query String): The UUID of the session to terminate.
 - * `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
 - Output schema: Returns HTTP 204 (No Content) indicating successful deletion.

6.3.2 Runtime

The system demonstrates the capability to robustly handle user sessions, including detecting pre-existing users during registration or login processes.

```
INFO: 127.0.0.1:56103 - "POST /system/v0/student/register HTTP/1.1" 409 Conflict
INFO: 127.0.0.1:56103 - "POST /system/v0/student/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:56103 - "POST /system/v0/student/session/start HTTP/1.1" 200 OK
2026-05-23 22:39:50 [INFO] TA.api.route: TA task 3a7f100c-0c53-4bba-9db2-814297925483 scheduled for session d6ccf424-6b72-4851-acba-a1806685285b.
```

Figure 6.10: Status 409 indicating a pre-existing student ID entry in the database. A new Session ID and access credentials are concurrently created.

6.4 Teaching Module

6.4.1 Service Endpoints

- POST /chat
 - Use case: Serves as the primary tutoring interface. It receives user queries alongside their active session IDs, subsequently executing the multi-agent LangGraph tutoring pipelines to formulate contextual responses.
 - Input schema:
 - * session_id (String): The UUID for the active chat session authenticated by the user.
 - * user_input (String): The actual query or message submitted by the student.
 - * language (String): The language preference toggle (e.g., "vn" or "eng").
 - Output schema:
 - * task_id (String): A unique identifier assigned to track the asynchronous background task.
 - * status (String): The initial state, typically indicating receipt of the request.
 - * agent (String): The default agent assigned to process the workflow.
- GET /chat/status/{task_id}
 - Use case: Sends system status back to the client while waiting. The streaming output can be included here as well.
 - Input schema:
 - * task_id (Path String): The unique identifier of the background task being polled.
 - Output schema:
 - * status (String): The current operational state (e.g., "working", "finished", or "Fail").
 - * agent_name, intent, thought (String, optional): Metadata illustrating the agent's current step, intended action, and reasoning.
 - * result (Object, optional): Available upon completion, containing the final message string and optional ui_action dictionaries.
 - * error (String, optional): Descriptive failure logs in the event of pipeline exceptions.

6.4.2 Runtime

Upon submitting a query, the client interface consistently monitors the agentic progress, displaying the current status and the active agent. Once the pipeline concludes, the service transmits the final text result back to the client.

```

Login Successful
Submitting query to http://127.0.0.1:8001/system/v0/ta/chat...
Task accepted - id=3a7f100c-0c53-4bba-9db2-814297925483, status=received, agent=TA
Waiting 2 seconds before first status check...
[ 2s] status-working | agent=TA_Router | intent=retrieve
[ 15s] status-working | agent=TA_Router | intent=retrieve
[ 32s] status-working | agent=TA_Router | intent=retrieve
[ 47s] status-working | agent=TA_Router | intent=retrieve
[ 62s] status-finished | agent=TA_Retrieve_Finish | intent=retrieve

--- TA Response ---
# Định nghĩa và mục tiêu của **Học máy** (Machine Learning) - **Học máy**

- **Học máy** (Machine Learning) là một **lĩnh vực nghiên cứu** (field of study) tập trung vào việc xây dựng các **chương trình** (programs) có khả năng **tự động cải thiện** (automatically improving) thông qua **kinh nghiệm** (experience).
- Mục tiêu **chính** (primary objective) của **Học máy** là **tăng cường hiệu suất** (enhance performance) trên các **nhiệm vụ** (tasks) bằng cách **học** (learning) từ **kinh nghiệm** và **đánh giá** (using a defined performance measure) dựa trên **độ đo hiệu suất** (performance measure).

## Gợi ý học tập tiếp theo
Nếu bạn muốn đi sâu hơn, hãy khám phá các **khái niệm nền tảng** như:
- **Học có giám sát** (Supervised Learning) - **Học có giám sát**
- **Học không giám sát** (Unsupervised Learning) - **Học không giám sát**
- **Học tăng cường** (Reinforcement Learning) - **Học tăng cường**

Bạn có thể bắt đầu bằng cách đọc các tài liệu giới thiệu về **Học có giám sát** để nắm vững cách **Mô hình** (models) học từ **dữ liệu gán nhãn** (labeled data).

Result saved to: test/TA/TA_v0_223818_2305.json
Time taken for question 1 [What is machine Learning]:
65.08477306365967

```

Figure 6.11: Client-side interface displaying the agentic interaction flow

Simultaneously, the system persistently logs and saves the outcomes of agentic tool executions and messages, facilitating potential reusability and analytical reviews.

```

EXAMPLES:
- Query: "Tôi mới bắt đầu, nên học gì?" -> recommend_new() (global)
- Query: "Học Deep Learning" -> recommend_new(course_filter="Deep Learning")

"""
    ),
    AIMessage(
        content="",
        tool_calls=[
            {
                "name": "recommend_new",
                "args": {
                    "course_filter": "Machine Learning"
                }
            }
        ]
    ),
    ToolMessage(
        name="recommend_new",
        content=""
    )
]

TOP 10 HUB CONCEPTS (by connectivity):

1. [Topic] Decision Tree Learning
- course: Machine Learning
- connections: 20
- mastery: 0
- desc: machine learning algorithm

2. [Concept] Decision Support System
- course: Machine Learning
- connections: 18
- mastery: 0
- desc: system that supports business or organizational decisions

3. [Concept] Reduced Error Pruning
- course: Machine Learning
- connections: 12
- mastery: 0
- desc: pruning using hold-out validation

4. [Concept] Heraldic Attribute
- course: Machine Learning

```

```

AIMessage(
    content=""
)

Dựa trên knowledge graph, các hub node phù hợp để bắt đầu học Machine Learning gồm:

1. Decision Tree Learning
- Thuật toán ML nền tảng dùng cho classification/regression.

2. Supervised Learning
- Paradigm quan trọng nhất cho người mới bắt đầu.

3. Inductive Bias
- Hiểu assumptions của model khi học dữ liệu.

4. Hold Out Set
- Khái niệm validation/test trong training pipeline.

5. Information Gain Ratio
- Metric quan trọng trong Decision Tree.

Đây là các node có độ kết nối cao, phù hợp làm entry point cho roadmap học Machine Learning.
"""

```

Figure 6.12: 2 snippet of Logged Agent Workflow result

Currently, the core teaching workflow remains under active development, while the retrieval and roadmap workflows have demonstrated promising initial results. Moving forward, comprehensive benchmarks and advanced evaluation metrics will be investigated to formally assess the system's efficiency in automated, intelligent education expertise.

Chapter 7

Conclusion

7.1 Achieved Results

The integration of the SmartEdu system has successfully demonstrated a working, agent-driven educational platform. By focusing on practical functionality and stable outputs, the system has achieved the following results:

- **Unified Knowledge Graph:** The system automatically builds a connected and non-fragmented Knowledge Graph. Concepts remain consistent throughout the course, providing a reliable database for all agent tasks.
- **Consistent AI Operations:** Guided by state-based harnessing, the multi-agent system runs stably. It reliably retrieves data, maps out learning roadmaps, and identifies backbone concepts without hallucinating or drifting off-topic.
- **Scalable System Design:** The backend architecture is modular and separates state management from AI processing, making it ready to scale into a microservices environment.
- **Multimodal Input Processing:** The platform successfully preserves the structural layouts and visual references of the original materials, allowing agents to "read" and utilize this rich context for better answers.
- **Reliable Error Handling:** The system includes a robust fallback mechanism. When specific context is missing, the agents safely switch to exploring the broader graph instead of generating fake information.

7.2 System Limitations

Despite its functional success, the current version of the system has several practical limitations that affect its speed and long-term usability:

- **Unclear Course-Level Paths:** Defining prerequisites for an entire course is still difficult. The exact threshold for deciding when to merge overlapping topics remains undecided, which can cause confusion in the macro-roadmap.
- **Lack of Self-Improvement:** The system's intelligence comes from static, specialized prompts (the harness), not the models themselves. The agents cannot learn from their own mistakes, meaning the system is not yet truly adaptive.

- **High Latency:** Processing time is a major bottleneck. A simple data retrieval takes up to 30 seconds, while complex tasks like generating a roadmap or a teaching response can take up to 60 seconds.
- **Limited Long-Term Tracking:** The context from which Teaching Assistant assesses situation are still tied to the present chat session. It currently lacks a reliable way to track and evaluate a student's progress over a long period.
- **Incomplete Application:** The frontend interface is not developed, and the application is still not hosted, limiting how users can visually interact with the system and their roadmaps.

7.3 Future Plan

To prepare the SmartEdu system for final deployment and defense, the remaining work is scheduled into an 8-week execution plan. This focuses on visual interfaces, speed optimization, and final testing:

1. **Weeks 1: Completion of Agentic Design on a Macro Scale** Improve agentic module performance for more challenging cases. Propose reliable metrics.
2. **Weeks 2: Interactive UI/UX Design.** Build the frontend interface and test system deployments. Provide additional features:
 - Let students visually navigate the Knowledge Graph, view their roadmaps dynamically, and chat easily with the system.
 - Streamline output to reduce response latency, display original PDF file with highlighter for deeper knowledge comprehension
3. **Weeks 3–4: Student Tracking and Exercises.** Add interactive exercises and a reliable scoring and evaluation system for student. Using these academic records, the system will apply clustering algorithms to group students with similar learning patterns for better long-term tracking.
4. **Weeks 5–6: Optimization of Agent components on a Micro Scale** Improve Agentic Memory, Algorithm construction and speed up inference. This step aims to cut down the current 30–60, while still improving quality.
5. **Weeks 7–8: System Benchmarking and Final Report.** Run practical tests to evaluate the TA's accuracy and its impact on students. Finally, summarize all results into the graduation thesis and prepare the visual presentation for the defense.

Bibliography

- [1] Qurat Ul Ain, Mohamed Amine Chatti, Jean Qussa, Amr Shakhshir, Rawaa Alatrash, et al. The edukgpipeline: An optimized pipeline for automatic educational knowledge graph construction. *arXiv preprint arXiv:2509.05392*, 2025.
- [2] Sydney Anuyah, Mehedi Mahmud Kaushik, Krishna Dwarampudi, Rakesh Shiradkar, Arjan Durrezi, and Sunandan Chakraborty. Automated knowledge graph construction using large language models and sentence complexity modelling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [3] Joao T. Aparicio, Elisabete Arsenio, Francisco Santos, and Rui Henriques. Using dynamic knowledge graphs to detect emerging communities of knowledge. *Knowledge-Based Systems*, 2024.
- [4] Zhendong Chu, Shen Wang, Jian Xie, Tinghui Zhu, Yibo Yan, Jinheng Ye, et al. Llm agents for education: Advances and applications. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [5] Docker. Docker overview, 2023.
- [6] Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can llms be good graph judge for knowledge graph construction?, 2025.
- [7] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. Stanford University, 3rd edition, 2026. Online manuscript released January 6, 2026.
- [8] LangChain Inc. Langchain documentation. <https://docs.langchain.com/>, 2022. Accessed: 2022-05-24.
- [9] LangChain Inc. Agent middleware documentation. <https://www.langchain.com/blog/agent-middleware/>, 2025. Accessed: 2025-12-05.
- [10] Yen-Ting Lee, Keerthi Koneru, Zahra Moslemi, Sheethal Kumar, and Ramesh Radhakrishnan. Aema: Verifiable evaluation framework for trustworthy and controlled agentic llm systems. *arXiv preprint arXiv:2601.11903*, 2026.
- [11] Chen Liang, Jianbo Ye, Zhaohui Wu, Bart Pursel, and C Lee Giles. Recovering concept prerequisite relations from university course dependencies. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
- [12] Chenxi Liu, Yongqiang Chen, Tongliang Liu, James Cheng, Bo Han, and Kun Zhang. On the thinking-language modeling gap in large language models. *arXiv preprint arXiv:2505.12896*, 2025.

- [13] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [14] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, 2024.
- [15] Congmin Min, Sahil Bansal, Joyce Pan, Abbas Keshavarzi, Rhea Mathew, and Amar Viswanathan Kannan. Towards practical graphrag: Efficient knowledge graph construction and hybrid retrieval at scale. *arXiv:2507.03226*, 2025.
- [16] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. Kggen: Extracting knowledge graphs from plain text with language models, 2025.
- [17] Davy Tsz Kit Ng, Eagle Kai Chi Chan, and Chung Kwan Lo. Opportunities, challenges and school strategies for integrating generative ai in education. *Computers and Education: Artificial Intelligence*, 8:100373, 2025.
- [18] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [19] Liangming Pan, Chengjiang Li, Juanzi Li, and Jie Tang. Prerequisite relation learning for concepts in moocs. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017.
- [20] Simplilearn. What is aws? an ultimate guide to amazon web services, 2023.
- [21] Sharon Solomon. Frontegg - what is oauth? definition, how it works, and tips for success, 2023.
- [22] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9, 2019.
- [23] Jing Wang, Zheng Li, Lei Li, Fan He, Liyu Lin, Yao Lai, Yan Li, Xiaoyang Zeng, and Yufeng Guo. Principle-guided verilog optimization: Ip-safe knowledge transfer via local-cloud collaboration. *10.48550/arXiv.2508.05675*, 2025.
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [25] Orion Weller, Michael Boratko, Iftexhar Naim, and Jinhyuk Lee. On the theoretical limitations of embedding-based retrieval. *arXiv preprint arXiv:2508.21038*, 2025.
- [26] C. M. Wyss. How to tame your llm: Semantic collapse in continuous systems. *arXiv preprint arXiv:2512.05162*, 2025.
- [27] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, Miami, Florida, USA, 2024. Association for Computational Linguistics.

- [28] B. Zhao, J. Sun, B. Xu, X. Lu, Y. Li, J. Yu, M. Liu, T. Zhang, Q. Chen, H. Li, et al. Edukg: a heterogeneous sustainable k-12 educational knowledge graph. *arXiv preprint arXiv:2210.12228*, 2022.